Software Systems Verification and Validation

Vescan Andreea, PhD, Assoc. Prof. Habil.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Outline

- Verification and Validation
- Software development life cycle Model
  - V-Model
  - Extended V-Model [CB03]
- Quality
  - Quality control vs. Quality Assurance
  - Quality definitions
- What is a bug?
  - First bug
  - Terms for software failures
  - Software error (or bug)
  - When? Why? Cost?
  - Famous Software bugs

Verification and Validation (SEI and NASA) [NTO5],[PYO8]

Software Engineering Institute

- **Verification**
  - assures the product is developed according to requirements, specifications and standards.
  - building the product correctly.
  - Are we building the product right?
- **Validation**
  - assures that the product will be usable on the market.
  - building the correct product.
  - Are we building the right product?

NASA - Software Assurance Guidebook and Standard [NAS]

- Verification and Validation
  - the process that assures that the software product:
    - every step in the product development is resulting in a correct (sub)product = **verification**.
    - will satisfy the requirement (functional and others) = **validation**.

Verification and Validation - comparison

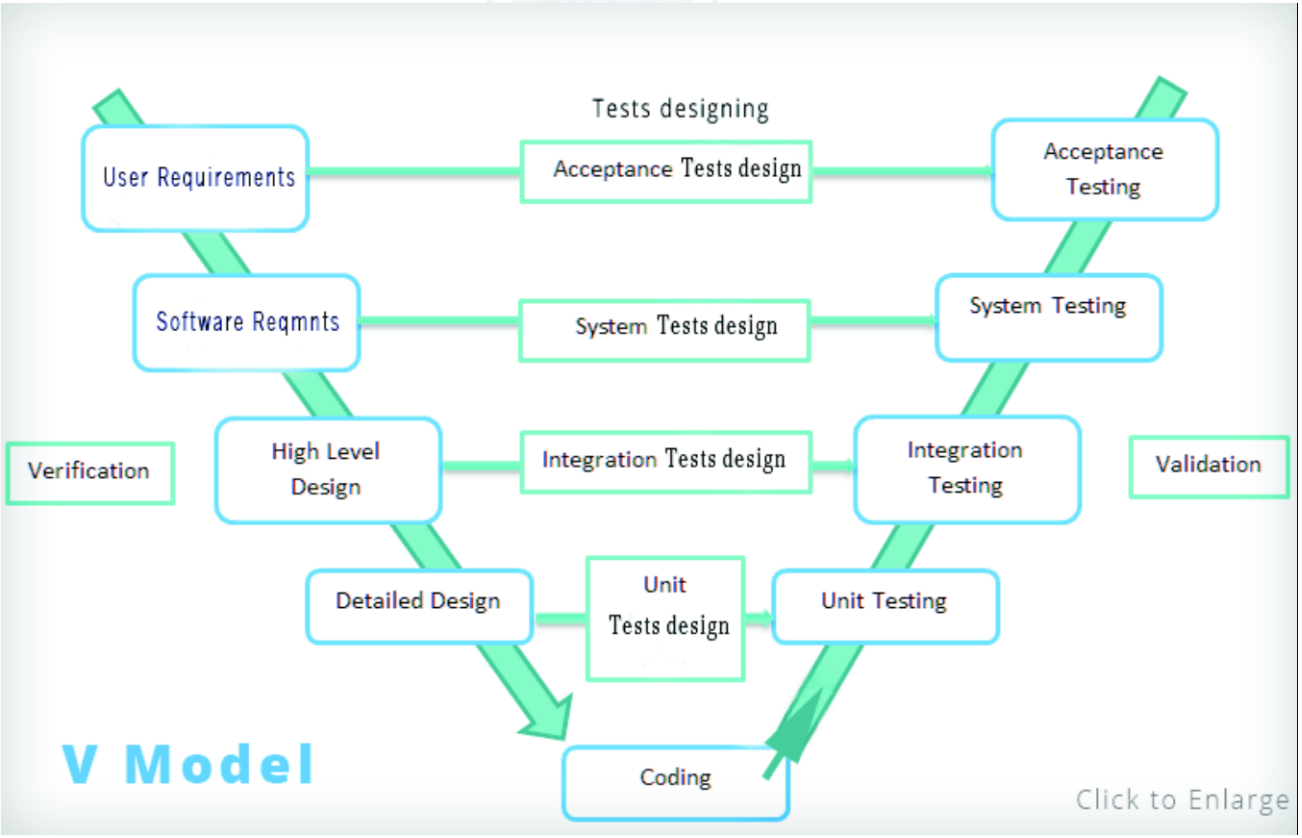
Verification	Validation
• evaluates if the product of a given development phase satisfies the requirements of that phase; • reviews products to ensure their quality (consistency, completeness, correctness); • static and dynamic analysis techniques.	• helps us at confirming that a product meets its intended use. • is performed toward the end of the system development to determine if the entire system meets the customer's needs and expectations; • is performed on the entire system by actually running the system in its real environment and using a variety of tests.

Outline

- Verification and Validation
- Software development life cycle Model
  - V-Model
  - Extended V-Model [CB03]
- Quality
  - Quality control vs. Quality Assurance
  - Quality definitions
- What is a bug?
  - First bug
  - Terms for software failures
  - Software error (or bug)
  - When? Why? Cost?
  - Famous Software bugs

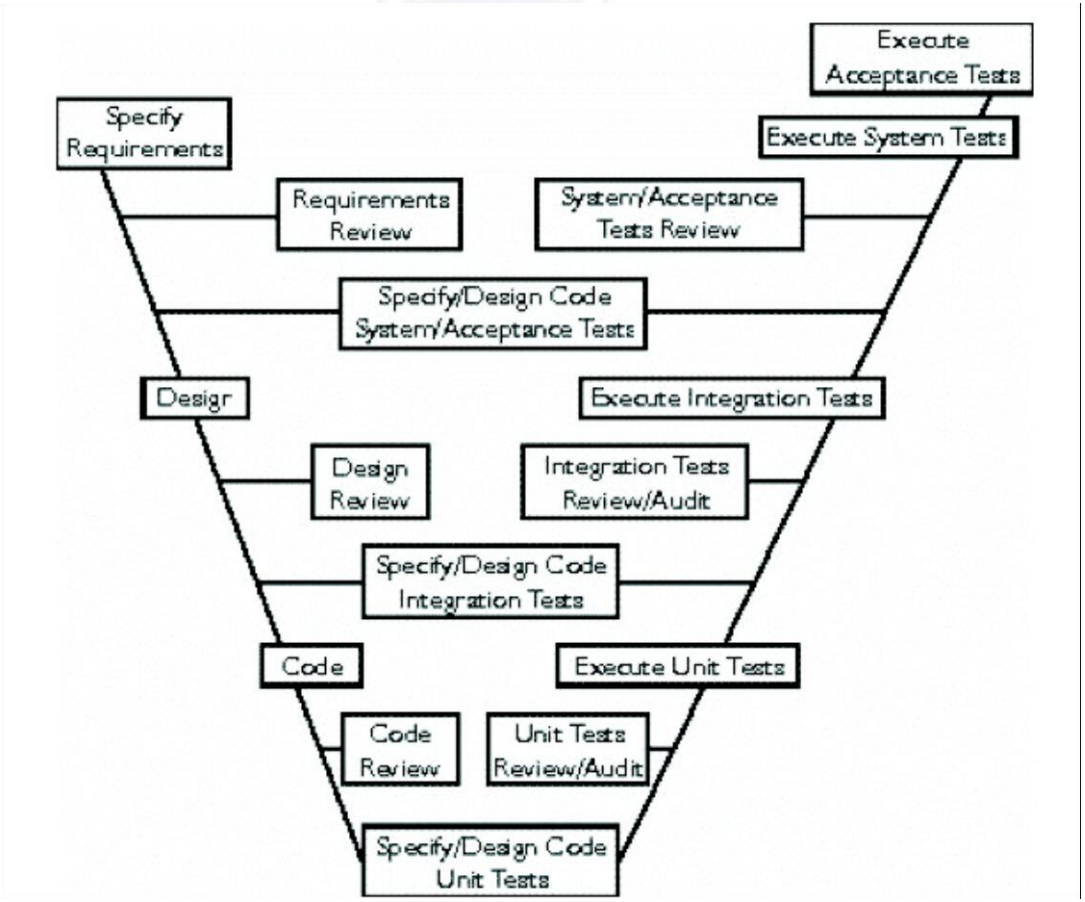
Software development life cycle Model

- V-Model



Software development life cycle Model

- Extended V-Model



Outline

- Verification and Validation
- Software development life cycle Model
  - V-Model
  - Extended V-Model [CB03]
- Quality
  - Quality control vs. Quality Assurance
  - Quality definitions
- What is a bug?
  - First bug
  - Terms for software failures
  - Software error (or bug)
  - When? Why? Cost?
  - Famous Software bugs

Quality control vs. Quality Assurance

Quality control	Quality Assurance
• QC = Quality of products • How do you control the quality of the work you have done? • Goal: detect problems in the work products	• QA = Quality of processes • How do you assure the quality of the work you are going to do? • Goal: Ensure adherence to processes, standards and plans

Quality definitions [BBST]

"Quality is conformance"

"Software quality: Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software." [Pre00]

"Quality is free." by Phil Crosby [Cro80]

conformance with requirements, i.e. conformance to the user's actual requirements which may or may not be written down in a specification.

Quality - conformance to the needs and not to documents.

"Quality - fitness for use." by Joseph Juran [JJ98]

satisfiers - anything that makes you like the application

dissatisfiers - anything that makes you like the application less.

Quality – according to who? (Program manager, Programmer, Tech writer, Tester)

"Quality - is value to some person." by Jerry Weinberg [Wei92]

quality is subjective - what's valuable for you may not be so valuable for me.

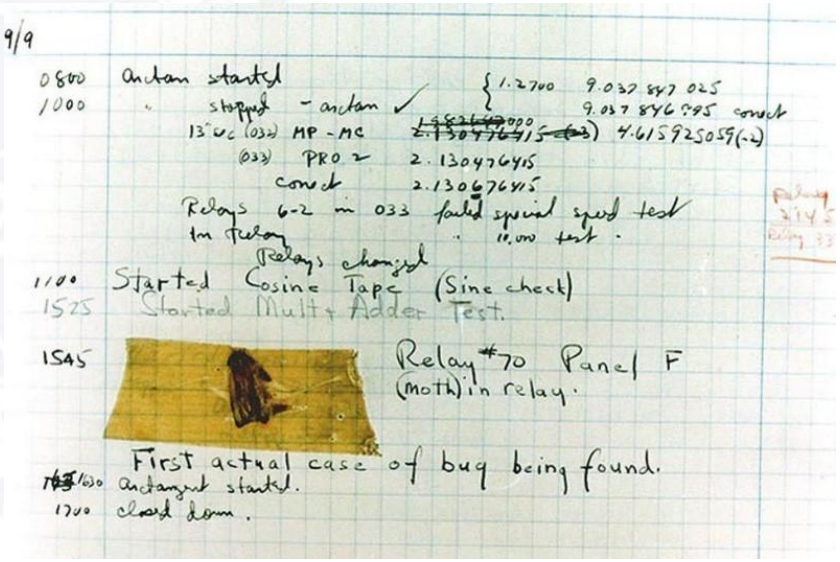
Outline

- Verification and Validation
- Software development life cycle Model
  - V-Model
  - Extended V-Model [CB03]
- Quality
  - Quality control vs. Quality Assurance
  - Quality definitions
- What is a bug?
  - First bug
  - Terms for software failures
  - Software error (or bug)
  - When? Why? Cost?
  - Famous Software bugs

What is a bug?

- First bug
  - Grace Hopper - About first software bug: (2'10")
    https://www.youtube.com/watch?v=lQS0hDqpVLE
  - 1947 - Harvard University - Mark II

- The first computer bug was born!



Well, okay, it died!

Terms for software failures [Pat05]

- **Failure**
  - A failure is said to occur whenever the external behavior of a system does not conform to that prescribed in the system specification.
- **Error**
  - An error is a state of the system. In the absence of any corrective action by the system, an error state could lead to a failure which would not be attributed to any event subsequent to the error.
- **Fault**
  - A fault is the adjudged cause of an error.
- Process of failure manifestation - represented as a behavior chain:

  fault → error → failure.

Software error (or bug)

- A bug is an aspect of the product that causes an unnecessary or unreasonable reduction in the quality of the product.
  - design weaknesses, documentation error, usability annoyances

Remark. Some aspects of a product do limit its quality but are not bugs!

- A bug is anything about the product that threatens its value.

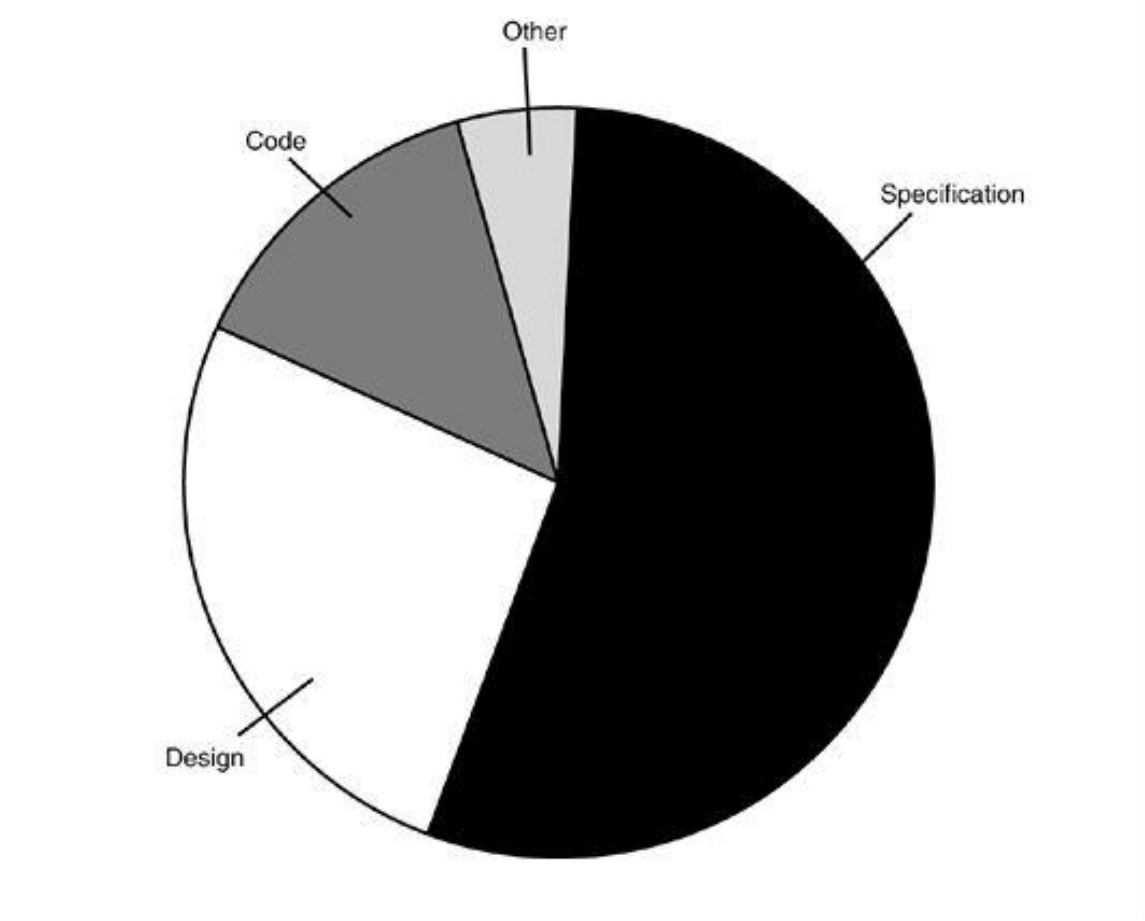
[James Bach, Michal Bolton] [BBST]

- In this course, all software problems will be called bugs.

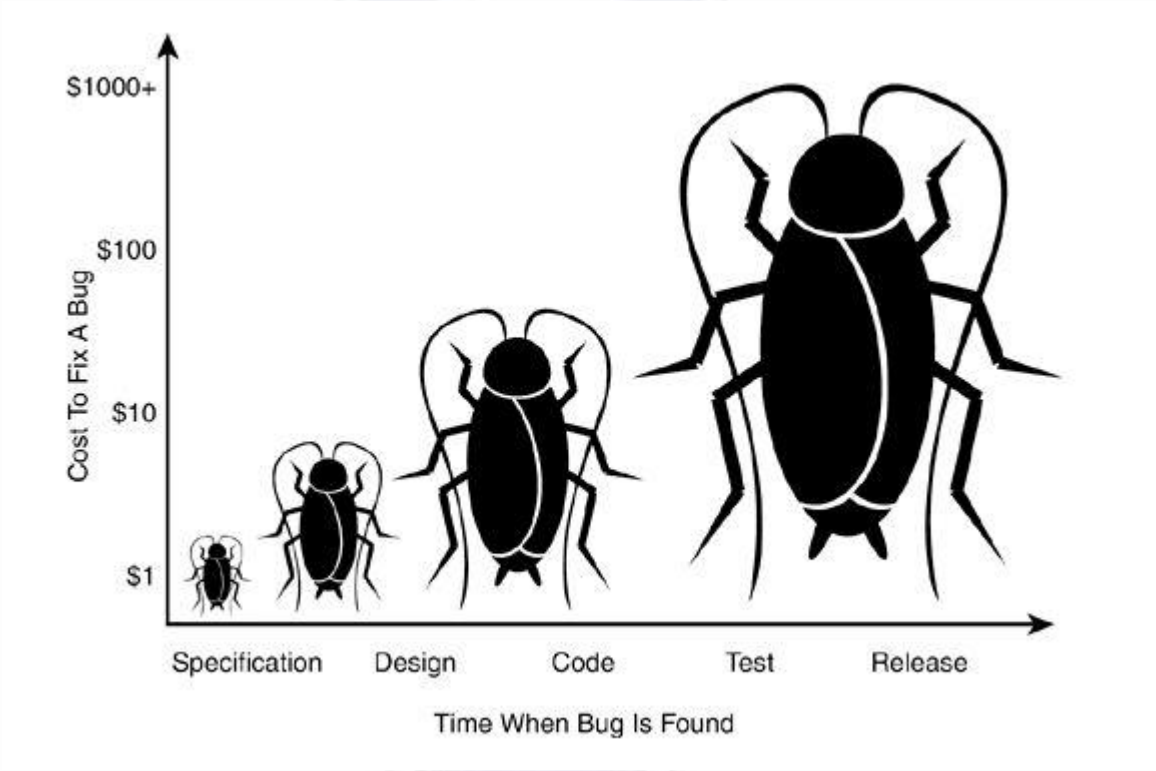
When a software bug occurs?

- A software bug occurs when one or more of the following rules is true [Pat05]:
  - The software doesn't do something that the product specification says it should do.
  - The software does something that the product specification says it shouldn't do.
  - The software does something that the product specification doesn't mention.
  - The software doesn't do something that the product specification doesn't mention but should.
  - The software is difficult to understand, hard to use, slow, or in the software tester's eyes will be viewed by the end user as just plain not right.

Why do bugs occur? [Pat05]



The cost of bugs [Pat05]



Famous Software bugs

- **Mariner 1 rochet – 1962 - diverted from its intended flight path shortly after launch.**
  - **Cause**: A programmer incorrectly transcribed a handwritten formula into computer code.
  - **Cost**: $ 18.5 million
- **World War III – almost – 1983 - The Soviet early warning system falsely indicated the United States had launched five ballistic missiles.**
  - **Cause**: A bug in the Soviet software failed to filter out false missile detections caused by sunlight reflecting off cloud-tops.
  - **Cost**: Nearly all of humanity
- **Therac-25 radiation therapy machine – 1985 - Canada Therac-25 radiation therapy machine malfunctioned and delivered lethal radiation doses to patients.**
  - **Cause**: Because of a subtle bug called a race condition, a technician could accidentally configure Therac-25 so the electron beam would fire in high-power mode without the proper patient shielding.
  - **Cost**: Three people dead, three people critically injured
- **Pentium Fails Long Division – 1993 - Intel highly-promoted Pentium chip occasionally made mistakes when dividing floating-point numbers within a specific range.**
  - **Cause**: The divider in the Pentium floating point unit had a flawed division table, missing about five of a thousand entries and resulting in these rounding errors.
  - **Cost**: $475 million, corporate credibility.
- **Disney's Lion King – 1995 - The Disney company released its first multimedia cd-rom game for children, The Lion King Animated Storybook. Several parents couldn't get the software to work.**
  - **Cause**: Disney failed to test the software on a broad representation of the many different PC models available on the market.
  - **Cost**: cd-rom replacements, corporate credibility.
- **Mars Climate Crasher – 1998 - After a 286-day journey from Earth, the Mars Climate Orbiter fired its engines to push into orbit around Mars. The engines fired, but the spacecraft fell too far into the planet atmosphere, likely causing it to crash on Mars.**
  - **Cause**: The software that controlled the Orbiter thrusters used imperial units (pounds of force), rather than metric units (Newtons) as specified by NASA.
  - **Cost**: $125 million.
- **Cancer Treatment -2000 - Radiation therapy software by Multidata Systems International miscalculated the proper dosage, exposing patients to harmful and in some cases fatal levels of radiation.**
  - **Cause**: The software calculated radiation dosage based on the order in which data was entered, sometimes delivering a double dose of radiation.
  - **Cost**: Eight people dead, 20 critically injured.

Outline

- Verification and Validation
- Software development life cycle Model
  - V-Model
  - Extended V-Model [CB03]
- Quality
  - Quality control vs. Quality Assurance
  - Quality definitions
- What is a bug?
  - First bug
  - Terms for software failures
  - Software error (or bug)
  - When? Why? Cost?
  - Famous Software bugs

Next Lecture (Still today!)

Inspection

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PhD, Assoc. Prof. Habil.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

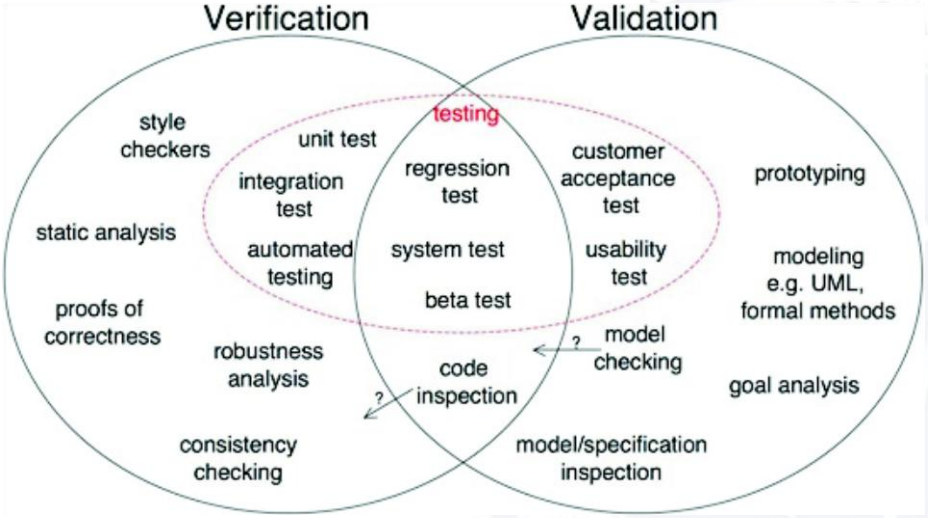
2024-2025

Software Systems Verification and Validation

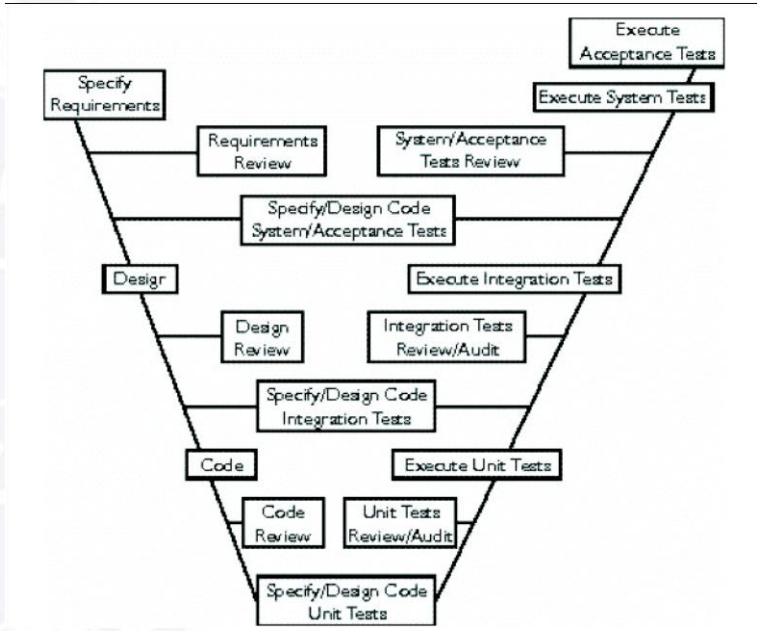
"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

What we will learn!



SDLC – V Model

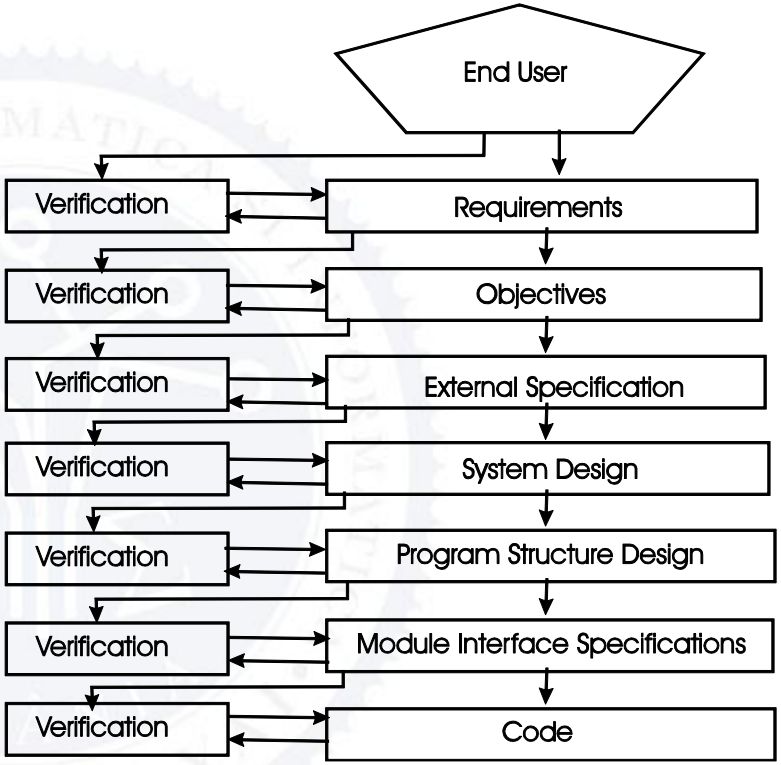


Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Human testing

- Prevent errors
  - introduction of a verification step at the end of each process.

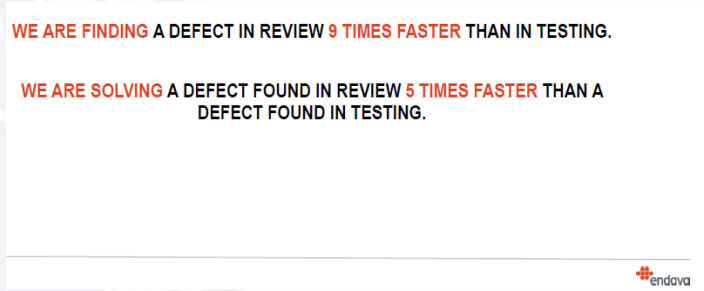


Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Human testing methods

- Is it useful? they contribute to productivity and reliability:
  - The earlier errors are found, the lower the cost of correcting the errors.
  - Psychological change of programmers when computer-based testing commences.
- Human testing methods are:
  - Inspections
  - Walkthroughs
  - Pair-programing



- Objective - to find errors but not to find solutions to the errors.
- Advantage - when an error is found it is usually located.
  - **Finds from 30% to 70% of the logic-design/coding errors in programs (?).**
- Inspection and computer-based testing are complementary.

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Inspection

- **Inspection** - process of trying to find defects in development documents during various phases of the software development process.
- Fagan Inspection team ([4 members])
  - Moderator - duties
    - Distributing materials and scheduling the inspection session.
    - Leading the session
    - Ensuring that the errors are subsequently corrected.
  - Author of the product (analyst, designer, programmer)
  - Secretary
  - Reader
- Checklists
- Time - 90-120 minutes

Inspection activities

- **Planning**
  - the moderator selects the team members
  - distribution of the materials to the members; task assignment
- **Presentation/Overview - not compulsory**
  - used to present details to the members of the inspection team
- **Individual preparation**
  - reading and understanding the received documentation
- **Inspection meeting**
  - critical observations of each individual inspectors - discussed
  - conclusions of the inspection - documented
- **Rework**
  - the author makes the required changes and correct the errors
- **Follow-up**
  - to verify if the modification did eliminate the errors
  - may be only between the author and the moderator

Inspection checklists

- Inspection scope - to find errors
- Depending on the analyzed document - special kind of errors
- **Specification Document**
  - Does the specification conform to the user's needs?
  - Are there ambiguities in the specification?
  - Do the input/output date are clearly stated? What about input/output conditions?
  - Are there requirements that are not present in the specification?
  - Are there performance conditions? What precise computation conditions?
- **Analysis Document**
  - Does the design conform to the specification?
  - Are all the functionalities from the specification specified?
- Is there an analysis documentation about the made decisions? Inspection scope - to find errors
- Depending on the analyzed document - special kind of errors
- **Specification Document**
  - Does the specification conform to the user's needs?
  - Are there ambiguities in the specification?
  - Do the input/output date are clearly stated? What about input/output conditions?
  - Are there requirements that are not present in the specification?
  - Are there performance conditions? What precise computation conditions?
- **Analysis Document**
  - Does the design conform to the specification?
  - Are all the functionalities from the specification specified?
  - Is there an analysis documentation about the made decisions?
- **Code**
  - Does the code conform to the design?
  - Are all the methods called?
  - Are all the variables initialized?
  - Problems with: infinite cycles, out of bound indexes, improper allocation of memory.
- **Test Document**
  - The test cases are well documented?
  - The test cases are well chosen?
  - Are the test data sufficient to coverage criterion?
  - For the integration testing, the order of integration is clear?
  - At regression testing is the testing continued?

Inspection advantages [CB03]

- Early error discovery
- Reduce product development time and cost
- Group method
- Mean to education
- The source of error is known (locating defect)
- Eliminates the debugging stress if few day remains until product release

- Inspection – more efficient than testing [CB03]
  - detecting, locating, repairing defect
  - a two-pass approach (individuals first and by the group)
  - checklist – calls attention to specific defect prone areas

Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Walkthrough

- **Walkthrough** [You79], [CB03] - process of trying to find defects in development documents during various phases of the software development process.
- Similar to Inspection
- **Team members** ([3-5] members)
  - Moderator ([CB03]- moderator = the producer of the reviewed material
    - ➔ a larger amount of material can be processed by the group)
  - Secretary
  - Tester
- **Procedures** are slightly different
  - Planning
  - Meeting - the participants “play computer” (no checklist)
  - No Individual preparation [CB03]
  - Rework [You79]
  - Follow-up
- Different error-detection technique
- Time - 90-120 minutes

Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Pair-Programming

- Variation of program inspection.
- Merges coding and inspection activities.
- The inspection activities
  - are not driven by checklists
  - are based on shared programming practice and style
- Programmers frequently alternate roles
- Is carried out in normal workdays, without excessive overtime and without severe schedule pressure.
- No mediator, so responsibility for open and non-defensive discussion of decisions/alternatives falls to the programmers.

Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing
- Modern Code Review
  - ICSE-2013

Expectations, Outcomes, and Challenges of Modern Code Review

Alberto Bacchelli
REVEAL @ Faculty of Informatics
University of Lugano, Switzerland
alberto.bacchelli@usi.ch

Christian Bird
Microsoft Research
Redmond, Washington, USA
cbird@microsoft.com

The Journal of Systems & Software 177 (2021) 110951
Contents lists available at ScienceDirect
The Journal of Systems & Software
journal homepage: www.elsevier.com/locate/jss

A systematic literature review and taxonomy of modern code review☆

Nicole Davila *, Ingrid Nunes
Universidade Federal do Rio Grande do Sul (UFRGS), Instituto de Informática, Porto Alegre, Brazil

  - MSR-2024

Improving Automated Code Reviews: Learning from Experience

Hong Yi Lin
The University of Melbourne
Melbourne, Australia
holin2@student.unimelb.edu.au

Patanamon Thongtanunam
The University of Melbourne
Melbourne, Australia
patanamon.t@unimelb.edu.au

Christoph Treude
Singapore Management University
Singapore, Singapore
ctreude@smu.edu.sg

Wachiraphan Charoenwet
The University of Melbourne
Melbourne, Australia
wcharoenwet@student.unimelb.edu.au

ABSTRACT

Modern code review is a critical quality assurance process that is widely adopted in both industry and open source software environments. This process can help newcomers learn from the feedback of experienced reviewers; however, it often brings a large workload and stress to reviewers. To alleviate this burden, the field of automated code reviews aims to automate the process, teaching large language models to provide reviews on submitted code, just as a human would. A recent approach pre-trained and fine-tuned the code intelligent language model on a large-scale code review corpus. However, such techniques did not fully utilise quality reviews amongst the training data. Indeed, reviewers with a higher level of experience or familiarity with the code will likely provide deeper insights than the others. In this study, we set out to investigate whether higher-quality reviews can be generated from automated code review models that are trained based on an experience-aware oversampling technique. Through our quantitative and qualitative evaluation, we find that experience-aware oversampling can increase the correctness, level of information, and meaningfulness of reviews generated by the current state-of-the-art model without introducing new data. The results suggest that a vast amount of high-quality reviews are underutilised with current training strategies. This work sheds light on resource-efficient ways to boost automated code review models.

  - ICSE-2021

Towards Automating Code Review Activities

Rosalia Tufano*, Luca Pascarella*, Michele Tufano†, Denys Poshyvanyk‡, Gabriele Bavota*
*SEART @ Software Institute, Università della Svizzera italiana (USI), Switzerland
†Microsoft, USA
‡SEMERU @ Computer Science Department, William and Mary, USA

  - ICSE-2022

Using Pre-Trained Models to Boost Code Review Automation

Rosalia Tufano
SEART @ Software Institute
Università della Svizzera italiana
Switzerland

Simone Masiero
SEART @ Software Institute
Università della Svizzera italiana
Switzerland

Antonio Mastropaolo
SEART @ Software Institute
Università della Svizzera italiana
Switzerland

Luca Pascarella
SEART @ Software Institute
Università della Svizzera italiana
Switzerland

Denys Poshyvanyk
SEMERU @ Computer Science Department
William and Mary
USA

Gabriele Bavota
SEART @ Software Institute
Università della Svizzera italiana
Switzerland

Improving Automated Code Reviews: Learning From Experience
MSR Technical Papers
Hong Yi Lin The University of Melbourne, Patanamon Thongtanunam University of Melbourne, Christoph Treude Singapore Management University, Wachiraphan (Ping) Charoenwet The University of Melbourne

Outline

- Human testing
- Human testing methods
  - Inspections
  - Walkthroughs
  - Pair-programing

Next Lecture

- Testing. Test planning.
- Test case design - Black-box testing

- Testing Management Tool
- Continuous integration - Jenkins

References

- [PY08] M. Pezzand and M. Young. *Software Testing and Analysis: Process, Principles and Techniques*. John Wiley and Sons, 2008.
- [Mye04] Glenford J. Myers, *The Art of Software Testing*, John Wiley & Sons, Inc., 2004
- [You79] E. Yourdon, Structured Walkthroughs, Prentice-Hall,Englewood Cliffs, NJ, 1979
- [CB03] Jean-Francois Collard and Ilene Burnstein. *Practical Software Testing*. Springer-Verlag New York, Inc., 2003.
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Testing. Test planning.
- Test case design - Black-box testing
- Continuous integration - Jenkins

Outline

- Testing – fundamental questions
- Black-box testing (domain testing)
  - Equivalence partitioning (EP)
  - Boundary-value analysis (BVA)
  - Advantages/Disadvantages
- Example - black-box testing
  - Example – EP+BVA
- Maven
- Continous integration tool - Jenkins

Domain testing

- The problem you have to solve
  - You cannot afford to run every possible test.
  - You need a method for choosing a few powerful tests that will represent the rest.
- Domain testing
  - Provides **a sampling strategy**
  - It provides **heuristics** for choosing a small number of tests that are powerful enough to represent the larger domain.
    - Equivalence
    - Boundaries
- Input versus output variables
  - Domain definitions
    - Specification:
      - Data (X) + preconditions
      - Result (Z)+ postcondition
- Primary dimension versus secondary dimension [BBST]
  - Primary dimension - reflects the reason you are entering data into the field
  - Secondary dimension - reflects the other ways the input can vary

Domain testing (Black-box testing)

- Equivalence partitioning
- Boundary-value analysis

Domain testing (Black-box testing)

Equivalence partitioning

- Equivalence class (EC) - definition [Mye04]
  - a partition of input domain of a program.
- Equivalence partitioning
  - to partition the input domain of a program into a finite number of equivalence classes such that you can reasonably assume that a test of a representative value of each class is equivalent to a test of any other value.

Domain testing (Black-box testing)

Test-case design by equivalence partitioning - steps

- Identifying the equivalence classes (EC)
  - Valid equivalence classes.
  - Invalid equivalence classes.
  - Default, empty, blank, null, zero, none.
  - Invalid, wrong, incorrect, garbage date.
- Defining the test cases
  - assign a unique number to each equivalence class;
  - Until all valid/invalid equivalence classes have been covered by (incorporated into) test cases:
    - write a new test case covering as many of the uncovered valid equivalence classes as possible;
    - write a test case that covers one, and only one, of the uncovered invalid equivalence classes.

Domain testing (Black-box testing)

Test-case design by equivalence partitioning - guidelines

- An input condition specifies a range of values [a,b].
  ➔ 1 valid EC, 2 invalid EC
- An input condition specifies the number of values “1 to 3 possibilities”.
  ➔ 1 valid EC and 2 invalid EC
- An input condition specifies a set of input values.
  ➔ 1 valid EC for each element in the set, 1 invalid EC
- An input condition specifies a must be situation.
  ➔1 valid EC, 1 invalid EC
- If there is any reason to believe that the program does not handle elements in an equivalence class identically, split the equivalence class into smaller equivalence classes.

Domain testing (Black-box testing)

Boundary-value analysis

- Boundary-value analysis - definition [Mye04]
  - focuses on the boundary areas of a programs input domain
- Boundary conditions
  - Situations directly on, above, and beneath the edges of input EC and output EC.
  - One or more elements should be selected such that each edge of the EC is the subject of a test.
  - BVA explores situations on and around the edges of the EP.

Domain testing (Black-box testing)

Test-case design by boundary-value analysis -guidelines

- An input condition specifies a range of values [a,b].
  - ➔ the ends of the range, situations just beyond the ends;
- An input condition specifies the number of values “1 to 3 possibilities”.
  - ➔ the minimum and maximum number of values, one beneath and beyond these values;
- An input condition specifies an ordered set of input values.
  - ➔ the first, the last elements of the set;
- The above rules are applied to the output conditions.

Domain testing (Black-box testing)

Advantages	Disadvantages
• No knowledge of implementation. • Tester independent of programmer. • User's point of view. • Ambiguities in spec. • After specifications is completed.	• A small number of inputs. • No clear spec. • Hard to design. • Unnecessary repetition of test. • Many program paths untested. • Specific segments of code?

Domain testing (Black-box testing)

Example

- **Problem statement**: Compute the number of participants with the maximum score (0 to 100 points possible) at a competition.

- Applied:
  - EP
  - BVA
- See example files
- **Lecture02_Demo**

Domain testing (Black-box testing)

Example

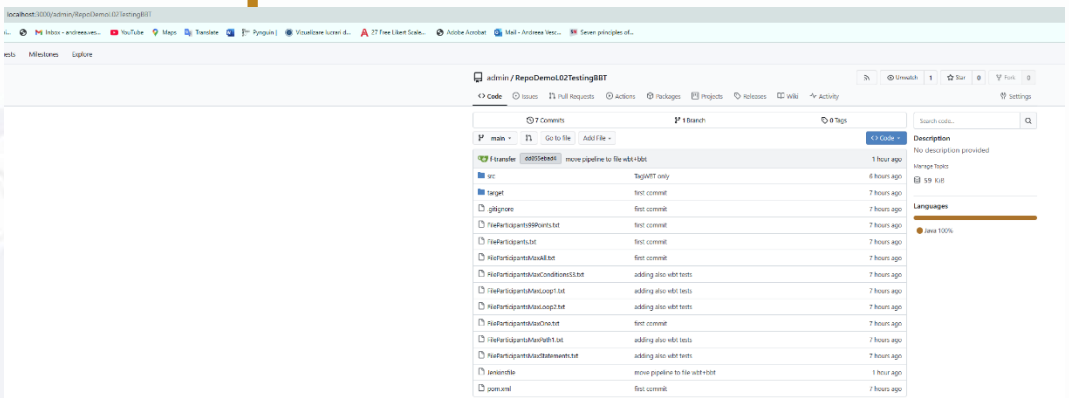
- Maven
  - goal - to allow a developer to comprehend the complete state of a development effort in the shortest period of time
  - https://maven.apache.org/what-is-maven.html
  - The Surefire Plugin is used during the test phase of the build lifecycle to execute the unit tests of an application.
    - https://maven.apache.org/surefire/maven-surefire-plugin/index.html
- Jenkins
  - Continuous integration tool

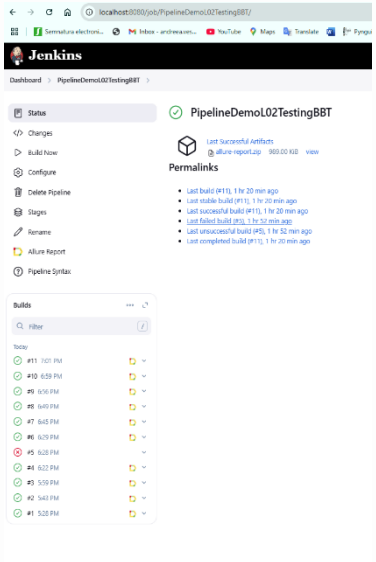
- File Info_CI_CD.pdf
  - Docker and Docker compose
- Create Repository in Gitea
- Add the source code to this project
- Create the Pipeline job
- Run the Job

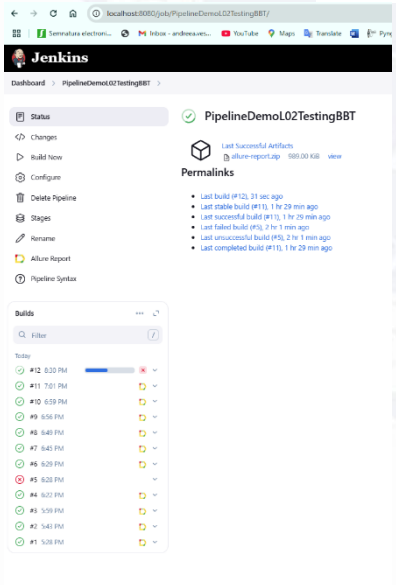
Domain testing (Black-box testing)

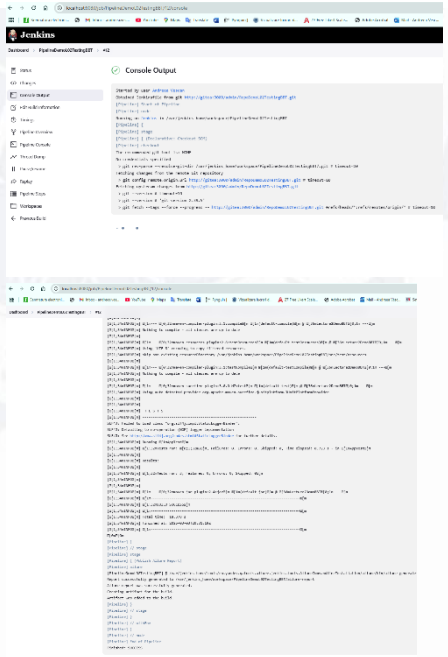
Example

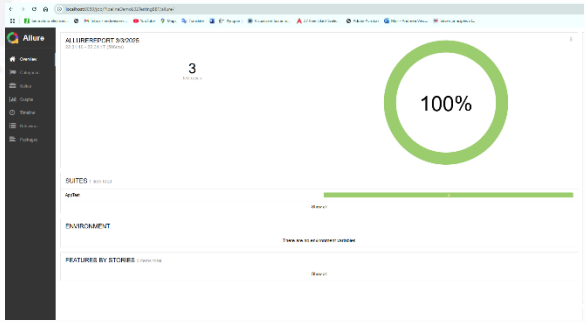
- File Info_CI_CD.pdf
  - Docker and Docker compose
- Create Repository in Gitea
- Add the source code to this project
- Create the Pipeline job
- Run the Job











Outline

- Testing – fundamental questions
- Black-box testing (domain testing)
  - Equivalence partitioning (EP)
  - Boundary-value analysis (BVA)
  - Advantages/Disadvantages
- Example - black-box testing
  - Example – EP+BVA
- Maven
- Continuous integration tool - Jenkins

References

- [PY08] M. Pezzand and M. Young. *Software Testing and Analysis: Process, Principles and Techniques*. John Wiley and Sons, 2008.
- [Mye04] Glenford J. Myers, *The Art of Software Testing*, John Wiley & Sons, Inc., 2004
- [You79] E. Yourdon, Structured Walkthroughs, Prentice-Hall,Englewood Cliffs, NJ, 1979
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010
- [BBST] BBST Testing course, http://testingeducation.org/BBST/
  - **Test design,**
    - **Lecture 5: Domain testing**
- Tutorials
  - Tutorials in Teams

Next Lecture

- Test case design - White-box testing

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

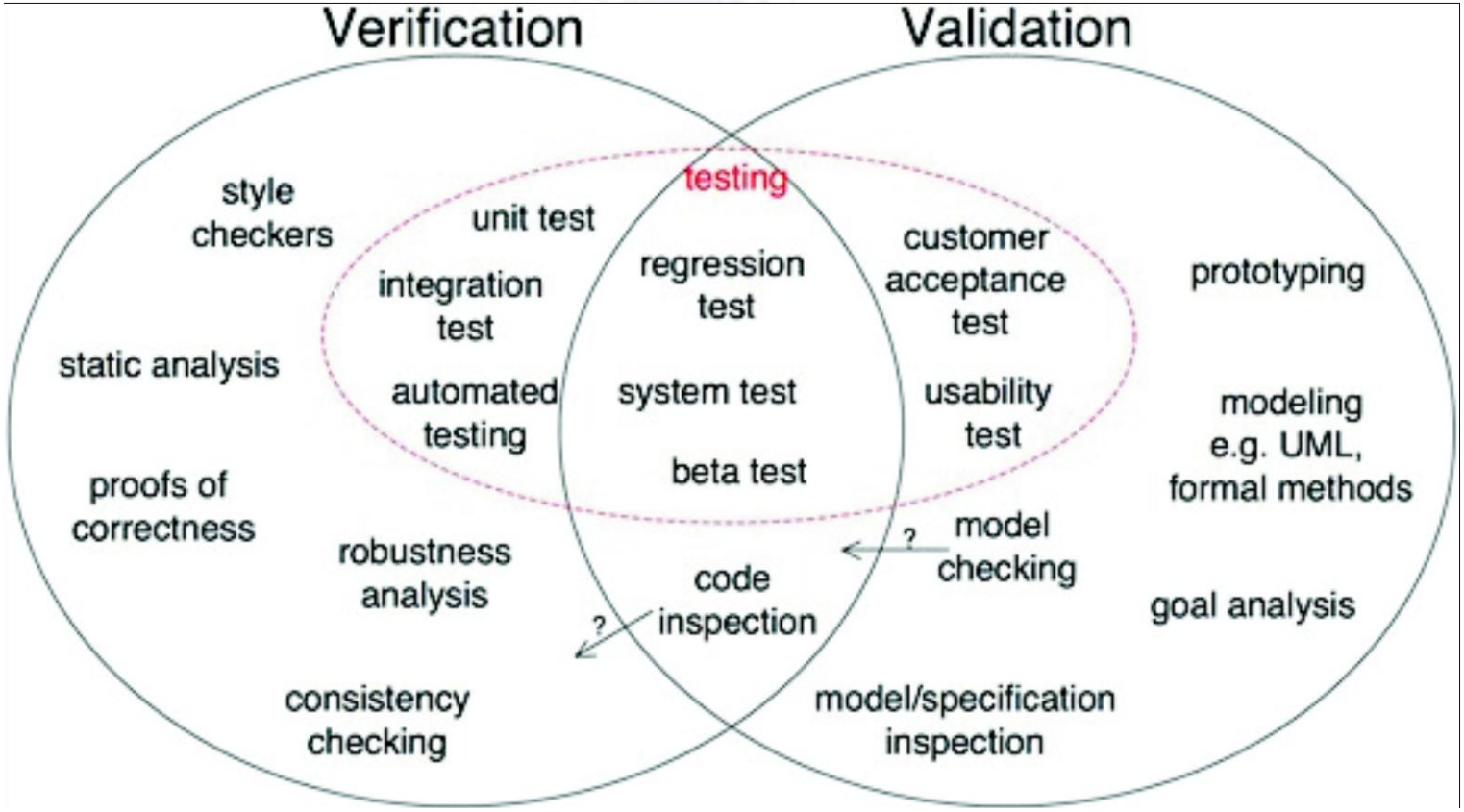
2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

What we will learn!



(Next)/Today Lecture

- Testing. Test planning.
- Test case design - Black-box testing
- Continuous integration - Jenkins

Outline

- Testing – fundamental questions (why, how, when, enough?)
- Software testing
  - Definitions (why)
- Testing strategy (how)
- Test planning (enough)
  - Test cases
  - Testing activities
  - Bug report
- Software testing (Required reading before Seminar 2)
  - Testing principles
  - Testing axioms
- When do we test?

Testing - fundamental questions

- What skills do we need to have to test?

- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

What skills do we need to have to test?

Play … detective!

What skills do we need to have to test?

Playing detective (1)

Context 1.

A man went into a party and drank some of the punch. He then left early. Everyone at the party who drunk the punch subsequently died of poisoning. Why did the man not die?

Context 2.

A detective who was mere days from cracking an international smuggling ring has suddenly gone missing. While inspecting his last-known location, you find a note:

710 57735 34 5508 51 7718

Currently there are 3 suspects: Bill, John, and Todd. Can you break the detective's code and find the criminal's name?

Context 3.

There is a man found dead in a circular mansion. The detective interviews the cook, maid, and babysitter. The cook said he couldn't have done it because he was preparing the meal. The maid said she couldn't have done it because she was dusting the corners. The babysitter said she couldn't because she was playing with the children. Who was lying?

What skills do we need to have to test?

Playing detective (2)

Can you state (some) similarities between **a detective** and **a tester**?

Detectives	Testers

Subject of **Investigation**

Crime – commission of an act that us forbidden, harmful act against the public

Computer program (CP) – instructions for a computer, communications among humans-computers

Used **Tools**

Statements by witnesses/suspects, recording equipment, evidence kit

Test plan, management tool, screen capture tools, testing techniques, bug reports

Job **Duties**

Researching/analyzing crimes, Identifying/interrogating suspects, testifying in court

Research for information, interrogating, Bug advocacy

They do not even know what crime has been committed yet!

Testing - fundamental questions

- What skills do we need to have to test?

- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

Why do we test?

Software testing - DEFINITION

- **Software testing [BBST]** - is an empirical technical investigation conducted to provide stakeholders with information about the quality of the product or service under test
- **False/incomplete definitions**
  - Testing is the process of demonstrating that errors are not present. “Testing can only reveal the presence of errors, never their absence.” [Dij75]
  - The purpose of testing is to show that a program performs its intended functions correctly.
  - Testing is the process of establishing confidence that a program does what it is supposed to do.
- **Definition**
  - Testing is the process of executing a program with the intent of finding errors. [Mye04]
    - Human beings tend to be highly goal-oriented.
    - Testing is a destructive process.
    - Successful and unsuccessful.

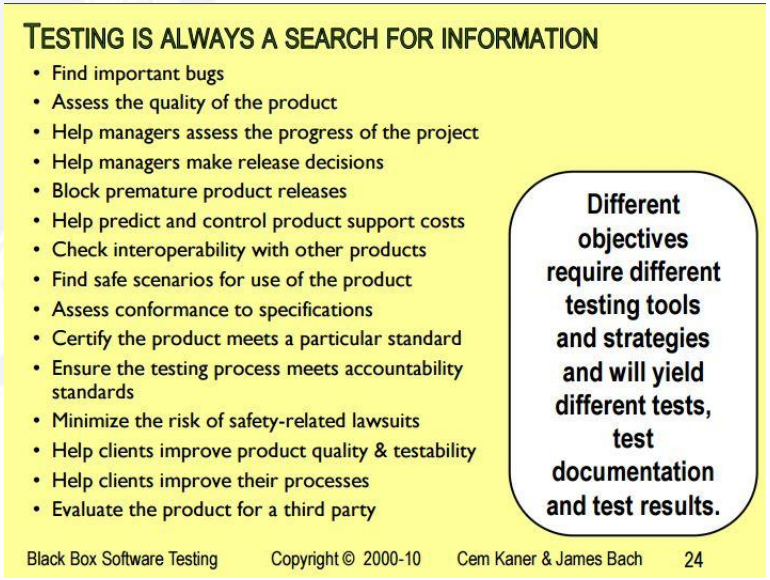
    - Error: the program does not do what it is supposed to do, BUT also if the program does what it is not supposed to do.

Why do we test?

We test a product to learn about its quality. [BBST]

- Everyone tests in context [BBST]
  - Harsh constraints
    - Complete testing is impossible
    - Finite project schedules and budget
    - Limited skills of the test group
  - Do testing – before/during/after – release
  - Improvement of product – might/might not be an objective of testing
  - Test in behalf of stakeholders
    - Project manager/customer/programmer/attorney

- A search for information [BBST]
  - **Question:** What kind of information are we most concerned about learning from this testing?
  - **Answer:** Our information objectives.



Testing - fundamental questions

- What skills do we need to have to test?

- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

How do we test?

Testing … in context

Context 1.

The teacher asked a group of students (development team) to design and implement a Java application for the given statement problem. The teacher just want to verify the students 's design and implementation skills.

You are working as a tester, and you are assigned to the development team early in the project. The project manager (i.e., the teacher) wants you to test along with the programmers as they write code. She wants you to help the programmers deliver clean code and she wants to help her, and her staff identify, understand and control the implementation.

Context 2.

The teacher asked a group of students (development team) to design and implement a Java application for the given statement problem. The teacher just want to verify the students 's design and implementation skills.

You are working as a tester, and you are assigned to the development team when all the required stated functionalities were implemented and individually tested. The project manager (i.e., the teacher) wants you to test the integration of the implemented functionalities.

Context 3.

The teacher asked a group of students (development team) to design and implement a Java application for the given statement problem. The resulted application will be used by the doctors in their university and by other family doctors.

You are working as a tester, and you are assigned to the development team close to the project's release date. The doctors expected it to work, and they needed to work well. The project manager (i.e., the teacher) expects you to test the product in ways that will help her understand whether the product is ready to release or not.

How do we test?

Strategy/Approach/Technique

- **Strategy definition** (https://en.oxforddictionaries.com/definition/strategy)
  - A plan of action designed to achieve a long-term or overall aim.
- **Approach definition** (http://www.dictionary.com/browse/approach)
  - the method used or steps taken in setting about a task, problem
- **Technique definition** (http://www.dictionary.com/browse/approach)
  - technical skill; ability to apply procedures or methods so as to effect a desired result

- **Testing strategy** = guiding framework for deciding what tests (what test techniques) are best suited to your product.
- **Approach = a way of thinking about …**
- **Technique = methods used to …**

How do we test?

Testing strategies [BBST]

- **Testing strategy is:**
  - The guiding framework for deciding what tests (what test techniques) are best suited to your product.
- **Context** and **information objectives** are (or should be) the drivers of any **testing strategy.**

- **Examples of context factors that drive and constrain testing**
  - Who are the stakeholders with influence?
  - What are the goals and quality criteria for the project?
  - What skills and resources (time, money, tools, data, etc) are available?
  - Potential consequences of potential failures?
  - How could it fail?

- **Common information objectives**
  - Find important bugs
  - Asses the quality of the product
  - Help managers make release decisions
  - Block premature product releases
  - Assess conformance to the specification
  - Find safe scenarios for the use of the product
  - Evaluate the product for a third party

How do we test?

Testing strategies [BBST]

- **Context** and **information objectives** are (or should be) the drivers of any **testing strategy.**

Selecting the Testing techniques ?

Techniques differ in core.

Attributes of "good" tests

- *Power*
- *Valid*
- *Value*
- *Credible*
- *Representative*
- *Non-redundant*
- *Motivating*
- *Reusable*
- *Performable*
- *Maintainable*
- *Information value*
- *Coverage*
- *Easy to evaluate*
- *Support troubleshooting*
- *Appropriately complex*
- *Accountable*
- *Affordable*
- *Opportunity cost*

How do we test?

Strategy/Approach/Technique (revisited)

Selecting the Testing techniques ?

http://www.testingeducation.org/BBST/testdesign/

- ***http://www.testingeducation.org/BBST/testdesign/BBSTTestDesign2011pfinal.pdf***
- ***Slides 67 to 72***
- COVERAGE-BASED TECHNIQUES FOCUS ON WHAT GETS TESTED
- TESTER-BASED TECHNIQUES FOCUS ON WHO DOES THE TESTING
- RISK-BASED TECHNIQUES FOCUS ON POTENTIAL PROBLEMS
- ACTIVITY-BASED TECHNIQUES FOCUS ON HOW YOU DO THE TESTING
- EVALUATION-BASED TECHNIQUES FOCUS ON YOUR ORACLE
- DESIRED-RESULT TECHNIQUES FOCUS ON A SPECIFIC DECISION OR DOCUMENT
- THERE ARE ALSO GLASS-BOX TECHNIQUES

Testing - fundamental questions

- What skills do we need to have to test?

- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

How do we organize the process of testing?

Test planning

- Test plan?
  - testers communicate what they intend to do.
  - takes the form of a written document, not only the creation of the document, but also planning the testing tasks.
  - The ultimate goal of test planning process is communicating the software test team's intend, expectations, understanding.

- Test planning topics
  - A test plan template? Important topics?
    - The test team's high-level expectations
    - People, places and things, Definitions
    - Inter-group responsibilities
    - What will and won't be tested
    - Test phases, test strategy, Resource requirements
    - Tester assignments
    - Test schedule, Test cases
    - Bug reporting

- Reasons for planning test cases
  - Organization
  - Repeatability
  - Tracking
  - Proof of testing

- VerVal matrix

How do we organize the process of testing?

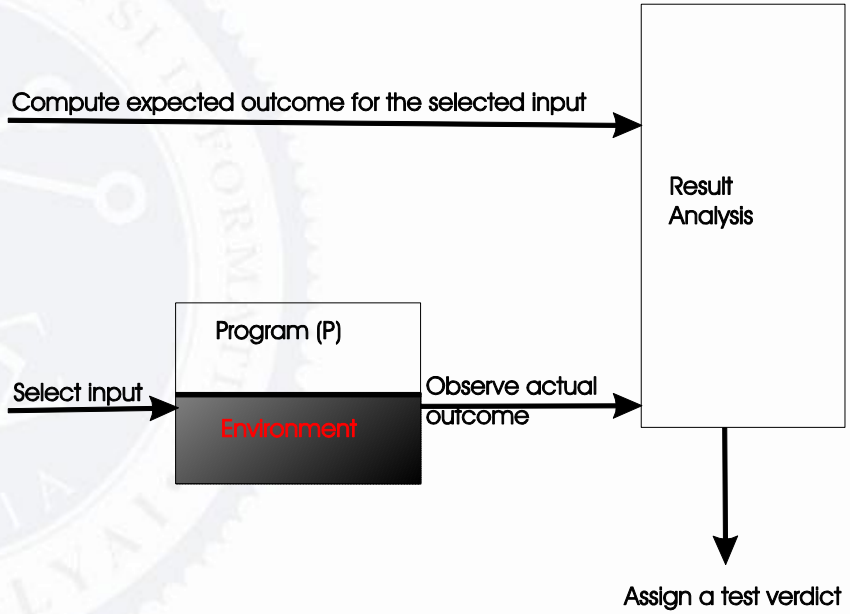
Test case

- Test case < i , r >
  - i domain D
  - r domain R.
  - for input i the expected result is r.
- Test case: “ A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement.” [IEE90]
- “Good” test case attributes:
  - High probability of finding an error.
  - Is not redundant.
  - “Best of breed”.
  - Neither too simple nor too complex.

How do we organize the process of testing?

Testing activities [NTO5]

- Identify an objective to be tested.
- Select inputs.
- Compute the expected outcome.
- Set up the execution environment of the program.
- Execute the program.
- Analyze the test result.



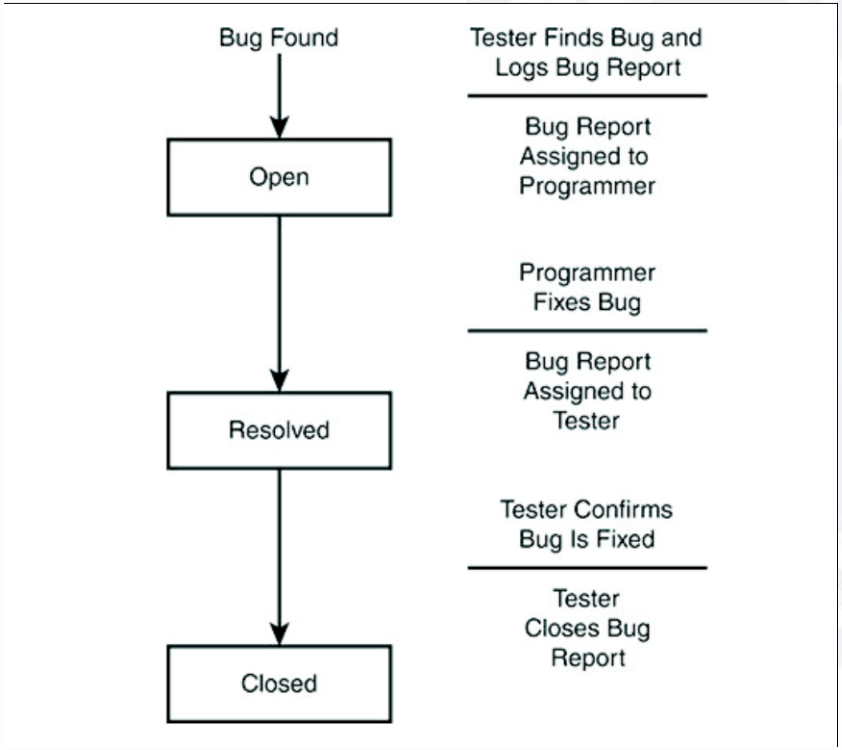
How do we organize the process of testing?

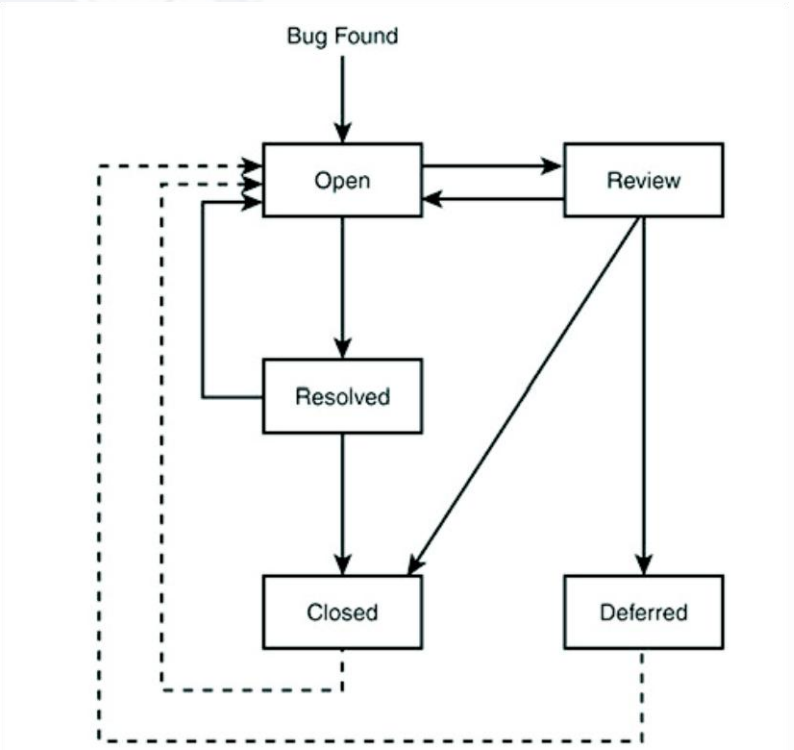
Reporting a bug

- Principles for reporting a bug [Pat05]
  - Report bugs as soon as possible
  - Effectively describe the bugs
  - Be nonjudgmental in reporting bugs
  - Follow up on your bug reports
- Isolating and reproducing bugs [Pat05] - suggestions in isolating a bug
  - Don’t take anything for granted
  - Look for time-dependent and race condition problems
  - White-box issues of boundary condition bugs, memory leaks, data overflows.
  - State bug
  - Resource dependencies and interactions with memory, network, hardware sharing.
  - Don’t ignore the hardware

How do we organize the process of testing?

A bug's Life Cycle [Pat05]





Testing - fundamental questions

- What skills do we need to have to test?

- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

When have we tested enough?

Program under test

- Program P [Fre10]
- P : D → R, where:
  - D - set of input data;
  - R - set of output data.

Types of testing [Fre10]

- Exhaustive testing - all the possible inputs.
  - if D is finite, then P is executed for all possible inputs.
- Selective testing
  - if D is not finite, then we choose inputs i from S (included in D).

- Cem Kaner, The Impossibility of Complete Testing, http://www.testingeducation.org/BBST/foundations/Kaner_impossibility.pdf

When have we tested enough?

Required reading (before Seminar 2)

Testing principles [Mye04]

- Definition of the expected output or result;
- Avoid testing your own program.
- A programming organization should not test its own programs.
- Thoroughly inspect the results of each test.
- Test cases for valid/invalid input conditions.
- Test if the program does not do what it is supposed to do, AND if the program does what it is not supposed to do.
- Do not throwaway test cases.
- Plan testing assuming errors will be found.
- The probability of the existence of more errors in a section of a program is proportional to the number of errors already found in that section.
- Testing is an extremely creative and intellectually challenging task.
- **Remark:** Principles rewritten in [CB03].

Testing axioms [Pat05]

- It is impossible to test a program completely.
- Software testing is a risk-based exercise.
- Testing can't show that bugs don't exist.
- The more bugs you find, the more bugs there are.
- The pesticide paradox.
- Not all the bugs you find will be fixed.
- When a bug's a bug it is difficult to say.
- Product specification are never final.
- Software testers aren't the most popular member of a project team.
- Software testing is a disciplined technical profession.

Testing - fundamental questions

- What skills do we need to have to test?

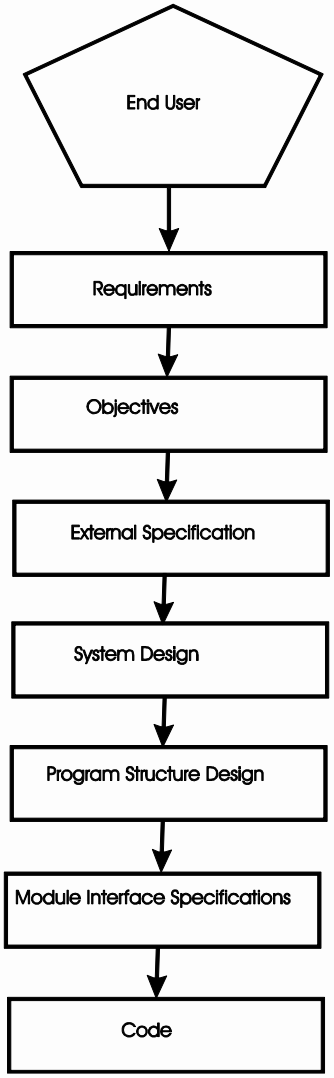
- Why do we test?
- How do we test?
- How do we organize the process of testing?
- When have we tested enough?
- When we test?

[Mye04] (chapter 3), [PY08] (chapter 18), [Fre10] (chapter 4)

When we test?

Software development process

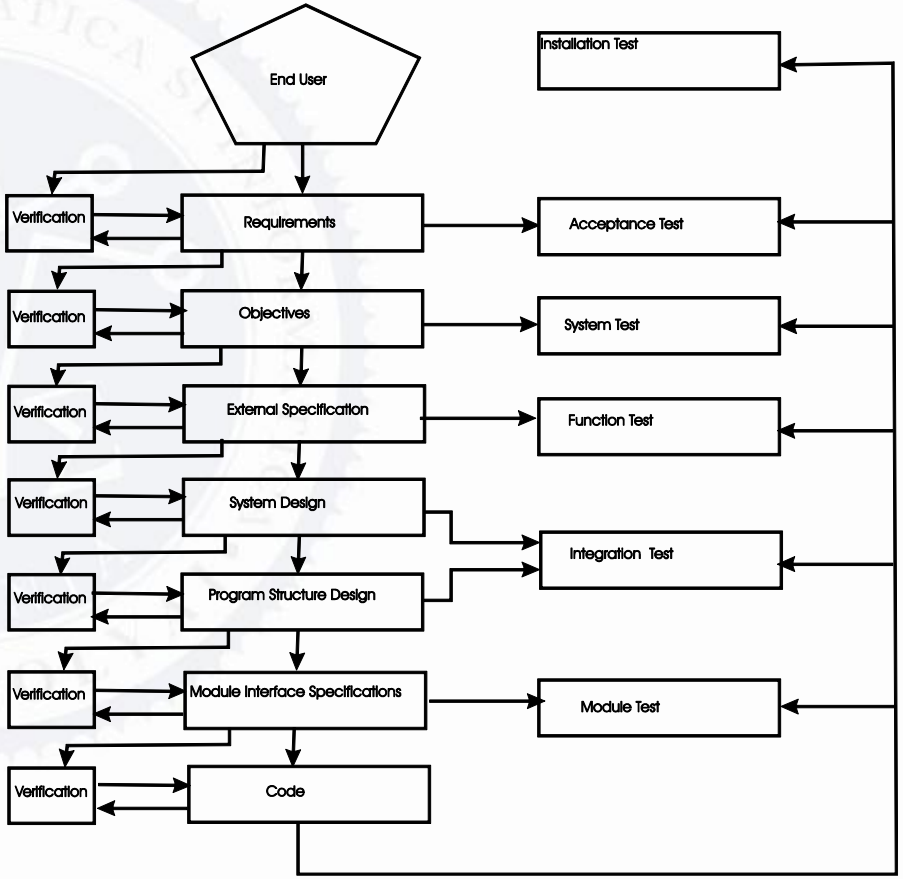
- user's needs are translated into requirements
- requirements are translated into objectives
- objectives are translated into external specification
- system design
- program structure design
- module interface specification
- code



When we test?

Development and testing processes

- Approaches to prevent errors:
  - More precision into the development process.
  - Introduction of a verification step at the end of each process.
  - Orient distinct testing processes toward distinct development processes.



When we test?

Levels of testing

Unit testing

- Testing individual subprograms, subroutines, procedures, the smaller building blocks of the program.
- **Motivations**:
  - Managing the combined elements of testing.
  - Module testing eases the task of debugging.
  - Module testing introduces parallelism into the program testing process.
- **Points of view**
  - The manner in which test cases are designed.
  - The order in which modules should be tested and integrated.
- Advice about performing the test.

- References: [Mye04] (chapter 5),[NT05] (chapter 3).

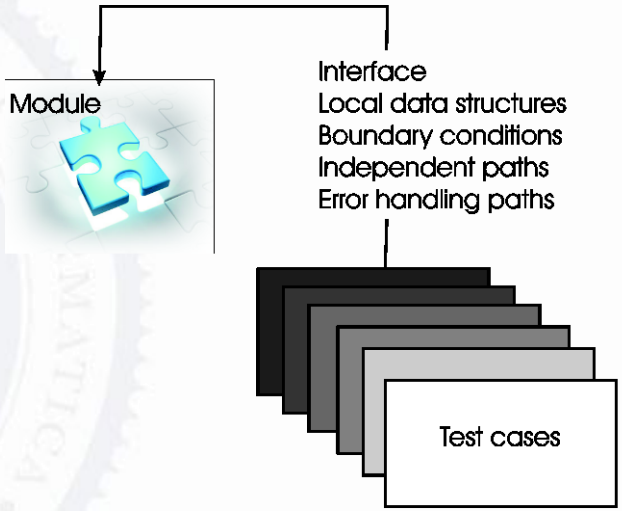
When we test?

Levels of testing

Unit testing

Test case design

- Information needed when designing test cases for a module:
  - specification of the module
  - the module's source code
- Test case design procedure for a module test is:
  - Analyze the logic of the module using white-box methods.
  - Applying black-box methods to the module's specification.



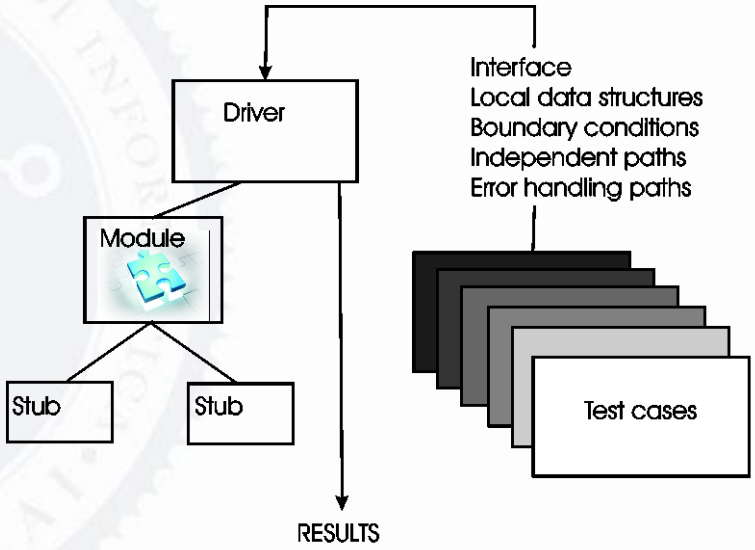
When we test?

Levels of testing

Unit testing

Unit test procedures

- Unit test environment
  - **driver** - a ”main program” that accepts test case data, passes such data to the component to be tested and prints relevant results;
  - **stub** - serve to replace modules that are subordinate the component to be tested.
    - uses the subordinate module’s interface
    - may do minimal data manipulation
    - prints verification of entry
    - returns control to the module undergoing testing.



References

- [Pat05] R. Patton. *Software Testing*. Sams Publishing, 2005.
- [Mye04] Glenford J. Myers, *The Art of Software Testing*, John Wiley & Sons, Inc., 2004
- [CB03] Jean-Francois Collard and Ilene Burnstein. *Practical Software Testing*. Springer-Verlag New York, Inc., 2003.
- [NT05] K. Naik and P. Tripathy. *Software Testing and Quality Assurance*. Wiley Publishing, 2005.
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010
- [BBST] BBST Testing course, http://testingeducation.org/BBST/
  - **Foundations of Software Testing,**
    - **Lecture 2: Strategy**
    - **Lecture 5: The Impossibility of Complete Testing**

Next Lecture

- Testing. Test planning.
- Test case design - Black-box testing
- Continuous integration - Jenkins

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Levels of testing

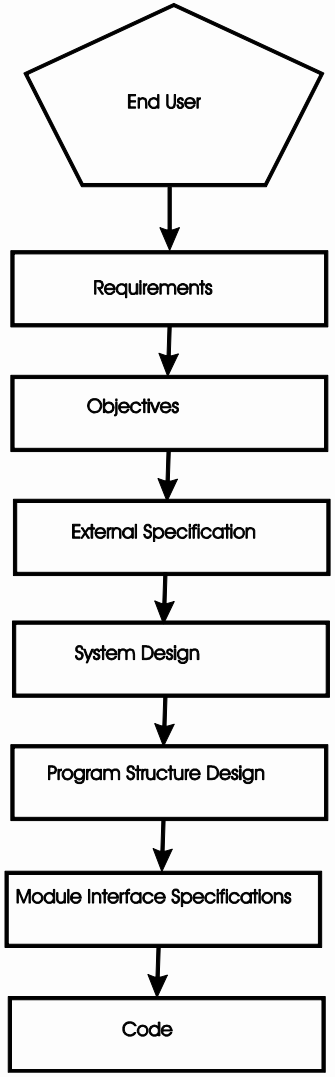
Outline

- Software development process
  - Software development process
  - Development and testing processes
- Levels of testing
  - Unit testing
  - Integration testing
  - Function testing
  - System testing
  - Acceptance testing
- Retesting vs regression testing

- Next lecture:
  - **Bogdan Iudean Lecture on CI/CD Pipelines**
  - **Bonus points - Research Education Study**
- Questions

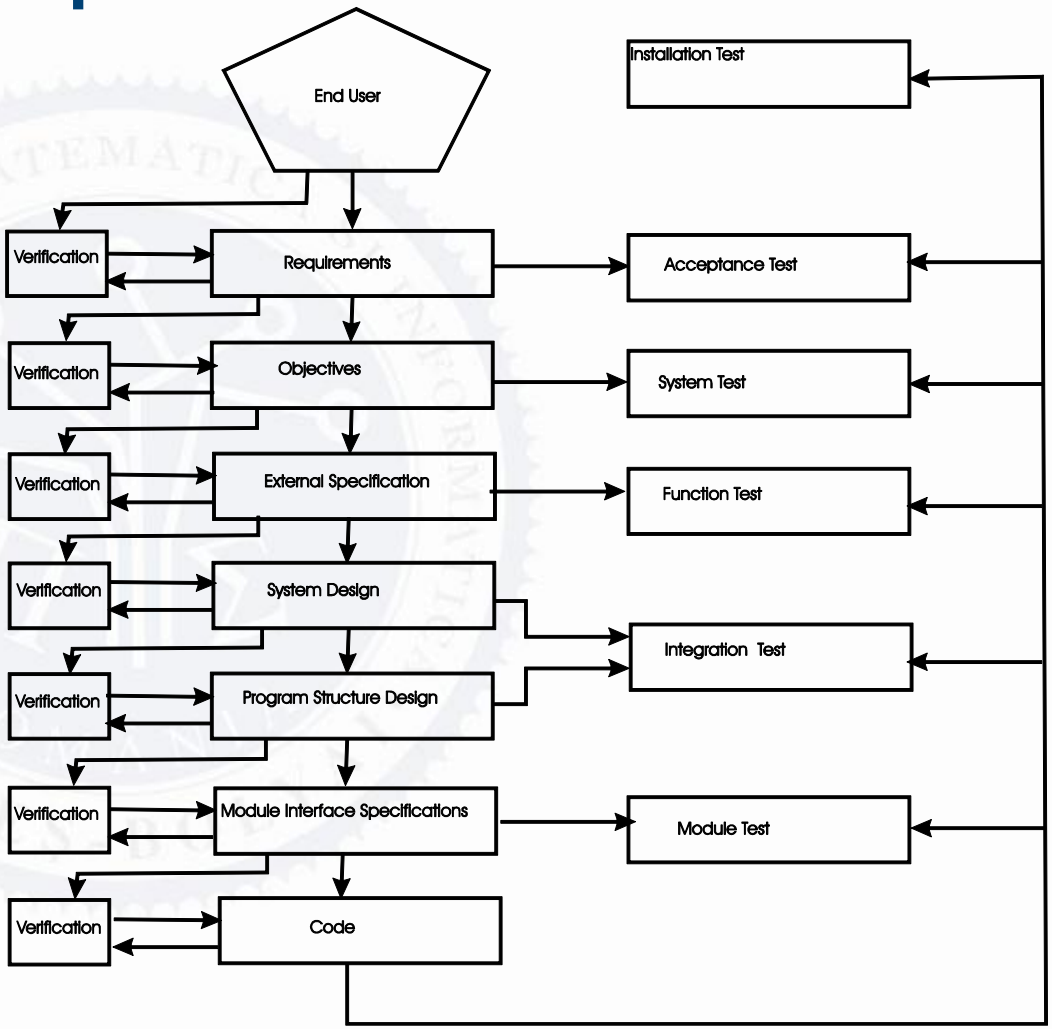
Software development process

- user's needs are translated into requirements
- requirements are translated into objectives
- objectives are translated into external specification
- system design
- program structure design
- module interface specification
- code



Software development process

- Approaches to prevent errors:
  - More precision into the development process.
  - Introduction of a verification step at the end of each process.
  - Orient distinct testing processes toward distinct development processes.



Outline

- Software development process
  - Software development process
  - Development and testing processes
- Levels of testing
  - Unit testing
  - Integration testing
  - Function testing
  - System testing
  - Acceptance testing
- Retesting vs regression testing

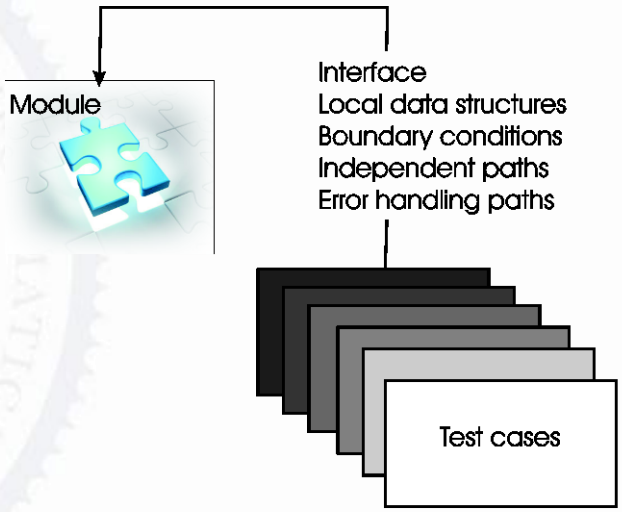
- Next lecture:
  - **Bogdan Iudean Lecture on CI/CD Pipelines**
  - **Bonus points - Research Education Study**
- Questions

Levels of testing

1.Unit testing

Test case design

- Information needed when designing test cases for a module:
  - specification of the module
  - the module's source code
- Test case design procedure for a module test is:
  - Analyze the logic of the module using white-box methods.
  - Applying black-box methods to the module's specification.

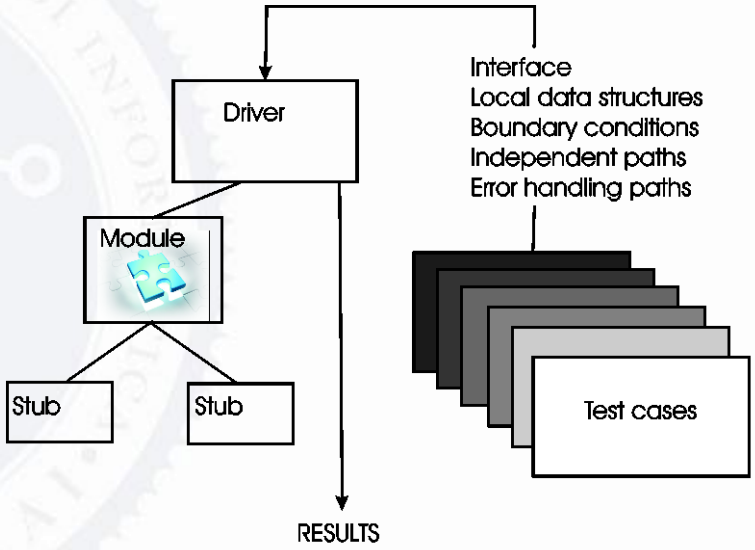


Levels of testing

1. Unit testing (cont.)

Unit test procedures

- Unit test environment
  - **driver** - a ”main program” that accepts test case data, passes such data to the component to be tested and prints relevant results;
  - **stub** - serve to replace modules that are subordinate the component to be tested.
    - uses the subordinate module’s interface
    - may do minimal data manipulation
    - prints verification of entry
    - returns control to the module undergoing testing.



Levels of testing

2. Integration testing

- Constructing the program structure while at the same time conducting tests to uncover errors associated with *interfacing*.
- **Importance** of integration testing:
  - Different modules are generally created by groups of different developers.
  - Unit testing of individual modules is carried out in a controlled environment by using test drivers and stubs.
  - Some modules are more error prone than other modules.
- Objectives:
  - putting the modules together in an incremental manner
  - ensuring that the additional modules work as expected without disturbing the functionalities of the modules already put together.
- Reference: [NT05] (chapter 7).

Levels of testing

2. Integration testing (cont)

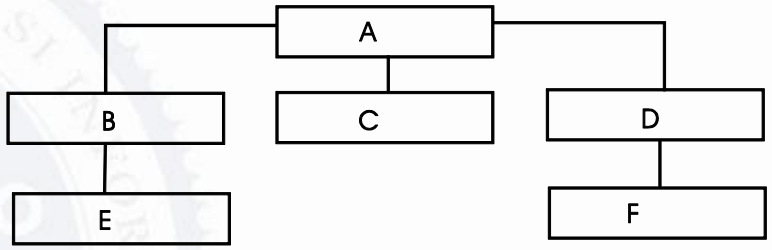
Techniques [Fre10]

- Big-bang
- Incremental
  - Top-down.
  - Bottom-up.
- Sandwich.

Levels of testing

2. Integration testing (cont)

Big bang testing

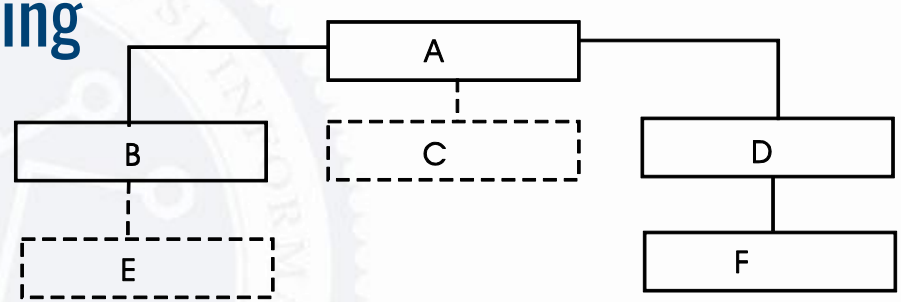


- Big-bang procedures:
  - Module test for each individual unit;
    - A driver module;
    - Several stub modules.
  - The modules are combined to form the program.
- **Observations**
  - more work for big-bang
  - mismatching interfaces/incorrect assumptions among modules - detected earlier with incremental
  - Debugging easier - incremental
  - Big-bang - appears to use less machine time
  - parallel activities – opportunity for big-bang

Levels of testing

2. Integration testing (cont)

Top-down incremental testing



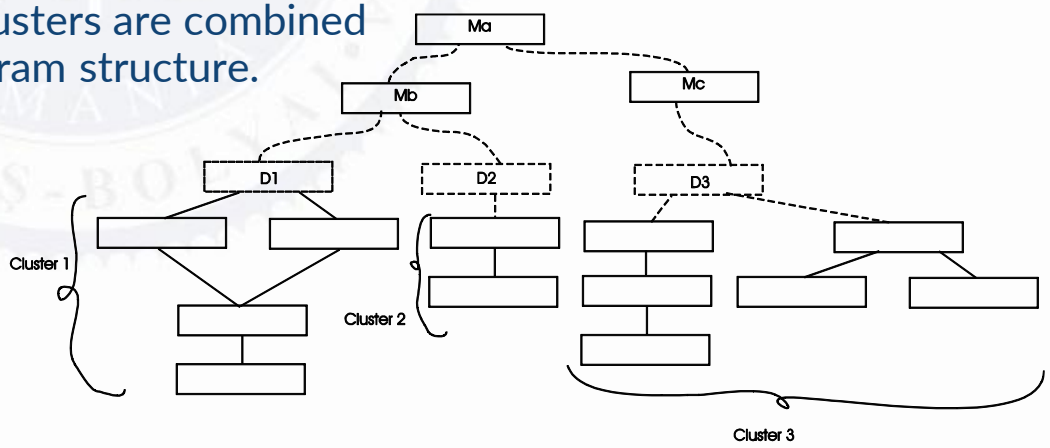
- Top-down integration manner:
  - Depth-first integration;
  - Breadth-first integration.
- Top-down integration process:
  - main control module = driver;
  - stubs=substituted for all components directly subordinate;
  - subordinates stub ← actual components;
  - tests are conducted as each component is integrated;
  - on completion of each set of tests, another stub ← real component;
  - regression testing may be conducted.

Levels of testing

2. Integration testing (cont)

Bottom-up incremental testing

- Bottom-up integration process:
  - low-levels components are combined into clusters;
  - a driver is written to coordinate test case input and output;
  - the cluster is tested;
  - drivers are removed and clusters are combined moving upward in the program structure.

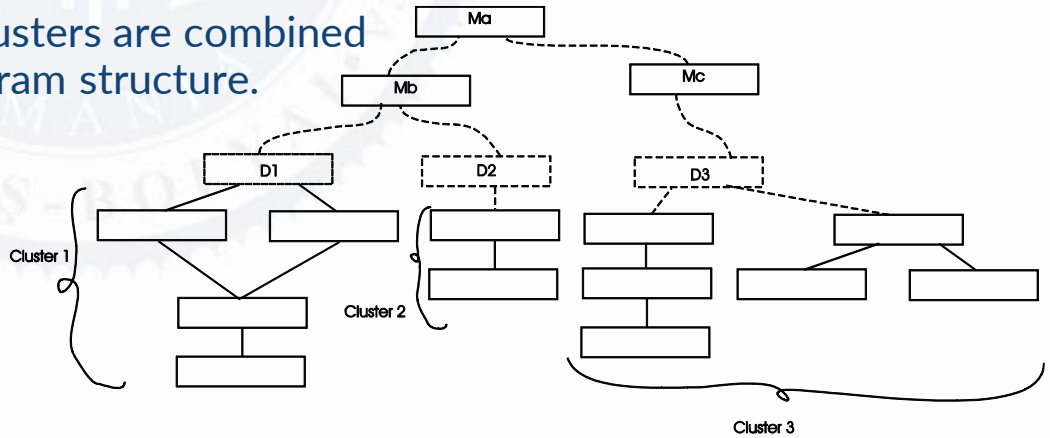


Levels of testing

2. Integration testing (cont)

Bottom-up incremental testing

- Bottom-up integration process:
  - low-levels components are combined into clusters;
  - a driver is written to coordinate test case input and output;
  - the cluster is tested;
  - drivers are removed and clusters are combined moving upward in the program structure.

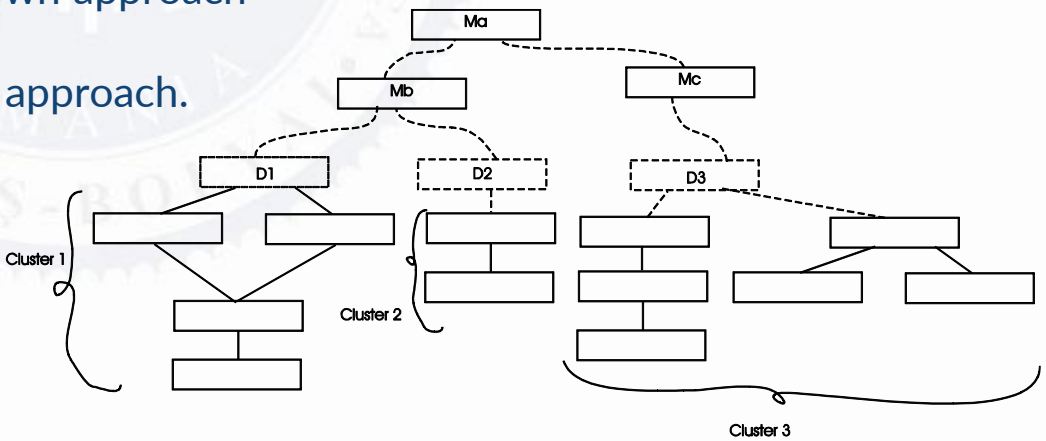


Levels of testing

2. Integration testing (cont)

Sandwich testing

- Sandwich procedures:
  - mix of the top-down and bottom-up approaches;
  - layers of a hierarchical system:
    - bottom-layer – using bottom-up module integration;
    - top-layer - using top-down approach integration;
    - middle-layer - big-bang approach.



Levels of testing

3. Function testing

- testing requirements described in the external specification of the system;
- a process of attempting to find discrepancies between the program and the external specification.
- a black-box activity
- uses system specification

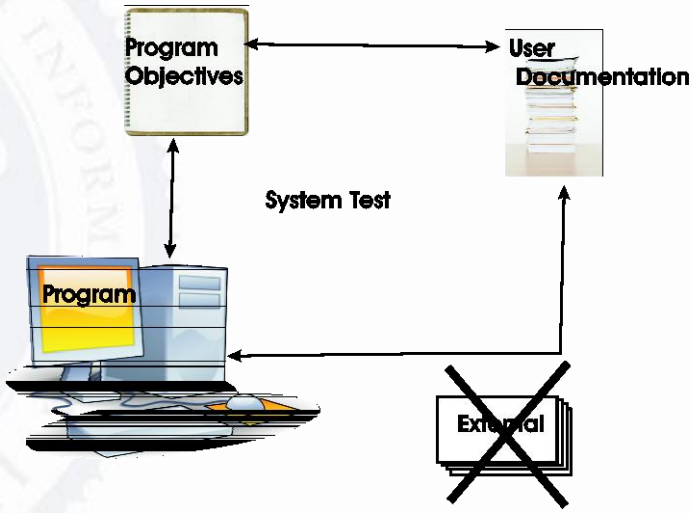
- References: [Mye04] (chapter 6), [NT05] (chapter 9), [PY08] (chapter 10).

Levels of testing

3. System testing

- compare the program - original objectives.
- Use external specification? no, may appear defects during the process of translating the objectives in external specifications;
- Use objectives documents? no, do not contain exact description of the external interfaces of the program;
- Use program's user documentation

- References:[Mye04] (Chapter 6), [NT05] (chapter 8), [PY08] (chapter 22).



Levels of testing

3. System testing (cont)

- the objectives does not offer information about the functionality of the system (interfaces of the modules being tested)
- there is no methodology for created test cases in system testing???
- the process of creating test cases: use imagination, creativity and experience
- References:[Mye04] (Chapter 6), [NT05] (chapter 8), [PY08] (chapter 22).

Levels of testing

3. System testing (cont)

System testing types

- In [Mye04] (Chapter. 6) there are 15 types of system testing:
  - Facility testing
  - Volume testing
  - Usability testing
  - Recovery testing
  - **Security testing** – Details in next lectures (Master Software engineering – Advanced Topics in Software Testing (optional course)
  - Stress testing
  - **Performance testing** – Details – IT firm EVOZONE – Lecture 5 invitation in 25 March 2025 (Room 6/II and not C335)
  - Storage testing
  - Configuration testing
  - Compatibility testing
  - Instability testing
  - Reliability testing
  - Serviceability testing
  - Documentation testing
  - Procedure testing

Levels of testing

4. Acceptance testing

- a process of comparing the program to its initial requirements and the current needs of its end user;
- not the responsibility of the development organization;
- the customer first performs an acceptance test to determine whether the product satisfies its needs.

- References: [NT05] (chapter 14), [PY08] (chapter 22).

Outline

- Software development process
  - Software development process
  - Development and testing processes
- Levels of testing
  - Unit testing
  - Integration testing
  - Function testing
  - System testing
  - Acceptance testing
- Retesting vs regression testing

- Next lecture:
  - **Bogdan Iudean Lecture on CI/CD Pipelines**
  - **Bonus points - Research Education Study**
- Questions

Testing level vs. Testing type

Testing level	Testing type
• set of activities that are associated to a phase of the software development product	• the mean by which an objective of a testing level is achieved

Examples

- Testing a function – unit level or integration level by bbt(domain)/wbt
- Testing of a non-functional characteristic – at system level by performance testing or usability testing
- Testing after eliminating a bug – at any level after debbuging/correcting the bug by appying retesting, confirmation testing
- Testing relating to eliminating a bug – at any level by regression testing to verify if the elimination of the bug doesn't have side-effects

Retesting (confirmation testing)

- Retesting
  - Execution of the test cases that revealed a bug that was reported
  - Goal – confirmation that the bug was eliminated
- Test cases – are the same with those already executed

Regression testing

- Regression testing - **the re-execution of some subsets of tests that have already been conducted** to ensure that changes have not propagated unintended side effects.
- Regression test suits - classes of test cases:
  - Tests to exercise all software functions.
  - Tests that focus on software functions that are likely to be affected by the change.
  - Tests that focus on the software components that have been changed.
- Reference: [PY08] (chapter 22).

- Regression testing – **new test cases** (Cem Kaner) [BBST]

Having fun learning about testing During Lecture 3

1. Guerilla testing
2. Dumb monkey testing
3. Smoke testing
4. Bug bashes (in testing)

1 A4 page information
Work in teams

- Definition/description
- Characteristics (5 to 10 points)
- Levels of testing
- Example = simple + real world application
- Interesting fact(s)

You are **NOT ALLOWED** to include "guest names."
Every person listed as a collaborator must contribute.

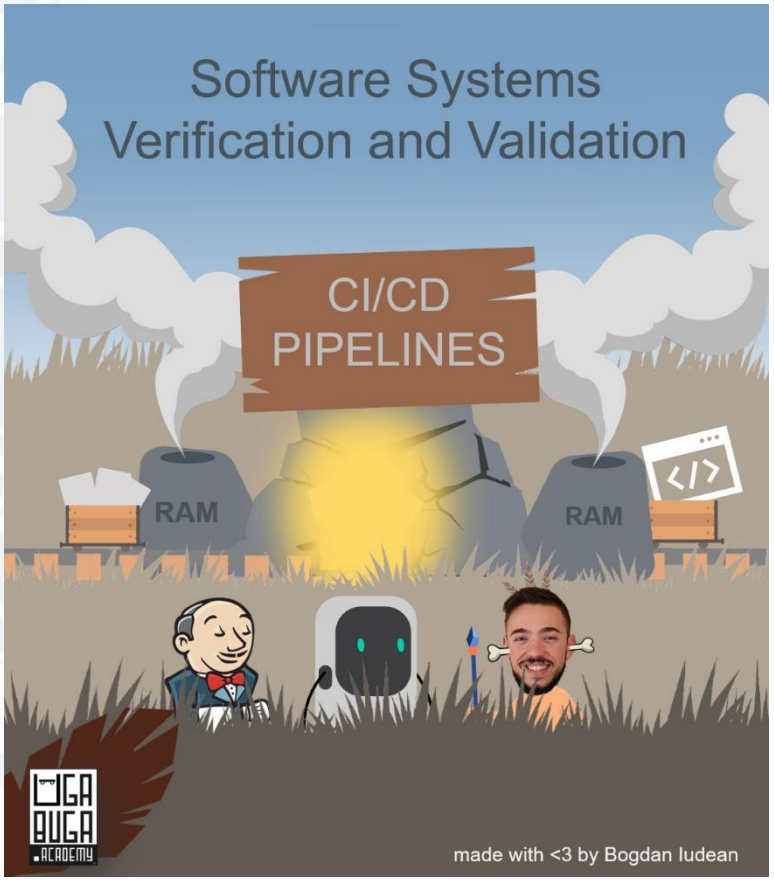
- **Poster - creating 25 XP**
  - **15 minutes to Create the poster**
  - **Each Team presents in class in 2 minutes**
    - **4 Testing Topics *5minutes=20 minutes**
  - **Submitted in Teams in BonusLecture3**
- **Poster – discussion Debriefing**
  - **5-10 minutes**
  - **Learning**
  - **Individual/Team**

Outline

Mentimeter

- Software development process
  - Software development process
  - Development and testing processes
- Levels of testing
  - Unit testing
  - Integration testing
  - Function testing
  - System testing
  - Acceptance testing
- Retesting vs regression testing

- Next lecture:
  - **Bogdan Iudean Lecture on CI/CD Pipelines**
  - **Bonus points - Research Education Study**
- Questions



References

- [Pat05] R. Patton. Software Testing. Sams Publishing, 2005.
- [PY08] M. Pezzand and M. Young. *Software Testing and Analysis: Process, Principles and Techniques*. John Wiley and Sons, 2008.
- [Mye04] Glenford J. Myers, *The Art of Software Testing*, John Wiley & Sons, Inc., 2004
- [You79] E. Yourdon, Structured Walkthroughs, Prentice-Hall,Englewood Cliffs, NJ, 1979
- [NT05] K. Naik and P. Tripathy. *Software Testing and Quality Assurance*. Wiley Publishing, 2005.
- [CB03] Jean-Francois Collard and Ilene Burnstein. *Practical Software Testing*. Springer-Verlag New York, Inc., 2003.
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010
- [BBST] BBST Testing course, http://testingeducation.org/BBST/

Next Lecture

- Bogdan Iudean Lecture on CI/CD Pipelines
- Bonus points - Research Education Study

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Test case design -White-box testing

- Continuous integration - Jenkins

Outline

- Testing - fundamental questions
- Testing strategy
- Levels of testing – Unit testing
- White -box testing
  - Control Flow Graph (CFG)
  - Cyclomatic complexity
  - Logic-Coverage Testing [Mye04] (statement, branch/decision, condition, decision-condition , multiple-condition coverage)
    - Path coverage criterion [NT05] (All-Path, Statement, Branch , Predicate Coverage Criterion)
    - Additional White box test design approaches [CB03] (Independent Path , Loop testing)
  - Advantages/Disadvantages
- Example - White-box testing
  1. Design of the test cases
  2. Maven project
  3. Jenkins – continuous integration tool

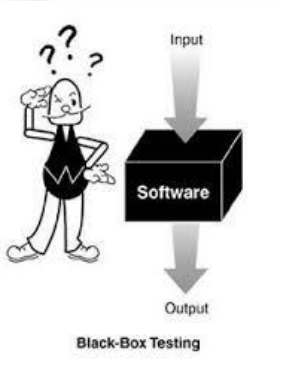
Testing - fundamental questions

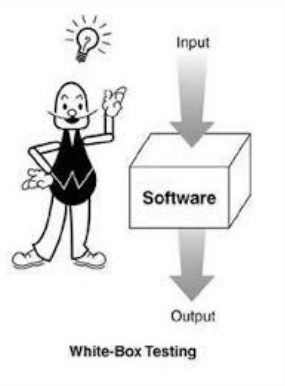
- **Why do we test?**
  - We test a product to learn about its quality. [BBST]
- **How do we organize the process of testing?**
  - Testing strategy problem
- **When have we tested enough?**
  - Testing measuring problem

Testing strategies [BBST]

- **Testing strategy is:**
  - The guiding framework for deciding what tests (what test techniques) are best suited to your product.
  - **Context** and **information objectives** are (or should be) the drivers of any **testing strategy.**

- **Selecting the Testing techniques ?**
  - Techniques differ in core.





Story ...

- Rewrite story ...
  Little Red Riding Hood!

Little Red Riding Hood!

- Story
  - http://www.eastoftheweb.com/short-stories/UBooks/LittRed.shtml

- **Input: RRH, W**
  - **Preconditions: RRH shouldn't tell strangers her direction.**
- **Result: r**
  - **Postconditions: (r=True and RRH shouldn't be late and RRH should arrive at grandma's house successfully) or (r=False and RRH is late at grandma)**
- **Algorithm NewRedRidingHood(RRH, Wolf, r) is:**

@ r= False
@ Red Riding Hood(RRH) receives basket for the grandma.
@ RRH starts the journey in the wood.
@ RRH meets the Wolf (W)
@ IF (W asked RRH about her direction)
@ RRH answers: "To my grandmother's!"
@ W suggested to pick up flowers.
@ If (RRH decides to pick up flowers)
@ She is late for her grandma.
@ W eats her grandma.
@ r = False
@ Else
@ She is not late for her grandma.
@ W does not eat her grandma.
@ RRH arrives at grandma's house successfully.
@ r = True
@ Else
@ r = True

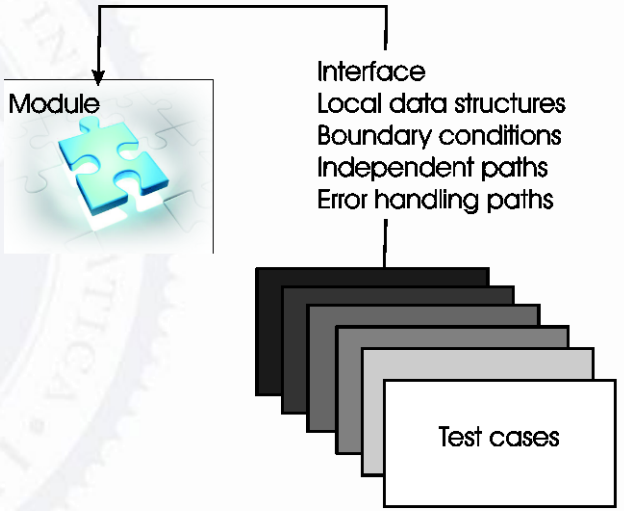
- **Is the correct algorithm for "safe" (r=True) version of the story?**

- Bug 1: RRH answers: "To my grandma" (actual result) but the expected result (RRH shouldn't tell strangers her direction)
- Bug 2: RRH decides to pick up flowers, so she is going to be late and thus the grandma is eaten

Levels of testing – Unit testing

Test case design

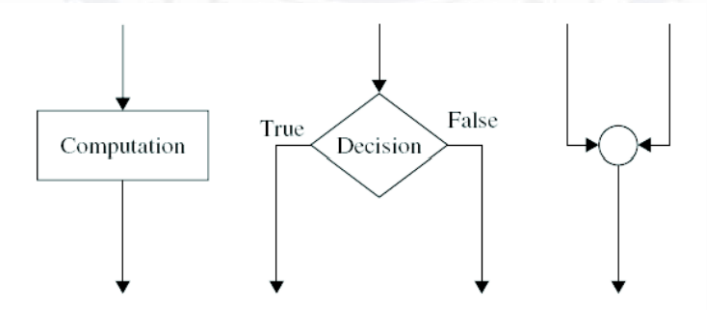
- Information needed when designing test cases for a module:
  - specification of the module
  - the module's source code
- Test case design procedure for a module test is:
  - Applying black-box methods to the module's specification.
  - Analyze the logic of the module using white-box methods.



White-box testing

Control Flow Graph

- A Control Flow Graph (CFG) is a graphical representation of a program unit.
- A CFG has exactly one entry node and exactly one exit node.
- Three symbols are used to construct a CFG
  - nodes - sequential statements, decision and looping predicates
  - edges - represent transfer of control
- Path in the CFG [NT05] - is represented as a sequence of computation and decision nodes from the entry node to the exit node.
- An independent path [CB03] is any path through the program that introduces at least one new set of processing statements or a new condition.



White-box testing

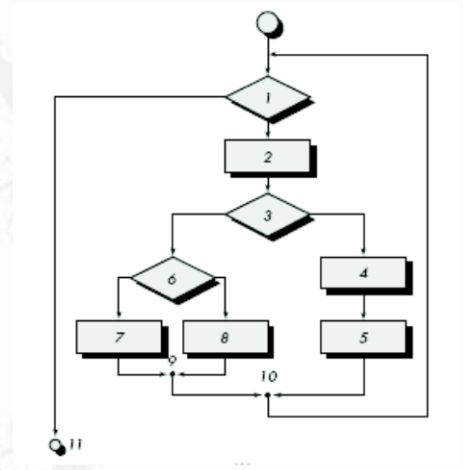
Cyclomatic complexity

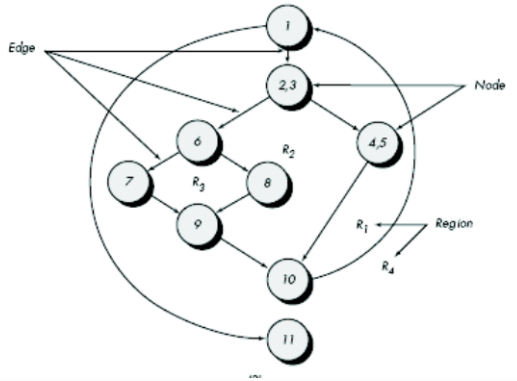
- Cyclomatic complexity
  - The number of independent paths in the basis set of a program and provides us with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.
  - CC = The number of regions of the flow graph.
  - CC = E - N + 2, where E - #edges, N - #nodes.
  - CC = P+1, where P - #predicate nodes

White-box testing

Cyclomatic complexity

- CC
  - CC = four regions = 4.
  - CC = 11 edges - 9 nodes + 2 = 4.
  - CC = 3 predicate nodes + 1 = 4.
- A set of independent paths:
  - path 1: 1-11.
  - path 2: 1-2-3-4-5-10-1-11.
  - path 3: 1-2-3-6-8-9-10-1-11.
  - path 4: 1-2-3-6-7-9-10-1-11.





Outline

- Testing - fundamental questions
- Testing strategy
- Levels of testing – Unit testing
- White -box testing
  - Control Flow Graph (CFG)
  - Cyclomatic complexity
  - Logic-Coverage Testing [Mye04] (statement, branch/decision, condition, decision-condition , multiple-condition coverage)
    - Path coverage criterion [NT05] (All-Path, Statement, Branch , Predicate Coverage Criterion)
    - Additional White box test design approaches [CB03] (Independent Path , Loop testing)
  - Advantages/Disadvantages
- Example - White-box testing
  1. Design of the test cases
  2. Maven project
  3. Jenkins – continuous integration tool

White-box testing

Logic-Coverage Testing [Mye04]

- [Mye04] – "… the ultimate white-box test is the execution of every path in the program, but complete path testing is not a realistic goal for a program with loops…."
- Select the minimum number of test cases such that we achieve:
  1. statement coverage
  2. branch/decision coverage
  3. condition coverage
  4. decision-condition coverage
  5. multiple-condition coverage

White-box testing

Logic-Coverage Testing [Mye04]

1. Statement coverage (sc)

- **Goal**: to execute every statement in the program at least once.
- Complete statement coverage is the weakest coverage criterion in program testing.
  - Any test suite that achieves less than statement coverage for new software is considered to be unacceptable.

White-box testing

Logic-Coverage Testing [Mye04]

2. Decision (branch) coverage (dc)

- A branch is an ongoing edge from a node.
  - All the rectangle nodes have at most one ongoing branch, except the exit node.
  - All the diamond nodes have two outgoing branches.
- Covering a branch means selecting a path that includes the branch.
- Complete branch coverage means selecting a number of paths such that every branch is at least one path.
  - selecting enough number of paths such that every condition evaluates to true at least once and to false at least once.

White-box testing

Logic-Coverage Testing [Mye04]

2. Decision (branch) coverage (dc) - issues

- Remark: dc ➔ sc
  - Why? Since every statement is on one subpath emanating from a branch statement or from the entry point of the program, every statement must be executed if every branch direction is executed.
- Exceptions:
  - Programs with no decisions.
  - Programs with multiple entry points. A given statement might be executed only if the program is entered at a particular entry point.
- A branch with multiple conditions - some decisions may remain uncovered.
  - if (a == 2 || b > 1) < statement > .
  - if the second condition it was written b < 1 by mistake, then the test case with a=2 wouldn't discover the error!

White-box testing

Logic-Coverage Testing [Mye04]

3. Condition coverage (cc)

- **Goal**: to write enough test cases to ensure that each condition in a decision takes on all possible outcomes at least once.
- cc ➔ dc (in general).
  - cc may cause (but does not always) every individual condition in a decision to be executed with both outcomes.
- Exceptions:
  - if (A&&B) < statement >
  - cc ➔ TC1 for A true, B false, and TC2 for A false and B true
  - But the statement is not executed (dc for True is not covered!
  - ➔there is a need for decision/condition coverage

White-box testing

Logic-Coverage Testing [Mye04]

4. Decision/condition coverage (dcc)

- **Goal**: requires sufficient test cases that:
  - each condition in a decision takes on all possible outcomes at least once;
  - each decision takes on all possible outcomes at least once;
  - each point of entry is invoked at least once.
- dc ➔ cc (in general)
- Exceptions:
  - When certain condition mask other conditions
  - Results of conditions in && and || expressions can mask or block the evaluation of other conditions (i.e. if an && condition is false then none of subsequent conditions in the expression need to be evaluated)
  - Thus, errors in logical expressions are not necessarily revealed by the condition-coverage and decision/condition coverage criteria

White-box testing

Hierarchy of strengths for sc, dec, cdc

- From weakest to strongest: sc, dc, cdc.
- The implication for this approach to test design is that the stronger the criterion, the more defects will be revealed by the tests.
- In most cases the stronger the coverage criterion, the larger the number of test cases that must be developed to ensure complete coverage.
- ➔the tester must decide (based on the type of code, reliability requirements, resources available) which criterion to select!

White-box testing

Logic-Coverage Testing [Mye04]

5. Multiple condition coverage (mcc)

- Goal: write sufficient test cases that:
  - all possible combinations of condition outcomes in each decision, and
  - all points of entry are invoked at lest once.
- mcc ➔ dcc (in general)
- Remark: A set of test cases satisfying the multiple-condition criterion also satisfies the decision coverage, condition coverage, and decision/condition coverage criteria.

White-box testing

Minimum test criterion

- For programs containing only one condition per decision:
  - Test cases to evoke all outcomes of each decision at least once, and
  - Test cases to invoke each point of entry at least once, to ensure that all statements are executed at least once.
- For programs containing decisions having multiple conditions:
  - Test cases to evoke all possible combinations of condition outcomes in each decision, and
  - all points of entry to the program, at least once.

White-box testing

Path coverage criterion [NT05]

- [NT05] – "A path is represented as a sequence of computation and decision nodes from the entry node to the exit node."

1. All-Path coverage criterion
2. Statement coverage criterion
3. Branch coverage criterion
4. Predicate Coverage Criterion

White-box testing

Path coverage criterion [NTO5]

1. All-Path coverage criterion

- The all-path selection criterion
  - is desirable but it is difficult to achieve in practice
  - is achievable but not practical

➔ reduced number of paths.

- Structural criteria are applied based on statements, edges and paths.

White-box testing

Path coverage criterion [NTO5]

2. Statement coverage criterion

- See [Mye04]

White-box testing

Path coverage criterion [NTO5]

3. Branch coverage criterion

- See [Mye04]

White-box testing

Path coverage criterion [NTO5]

3. Predicate coverage criterion

- There is a need to design test cases such that a path is executed under all possible conditions.

- If all possible combinations of truth values of the conditions affecting a selected path have been explored under some tests, then we say that *predicate coverage* has been achieved.
  - Lecture - In Class Work
  - 5 minutes – Read document "PredicateCoverage.pdf" posted on Teams – Lecture 3.
  - Post the solution for Figure 4.9a) for Predicate Coverage.
  - Post the solution for Figure 4.9b) for Predicate Coverage.

White-box testing

Additional White box testing design approaches [CB03]

- [CB03] – "A path is represented as a sequence of computation and decision nodes from the entry node to the exit node."
    1. Independent Path coverage criterion
    2. Loop testing

White-box testing

Additional White box testing design approaches [CB03]

1. Independent Path coverage [CB03]

- Path in the CFG [NT05] - is represented as a sequence of computation and decision nodes from the entry node to the exit node.
- An independent path [CB03] is any path through the program that introduces at least one new set of processing statements or a new condition.
  - ➔Construct the set of independent paths for a graph.
    - ➔This set is called: ..... [CB03]
- ➔Remark:
  - ➔coverage based on independent path testing ? complete path coverage

White-box testing

Additional White box testing design approaches [CB03]

2. Loop testing [CB03]

- Simple loops - n is the maximum number of allowable passes through the loop:
  - Skip the loop entirely.
  - Only one pass through the loop.
  - Two passes through the loop.
  - m passes through the loop where m < n.
  - n-1, n, n + 1 passes through the loop.
- Nested loops
  - Start at the innermost loop. Set all other loops to minimum values.
  - Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter .
  - Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values.
  - Continue until all loops have been tested.

White-box testing

Advantages	Disadvantages
• Code coverage • Testing can be commenced at an earlier stage. • Find the fault.	• A skilled tester is needed to carry out this type of testing. • No ambiguities in spec. may be found. • After code is written.

White-box testing

Example

- **Problem statement**: Compute the number of participants with the maximum score (0 to 100 points possible) at a competition.

- Applied:
  - Construction of the CFG.
  - CC metric
  - Coverage: statements, conditions/decisions, paths, loops.
- See example files
- **Lecture03_Demo**

- Maven
  - goal - to allow a developer to comprehend the complete state of a development effort in the shortest period of time
  - https://maven.apache.org/what-is-maven.html
- Jenkins
  - Continuous integration tool

White-box testing

Example

- Maven
  - goal - to allow a developer to comprehend the complete state of a development effort in the shortest period of time
  - https://maven.apache.org/what-is-maven.html
  - The Surefire Plugin is used during the test phase of the build lifecycle to execute the unit tests of an application.
    - https://maven.apache.org/surefire/maven-surefire-plugin/index.html
- Jenkins
  - Continuous integration tool

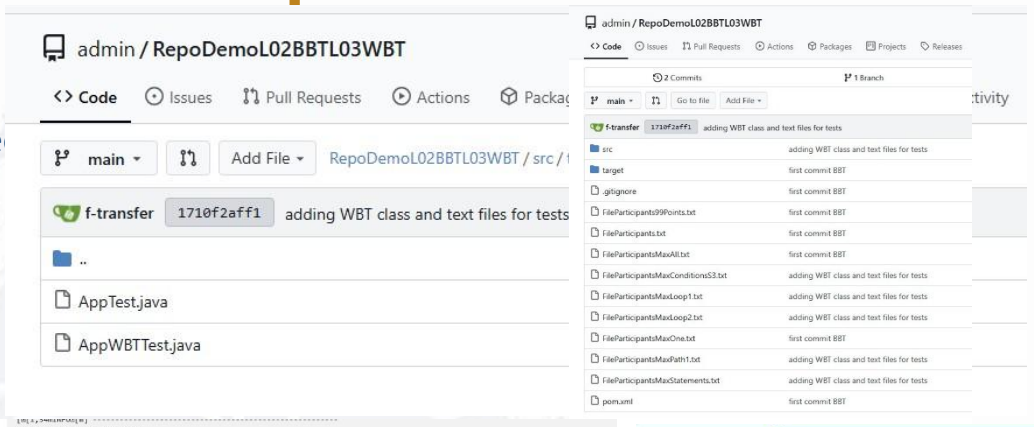
- File Info_CI_CD.pdf
  - Docker and Docker compose
- Create Repository in Gitea
- Add the source code to this project
- Create the Pipeline job
- Run the Job

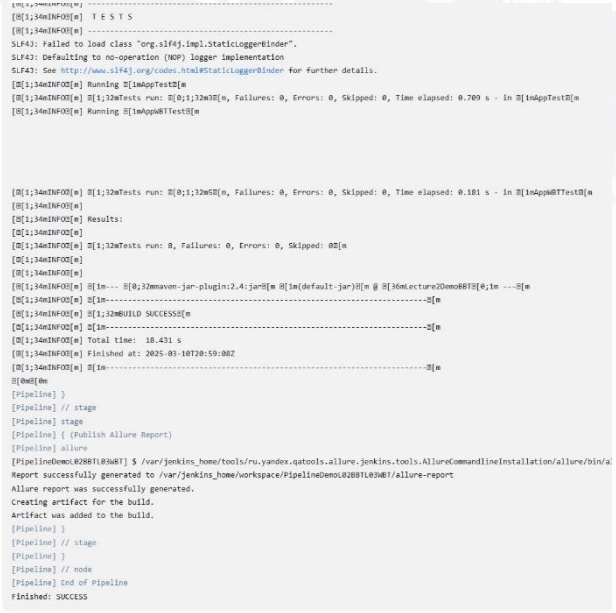
White-box testing

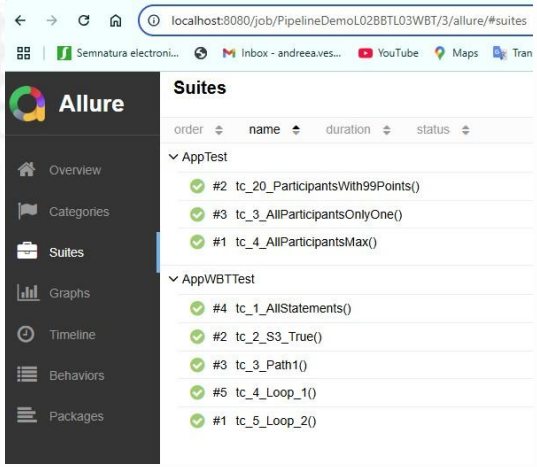
Example

- File Info_CI_CD.pdf
  - Docker and Docker compose
- Create Repository in Gitea
- Add the source code to this proje
- Create the Pipeline job
- Run the Job

```
d:\temp\ssvv2025\RepoDemoL02BBTL03WBT>git commit -m "adding WBT class and text files for tests"
[main 1710f2a] adding WBT class and text files for tests
 6 files changed, 86 insertions(+)
 create mode 100644 FileParticipantsMaxConditionsS3.txt
 create mode 100644 FileParticipantsMaxLoop1.txt
 create mode 100644 FileParticipantsMaxLoop2.txt
 create mode 100644 FileParticipantsMaxPath1.txt
 create mode 100644 FileParticipantsMaxStatements.txt
 create mode 100644 src/test/java/AppWBTTest.java
```







Outline

- Testing - fundamental questions
- Testing strategy
- Levels of testing – Unit testing
- White -box testing
  - Control Flow Graph (CFG)
  - Cyclomatic complexity
  - Logic-Coverage Testing [Mye04] (statement, branch/decision, condition, decision-condition , multiple-condition coverage)

    - Path coverage criterion [NT05] (All-Path, Statement, Branch , Predicate Coverage Criterion)

    - Additional White box test design approaches [CB03] (Independent Path , Loop testing)
  - Advantages/Disadvantages
- Surprise!
- Example - White-box testing
  1. Design of the test cases
  2. Maven project
  3. Jenkins – continuous integration tool

References

- [Pat05] R. Patton. Software Testing. Sams Publishing, 2005.
- [PY08] M. Pezzand and M. Young. *Software Testing and Analysis: Process, Principles and Techniques*. John Wiley and Sons, 2008.
- [Mye04] Glenford J. Myers, *The Art of Software Testing*, John Wiley & Sons, Inc., 2004
- [You79] E. Yourdon, Structured Walkthroughs, Prentice-Hall,Englewood Cliffs, NJ, 1979
- [NT05] K. Naik and P. Tripathy. *Software Testing and Quality Assurance*. Wiley Publishing, 2005.
- [CB03] Jean-Francois Collard and Ilene Burnstein. *Practical Software Testing*. Springer-Verlag New York, Inc., 2003.
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010
- [BBST] BBST Testing course, http://testingeducation.org/BBST/
  - **Foundations of Software Testing**
    - **Lecture 5: The Impossibility of Complete Testing**
- Tutorials - SSVV lecture's homepage.
  - www.cs.ubbcluj.ro/~avescan

Next Lecture (Today)

- Levels of testing

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

4.5.4 Predicate Coverage Criterion

We refer to the partial CFG of Figure 4.9*a* to explain the concept of predicate coverage. OB1, OB2, OB3, and OB are four Boolean variables. The program computes the values of the individual variables OB1, OB2, and OB3— details of their computation are irrelevant to our discussion and have been omitted. Next, OB is computed as shown in the CFG. The CFG checks the value of OB and executes either OBlock1 or OBlock2 depending on whether OB evaluates to true or false, respectively.

We need to design just two test cases to achieve both statement coverage and branch coverage. We select inputs such that the four Boolean conditions in Figure 4.9*a* evaluate to the values shown in Table 4.6. The reader may note that we have shown just one way of forcing OB to true. If we select inputs so that these two cases hold, then we do not observe the effect of the computations taking place in nodes **2** and **3**. There may be faults in the computation parts of nodes **2** and **3** such that OB2 and OB3 always evaluate to false.

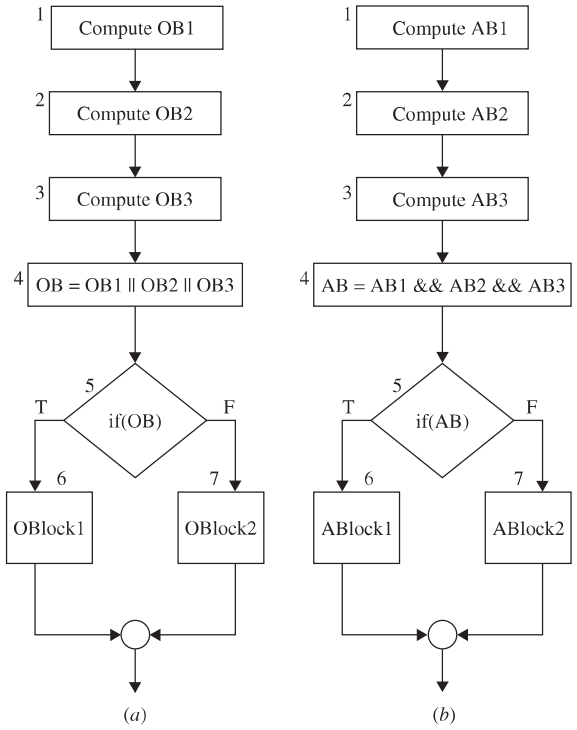


Figure 4.9 Partial CFG with (*a*) OR operation and (*b*) AND operation.

TABLE 4.6 Two Cases for Complete Statement and Branch Coverage of CFG of Figure 4.9*a*

Cases	OB1	OB2	OB3	OB
1	T	F	F	T
2	F	F	F	F

Therefore, there is a need to design test cases such that a path is executed under all possible conditions. The False branch of node **5** (Figure 4.9*a*) is executed under exactly one condition, namely, when OB1 = False, OB2 = False, and OB3 = False, whereas the true branch executes under *seven* conditions. If all possible combinations of truth values of the conditions affecting a selected path have been explored under some tests, then we say that *predicate coverage* has been achieved. Therefore, the path taking the true branch of node **5** in Figure 4.9*a* must be executed for all seven possible combinations of truth values of OB1, OB2, and OB3 which result in OB = True.

A similar situation holds for the partial CFG shown in Figure 4.9*b*, where AB1, AB2, AB3, and AB are Boolean variables.

4.6 GENERATING TEST INPUT

In Section 4.5 we explained the concept of path selection criteria to cover certain aspects of a program with a set of paths. The program aspects we considered were all statements, true and false evaluations of each condition, and combinations of conditions affecting execution of a path. Now, having identified a path, the question is how to select input values such that when the program is executed with the selected inputs, the chosen paths get executed. In other words, we need to identify inputs to force the executions of the paths. In the following, we define a few terms and give an example of generating test inputs for a selected path.

1. *Input Vector:* An input vector is a collection of all data entities read by the routine whose values must be fixed prior to entering the routine. Members of an input vector of a routine can take different forms as listed below:

- Input arguments to a routine
- Global variables and constants
- Files
- Contents of registers in assembly language programming
- Network connections
- Timers

A file is a complex input element. In one case, mere existence of a file can be considered as an input, whereas in another case, contents of the file are considered

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

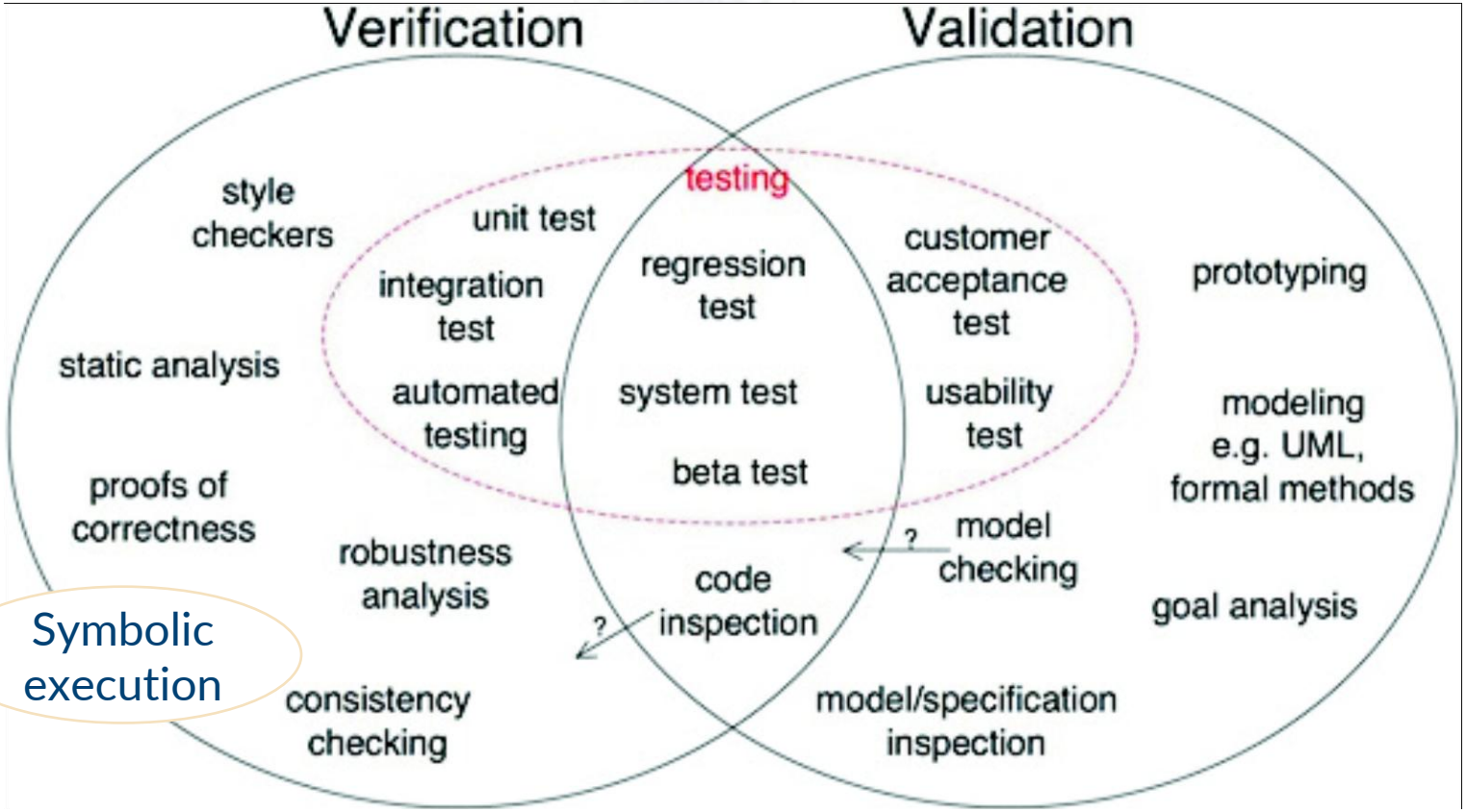
(Next)/Today Lecture

- Agile testing
- Exploratory testing

Outline

- Agile Testing
  - Video presentations – 10 minutes
- Exploratory testing
  - Definition
  - About
  - Video presentations and links
  - Session Based Test Management [SBTM]
    - About
    - Ready for rent?
      - 5 min. - about
      - 25 min. activity
      - 10 min. debriefing
      - 10 min. - questionnaire

What we will learn!



Outline

- Agile Testing
  - **Video presentations – 10 minutes**

- **Video presentations (from 2019)**
  - **Created by a team**
  - **Each video ~ 5 minutes**
    - **Audio + written + visual**
  - **Send by email the link for your video – 1 day before the presentation**

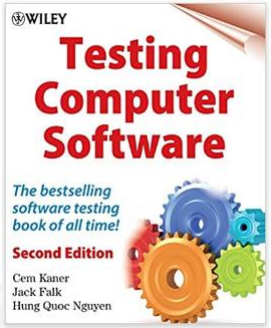
- Team 1
- **Team 2 - 3:44 – Agile testing - quadrants**
- **Team 3 - 5:27 – Agile testing - methods**
- Team 4

Outline

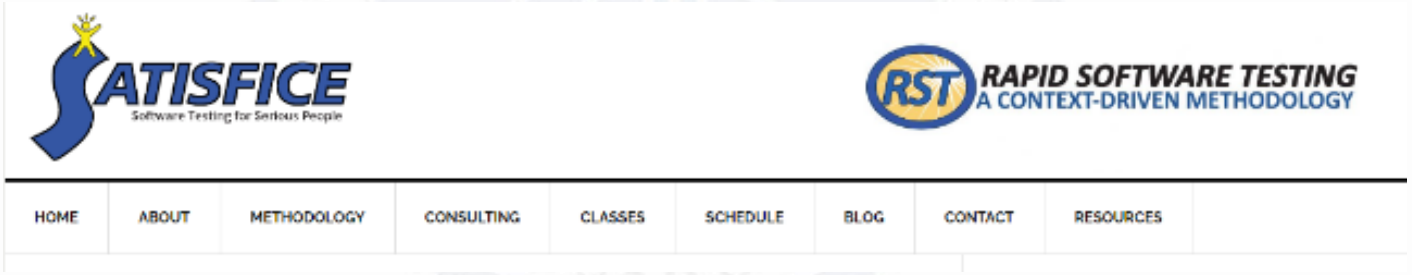
- Agile Testing
  - Video presentations – 10 minutes
- **Exploratory testing**
  - Definition(s)
  - About
  - Video presentations and links
  - Session Based Test Management [SBTM]
    - About
    - Ready for rent?
      - 5 min. - about
      - 25 min. activity
      - 10 min. debriefing
      - 10 min. – questionnaire

Exploratory testing – Definition(s)

- History of definitions of Exploratory testing
  - https://www.satisfice.com/blog/archives/1504

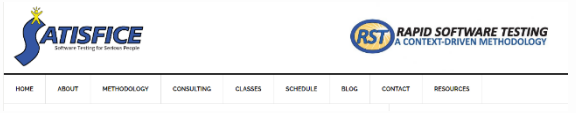


- James Bach, Test consultant and founder of Satisfice Inc., helps understand Exploratory testing in an exclusive interview. (see next slide)
  - https://www.youtube.com/watch?v=I-ItEKt_N_s&ab_channel=MindtreeLtd.

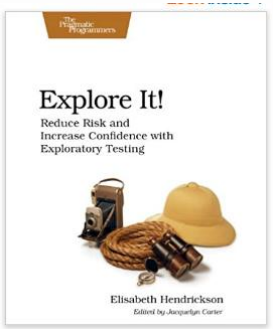


Exploratory testing – About

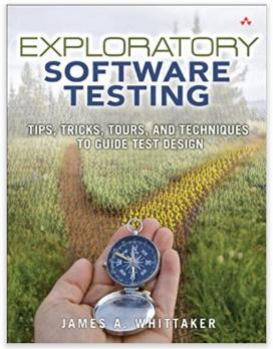
- James Bach, Test consultant and founder of Satisfice Inc., helps understand Exploratory testing in an exclusive interview. (6 min. video)
  - https://www.youtube.com/watch?v=I-ItEKt_N_s&ab_channel=MindtreeLtd.



- Elisabeth Hendrickson
  - An explanation of the Who, What, When, Why, and How of exploratory testing. (total 40 min.) Seconds 25''-2min10'',
    Experiential Learning 35''40'' (1 minute)
  - https://www.youtube.com/watch?v=9FKY1Is0lgs&ab_channel=ElisabethHendrickson

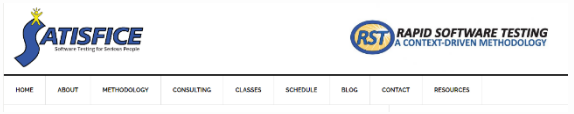


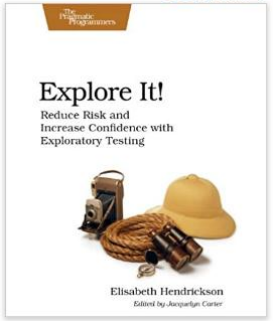
- James Whittaker
  - Large-scale Exploratory Testing: Let's Take a Tour (total 47 min.)
  - https://www.youtube.com/watch?v=fNkYz1hB7r0&ab_channel=TechWellHappenings
- Maaret Pyhajarvi
  - https://www.youtube.com/watch?v=yXupa6Gtt3E&ab_channel=BrowserStack
  - Min. 34min (34'40'') to 37 min

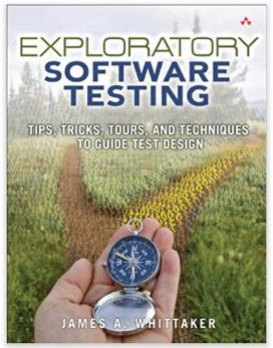


Exploratory testing – Video presentations and links

- History of definitions of Exploratory testing
  - https://www.satisfice.com/blog/archives/1504
- James Bach, Test consultant and founder of Satisfice Inc., helps understand Exploratory testing in an exclusive interview. (6 min. video)
  - https://www.youtube.com/watch?v=I-ItEKt_N_s&ab_channel=MindtreeLtd.
- Elisabeth Hendrickson
  - An explanation of the Who, What, When, Why, and How of exploratory testing. (40 min.)
  - https://www.youtube.com/watch?v=9FKY1Is0lgs&ab_channel=ElisabethHendrickson
- James Whittaker
  - Large-scale Exploratory Testing: Let's Take a Tour (47 min.)
  - https://www.youtube.com/watch?v=fNkYz1hB7r0&ab_channel=TechWellHappenings
- Maaret Pyhajarvi
  - https://www.youtube.com/watch?v=yXupa6Gtt3E&ab_channel=BrowserStack
  - (44 min.)







Outline

- Agile Testing
  - Video presentations – 10 minutes
- Exploratory testing
  - Definition(s)
  - About
  - Video presentations and links
  - **Session Based Test Management [SBTM]**
    - About
    - Ready for rent?
      - 5 min. - about
      - 25 min. activity
      - 10 min. debriefing
      - 10 min. - questionnaire

Session Based Test Management [SBTM]

- A method [SBTM] for measuring and managing **exploratory testing**.
- http://www.satisfice.com/sbtm/
- http://www.satisfice.com/articles/sbtm.pdf
- An approach to testing with:
  - No test cases
  - No expected results
  - No pass/fails metrics
- Testing on a project is divided up into focused "sessions" of exploratory testing.
- A test session
  - A dedicated and uninterrupted period of exploratory testing.
  - Focused on a set objectives defined in a "charter"
  - Generates notes and lightweight reporting on the testing carried out.
  - The testing task is expected to change and evolve.

- **Exploratory testing**
  - **Is not random**
  - **Is creative and requires intelligent thoughts.**

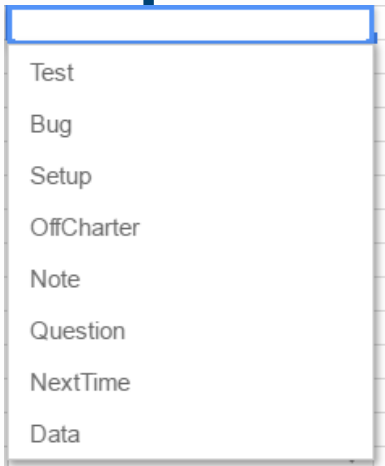
Session Based Test Management [SBTM]

- **Planning**
  - Charter: a single sentence that sets the scope of the testing
    - What you will test
    - How you plan to test it
    - The information you hope to provide.
  - That scope should be achieved in 90 minutes.
  - Charter are basis of estimation and coverage.
- **Testing**
  - A dedicated and uninterrupted period of testing (no meetings, no email)
  - Use testing experience or instinct; use your domain knowledge
  - Test scenarios could be ok, but not pre-scripted steps
  - Use the software, observe it and be alert of new questions or test ideas
  - Encourage thinking beyond specific features or tasks in isolation.
  - Fully imagine and describe users of the software
- **Note taking/ Reporting**
  - Not just pass/fail count: What you did. Why you did it. What you saw. What questions and ideas you had. Any frustrations/blocks.

Session Based Test Management [SBTM]

- Jonathan Bach – course on Session Bases Test Management
- Example:
  - Applied: Sanity and Survey Testing Focus Writer.
  - Software: Focus Writer.

- **Computed metrics**

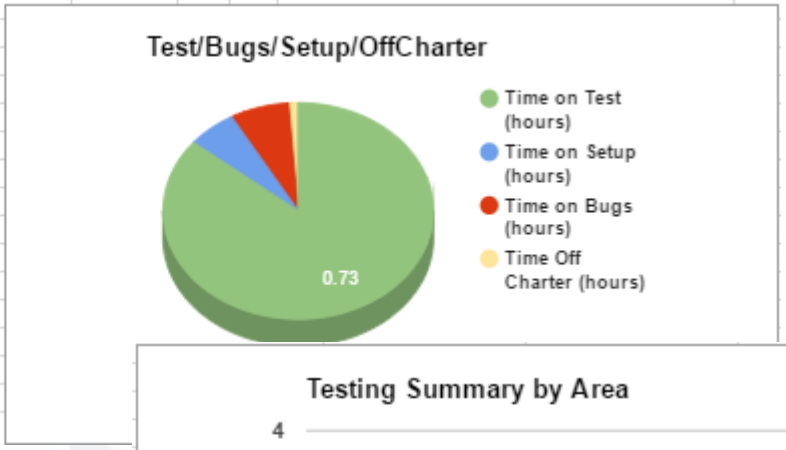


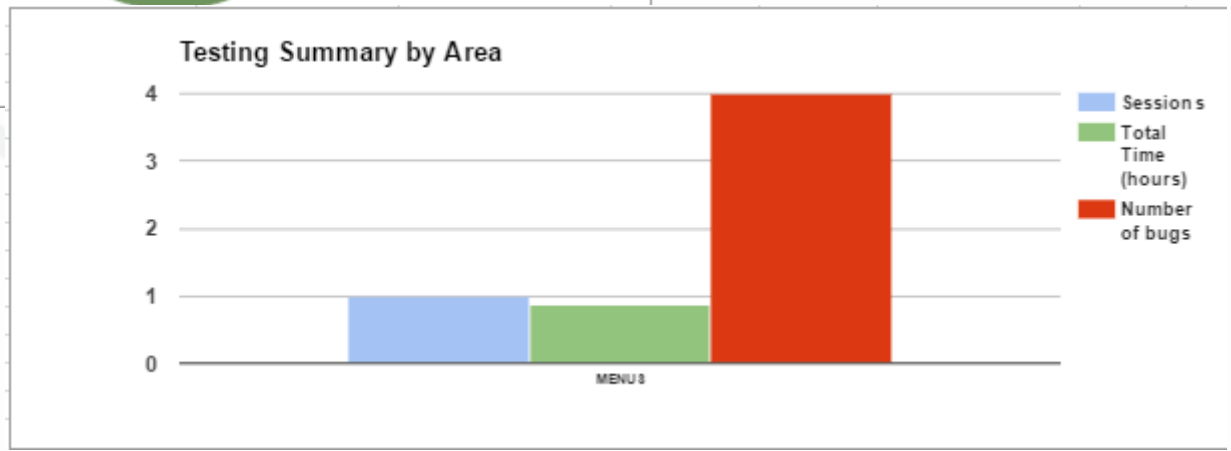
Session end.Duration		52
Number of bugs		4
Time on Test (mins)		44
Time on Setup (mins)		3
Time on Bugs (mins)		3
Time Off Charter (mins)		1
Number of questions		5
Number of ideas for next time		2
Session Tester	alexandra.casapu@altom.ro	
Session Charter	Perform Sanity Testing and Survey Testing for FocusWriter	
Planned session time (mins)		60
Environment Info	Windows, Product Version: FocusWriter 1.6.2, Mac OSX version 1.6.2, on MacOSX Sierra	
Area	Menus	
Current Active Tag	Test	
Time remaining (mins):		9
Session start	SESSION ENDED!	

Session Based Test Management [SBTM]

- Jonathan Bach – course on Session Based Test Management [RapidSoftwareTestingCourse_Altom]
- Example:
  - Applied: Sanity and Survey Testing Focus Writer.
  - Software: Focus Writer.

Overall Summary	
Total Sessions	1
Total Time (hours)	0.87
Total Bugs	4
Time on Test (hours)	0.73
Time on Setup (hours)	0.05
Time on Bugs (hours)	0.06
Time Off Charter (hours)	0.01
Total Questions	5
Total Next Time Ideas	2



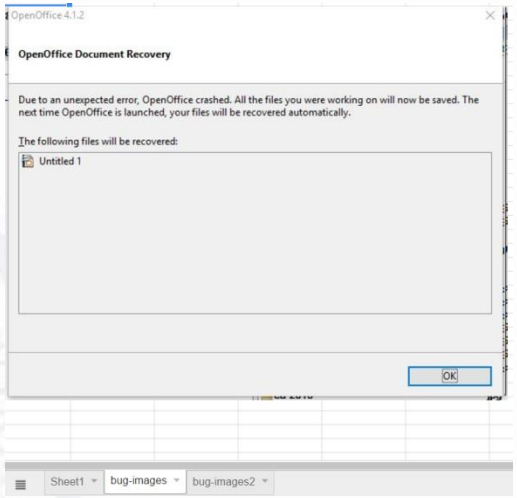


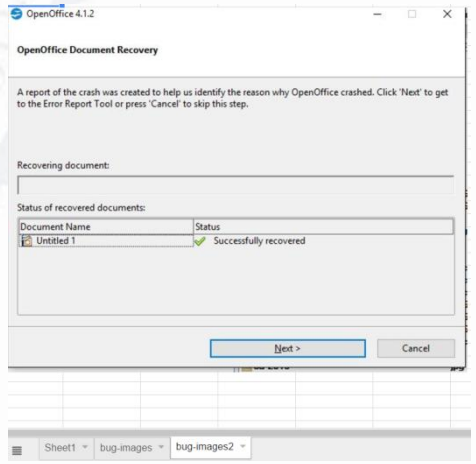
Session Based Test Management [SBTM]

- Tabara de testare 2016
- Example:
  - Software: OpenOfficeImpress.
- **Computed metrics**

Session Tester	andreea.vescan@gmail.com	
Session Charter	AndreeaVescan	
Planned session time (mins)	30 minutes	
Environment Info	OpenOfficeImpress	
Area	Sa aflam daca sunt buguri in Impress care ar putea determina utilizatorii sa foloseasca solutii similare	
Current Active Tag	Bug	
Time remaining (mins):	#NUM!	
Session start	START ADDING NOTES BELOW	
Test	create new template with a special fornt for letters (to tell an emotional story by a professional trainer/presenter) to raise the impact of the content	
Note	Am creat un myOwnTemplate si am incercat sa vad daca pentru mine am gasit un tip de font apropiat de scrisul de mana si am gasit...satisfacator	
Bug	nu s-a salvat formatul creat de mine	vezi imaginile de la bugs
Note	am incercat sa caut pe internet fonturi care sa existe in ppt si nu in openoffice care sa fie in format de scrisoare,m-am uitat si in lista de font de la ppt dar la prima vedere nu verificat cateva care mi-au placut daca se gasesc si in openoffice	
Test	am verificat pe internet daca au mai fost alti utilizatori nemultumiti de fonturile existente in openoffice	
Note	ce tipuri de utilizatori sunt? cat de prioritar pentru ei era sa fie fontul cerut? Cat de critic?	
Note	how to install new fonts in openoffice, v	
Note		
Test	cauta pe forumuri despre bud publicity for openoffice	
Note	https://forum.openoffice.org/en/forum/viewtopic.php?f=10&t=33191	
Note	Difficulty aligning bullets in slide master	

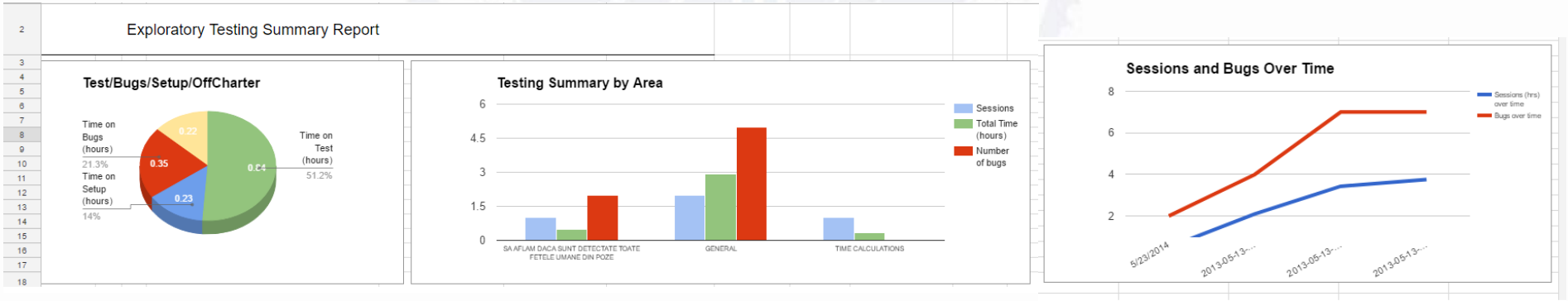
bug-images | bug-images2





Session Based Test Management [SBTM]

- Use SBTM for an open source application
  - **Suggestion**: Use SBTM for your Bachelor Thesis.
- SBTM template available here:
  - https://altom.com/version-2-1-of-the-sbtm-session-template-was-released/
  - **Use your gmail account.**
  - Instructions how to use the template
    - https://altom.com/sbtm-with-itester-and-google-drive/



Grading – Seminar (2)

Portofolio activities

- **1. Portofolio about Inspection**
  - **Details provided in Lecture 1**
  - **To be submitted in Seminar 3**
  - **Team: 3 persons/team**
- **2. Portofolio about Test Case Design**
  - **Details provided in Lecture 3**
  - **To be submitted in Seminar 4**
  - **Team: 3 persons/team**
- **3. Portofolio about Exploratory Testing**
  - **Details provided in Lecture 7 (6)**
  - **To be submitted in Seminar 5**
  - **Team: 3 persons/team**

Session Based Test Management [SBTM]

- Demo: Session Testing
- Mission: Ready For Rent?
  - 5 min. - about
  - **25 min. activity**
  - **5 min. results + voting**
  - 10 min. – questionnaire (+ Debriefing)
    - **Debriefing: PROOF**
      - **Past + Results + Outlook + Obstacles + Feeling**
- Bonus points = votes
  - I: 25 XP
  - II: 15 XP
  - III: 5 XP

Mentimeter

- Each team – at home (by Friday, 11 April 2025, h: 20:00)
  - Create the template with your results, downloaded and submitted in teams under assignment **Bonus_SSVV_Exploratory_study**
- SBTM template available here:
  - https://altom.com/version-2-1-of-the-sbtm-session-template-was-released/
  - **Use your gmail account.**
  - Instructions how to use the template
    - https://altom.com/sbtm-with-itester-and-google-drive/

Session Based Test Management

Ready For Rent?

Mission = find "bugs" in the way the house was constructed that make the users (for rent, for buy) to search another house.

Users = Family with 3 kids (2 toddlers + 1 teenager)

Time = 20 minutes

Session Testing

Test	
Bug	
Setup	
Offchart	
Note	
Next time	
Question	

Debriefing = P R O O F

Past Result Outlook Obstacles Feeling

Outline

- Agile Testing
  - Video presentations – 10 minutes

- Exploratory testing
  - Definition
  - About
  - Video presentations and links
  - Session Based Test Management [SBTM]
    - About
    - Ready for rent?
      - 5 min. - about
      - 25 min. activity
      - 10 min. – questionnaire
      - 10 min. debriefing

References

- Lisa Crispin, Janet Gregory, **Agile Testing: A Practical Guide for Testers and Agile Teams (Addison-Wesley Signature),** Addison-Wesley Professional; 1 edition (January 9, 2009)

- Janet Gregory, Lisa Crispin, **More Agile Testing: Learning Journeys for the Whole Team (Addison-Wesley Signature Series (Cohn)) ,** Addison-Wesley Professional; 1 edition (October 16, 2014)

- [SBTM] Jonathan Bach, **Session Based Test Management,** http://www.satisfice.com/sbtm/
  http://www.satisfice.com/articles/sbtm.pdf

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

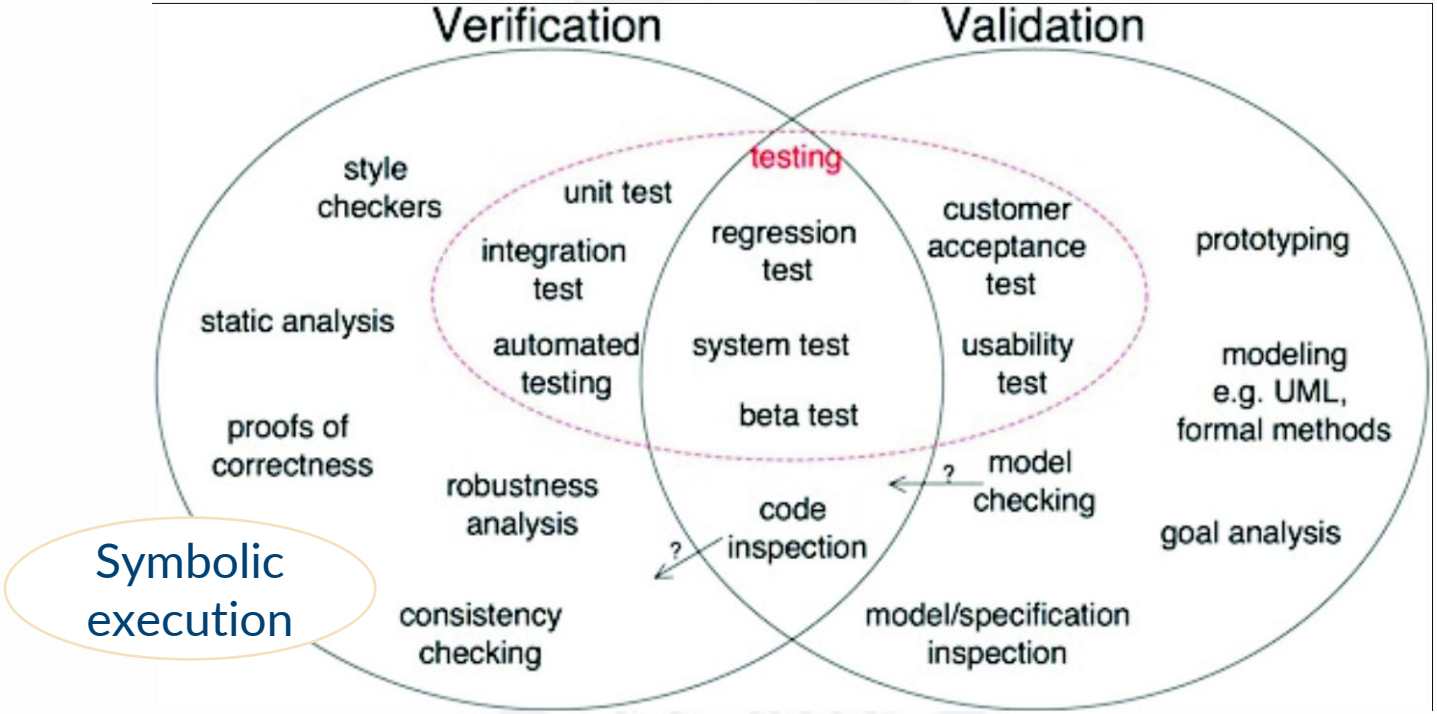
"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Symbolic execution

What we will learn!



- http://www.easterbrook.ca/steve/2010/11/the-difference-between-verification-and-validation/

Outline

- Static analysis, Testing, Symbolic execution
- Conventional vs Symbolic execution
- Symbolic execution for sequential, alternative, repetitive structures
  - Sequential structure execution
  - Alternative structure execution
  - Repetitive structure execution
- Symbolic Execution Tree
  - Symbolic Execution Tree
  - Properties
- Questions

- Next lecture
  - Model checking

Static analysis
Symbolic execution

- Bugs that are missed by testing: rare features, rare circumstances, nondeterminism.
  - ➔ Static analysis
    - Can analyze all possible runs of a program
    - But can it find deep, difficult bugs?
      - Abstraction let us model all possible runs
      - Static analysis abstraction <> developer abstraction
- Testing works
  - reported bugs are real bugs, but each test only explores **one** possible execution. asssert (f(5)==6)
  - We ***hope*** test cases generalize, but no guarantees!
  - ➔Symbolic execution **generalizes** testing
    - ➔y=α, assert(f(y)==2*y+1)
- **Remarks**:
  - symbolic execution is not meant to inspect the quality of the code.
    - static analysis deals with issues of path feasibility,
    - dynamic analysis tends to deal with path coverage.
  - Symbolic analysis is sort of in between and deals with state space explosion by logically forking the analysis at branches and solving for a set of satisfiable constraints.

Symbolic execution - research

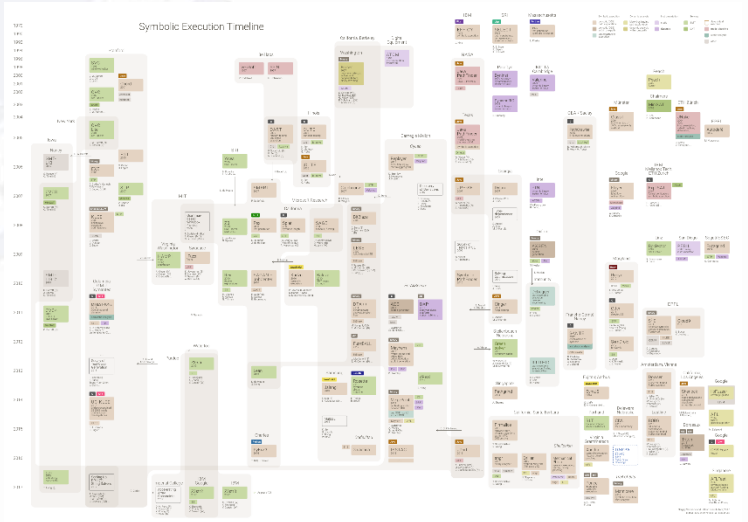
- **1976 - King [Kin76], Clarke [Cla76]**
- ....
- 2005 - Microsoft: DART [God05]
- 2006 - Univ. Stanford: EXE, Univ. Illinois: CUTE and jCUTE [SA06]
- ...
- 2008 - KLEE (Stanford) [CDE08]
- ...
- 1999 - 2016 - NASA: Symbolic (Java) Path Finder [PV09], [CS13]
  - http://javapathfinder.sourceforge.net/
  - http://babelfish.arc.nasa.gov/trac/jpf

- Modern Symbolic Execution Techniques
  - mix concrete and symbolic execution
  - Concolic testing (DART – Directed Automated Random Testing)
  - EGT (Execution-Generated Testing)
- 2017 -Learn&Fuzz: Machine Learning for Input Fuzzing
  - https://patricegodefroid.github.io/
- 2018
  - Chopped Symbolic Execution (ICSE) (2006 -EXE)
  - Shadow Symbolic Execution for Testing Software Patches
  - https://www.doc.ic.ac.uk/~cristic/
- 2018 -Deep Reinforcement Fuzzing, Konstantin Böttinger, Patrice Godefroid, Rishabh Singh
- 2022 – SBSE conference - Fuzzing vs SBST Intersections and Differences

- SAGE (2005 -DART)
  - https://patricegodefroid.github.io/
  - https://channel9.msdn.com/blogs/peli/automated-whitebox-fuzz-testing-with-sage -

    video (11 minutes)
- https://www.microsoft.com/en-us/security-risk-detection/
- PEX
  - https://www.microsoft.com/en-us/research/project/pex-and-moles-isolation-and-white-box-unit-testing-for-net/?from=http%3A%2F%2Fresearch.microsoft.com%2Fen-us%2Fprojects%2Fpex%2F

- Symbolic execution timeline



Symbolic execution

What is symbolic execution ?

- Symbolic execution
  - Execution of program with symbols as argument.
  - Symbolic execution supplies symbols (as input to a program) representing arbitrary values.
  - int FunctionName(1, 2) ➔ int FunctionName(a1 , a2)

- The execution proceeds as in a normal execution except that values may be symbolic formulae over the input symbols.

- Symbolic execution
  - Produces a concrete input (a test case) on which the program will fail to meet the specification.
  - But it cannot, in general, prove the absence of errors

- Key idea
  - Evaluate the program on symbolic input values
  - Use an automated theorem prover to check whether there are corresponding concrete input values that make the program fail.

Symbolic execution

Symbolic state

- **Symbolic state**
  - Set of (particular) concrete states, yet not instantiated.
  - Symbolic states represent sets of concrete states.

- A **symbolic state** is described by:
  - **Variables**, i.e. symbolic values/expressions for variables;
  - **Path condition** - a conjunct of constraints on the symbolic input values;
  - **Program counter** - the statement that is executed.
- All paths in the program form its execution tree, in which some paths are feasible, and some are infeasible.
- Symbolic execution is a bug finding technique based on automated theorem proving:
  - Evaluates the program on symbolic inputs, and a solver finds concrete values for those inputs that lead to errors.

Outline

- Static analysis, Testing, Symbolic execution
- Conventional vs Symbolic execution
- Symbolic execution for sequential, alternative, repetitive structures
  - Sequential structure execution
  - Alternative structure execution
  - Repetitive structure execution
- Symbolic Execution Tree
  - Symbolic Execution Tree
  - Properties
- Questions

- Next lecture (still today)
  - Model checking

Conventional vs Symbolic execution

Conventional execution (CE)

- Function Sum
- Normal execution result of Sum(1,3,5)
- 1 : int Sum(int a, int b, int c)
- 2 : int x := a + b;
- 3: int y := b + c;
- 4: int z := x + y - b;
- 5: return z;
- 6:

	a	b	c	x	y	z
1	1	3	5	-	-	-
2	1	3	5	4	-	-
3	1	3	5	4	8	-
4	1	3	5	4	8	9
5	1	3	5	4	8	9

Conventional vs Symbolic execution

Symbolic execution (SE)

- Function Sum
- Normal execution result of Sum(1,3,5)
- 1 : int Sum(int a, int b, int c)
- 2 : int x := a + b;
- 3: int y := b + c;
- 4: int z := x + y - b;
- 5: return z;
- 6:

	a	b	c	x	y	z
1	α	β	γ	-	-	-
2	α	β	γ	α+β	-	-
3	α	β	γ	α+β	β+γ	-
4	α	β	γ	α+β	β+γ	α+β+γ
5	α	β	γ	α+β	β+γ	α+β+γ

Symbolic execution

Symbolic execution for **sequential**, alternative, repetitive structures

- **Sequential structure execution**
  - path condition
    - condition to execute a statement;
  - when the symbolic execution starts, the value(pc) = true
  - the condition is updated from one statement to other
    - If τ represents the condition to execute statement < I > then $pc' = pc \wedge \tau(I)$

Symbolic execution

Symbolic execution for **sequential**, alternative, repetitive structures

Conventional - Sequential execution - Symbolic

	a	b	c	x	y	z
1	1	3	5	-	-	-
2	1	3	5	4	-	-
3	1	3	5	4	8	-
4	1	3	5	4	8	9
5	1	3	5	4	8	9

```
1 : int Sum(int a, int b, int c)
2 :     int x := a + b;
3:      int y := b + c;
4:      int z := x + y - b;
5: return z;
6:
```

	a	b	c	x	y	z
1	α	β	γ	-	-	-
2	α	β	γ	α+β	-	-
3	α	β	γ	α+β	β+γ	-
4	α	β	γ	α+β	β+γ	α+β+γ
5	α	β	γ	α+β	β+γ	α+β+γ

Symbolic execution

Symbolic execution for sequential, **alternative**, repetitive structures

- **Alternative structure execution**
- Symbolic execution of an IF statement
  - **if** (η) **then**
    A
    else
    B.
- **During symbolic execution** ➔ value(η) could be true, false, or some symbolic formula over the input symbols.
  ➔ "unresolved" execution of a conditional statement
- Path Condition (Initial value of pc is true)
  - pc → η
  - pc → $\neg\eta$

Symbolic execution

Symbolic execution for sequential, **alternative**, repetitive structures

Conventional - Alternative execution - Symbolic

	x	b	If condition
1	6	-	-
2	6	False	-
3	6	False	6 modulo 2=0
4	6	True	6 modulo 2=0
6	6	True	6 modulo 2=0

```
1 : boolean IsEven(int x)
2 : boolean b := False;
3: If (x modulo 2 ==0) ther
4:        b:=true;
   else
5:        b:=false;
6: IsEven:=b;
```

	x	b	Path condition
1	α	-	True
2	α	False	True
3	α	False	α modulo 2=0
Case (α modulo 2=0) is True			
3	α	False	α modulo 2=0
4	α	True	α modulo 2=0
6	α	True	α modulo 2=0
Case (not (α modulo 2=0)) is True			
5	α	False	not(α modulo 2=0)

Symbolic execution

Symbolic execution for sequential, alternative, **repetitive** structures

- Symbolic execution of a WHILE statement

```
while (η)
    A
endWh;
B
```

- During symbolic execution ➔ value(η) could be true, false, or some symbolic formula over the input symbols.
  - ➔ "unresolved" execution of a conditional statement
- Condition to execute A: pc for executing "while" and η.
- Condition to execute B: pc for executing "while" and $\neg\ \eta$.

Symbolic execution

Symbolic execution for
sequential, alternative, **repetitive** stru

Conventional - Repetitive execution -

	x	y	z	u	While condition
1	5	3	-	-	
2	5	3	1	-	
3	5	3	1	1	
4	5	3	1	1	1<=3
5	5	3	5	1	
6	5	3	5	2	
4	5	3	5	2	2<=3
5	5	3	25	2	
6	5	3	25	3	
4	5	3	5	3	3<=3
5	5	3	75	3	
6	5	3	75	4	
4	5	3	75	4	not 4<=3
7					
8	5	3	75	4	

```
1 : Power(int x, int y, int z)
2 :     z := 1;
3:      u:=1
4:      while(u ≤ y)
5:          z:=z*x;
6:          u:=u+1
7:      endwh;
8:
```

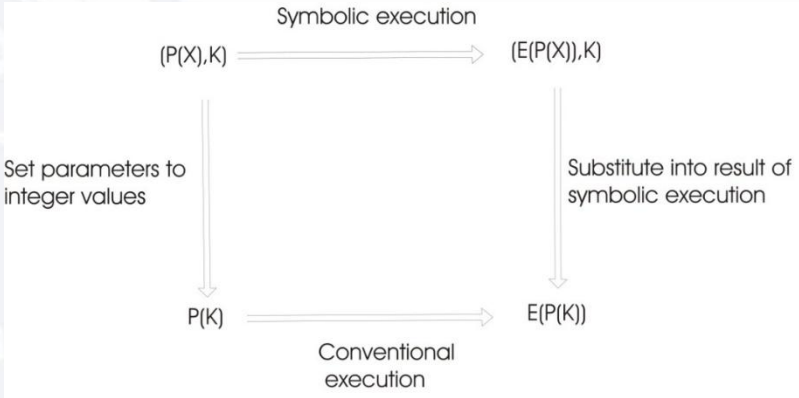
Symbolic

	x	y	z	u	Path condition	Remarks
1	α	β	-	-	True	
2	α	β	1	-		
3	α	β	1	1		
4	α	β	1	1	1<=β	
Case not(1<=β), ➔ 1>β						
4	α	β	1	1	1>β	
8	α	β	1	1		β=0 and z=1
Case (1<=β)						
4	α	β	1	1	1<=β	
5	α	β	α	1	1<=β	
6	α	β	α	2	1<=β	
7						
4	α	β	α	2	2<=β and 1<=β	
Case not(2<=β) and 1<=β, ➔ 2>β and 1<=β						
4	α	β	α	2	2>β and 1<=β	
8	α	β	α	2		β=1 and z=α
Case (2<=β) and 1<=β						
4	α	β	α	2	2<=β and 1<=β	
5	α	β	α^2	2	2<=β and 1<=β	
6	α	β	α^2	3	2<=β and 1<=β	
7						
4	α	β	α^2	3	3<=β and 2<=β and 1<=β	
Case not(3<=β) and 2<=β and 1<=β ➔ 3>β and 2<=β and 1<=β						
4	α	β	α^2	3	3>β and 2<=β and 1<=β	
8	α	β	α^2	3		β=2 and z=α^2

Symbolic execution

Commutativity

- The same result is obtained using normal execution or using symbolic execution.
- Conventional execution (CE)
  - Sum(a, b, c) → Sum(1, 3, 5)
  - Sum(1, 3, 5) = 9
- Symbolic execution (SE)
  - Sum(a, b, c) = $\alpha + \beta + \gamma$
  - Instantiate the symbolic result
    - →$\alpha = 1, \beta = 3, \gamma = 5$
    - →1+3+5=9



Outline

- Static analysis, Testing, Symbolic execution
- Conventional vs Symbolic execution
- Symbolic execution for sequential, alternative, repetitive structures
  - Sequential structure execution
  - Alternative structure execution
  - Repetitive structure execution
- Symbolic Execution Tree
  - Symbolic Execution Tree
  - Properties
- Questions

- Next lecture (still today)
  - Model checking

Symbolic execution

Symbolic Execution Tree

- We can generate symbolic execution tree characterizing the execution paths followed during the symbolic execution.
  - Associate a node with each statement executed.
  - Associate a directed arc connecting the associated nodes with each transition between statements.
  - For IF statement execution, the associated node has two arcs leaving the node which are labeled “T” and “F” for the true and false part, respectively.
  - Associate the complete current execution state, i.e. variable values, statement counter, and pc with each node.

Symbolic execution

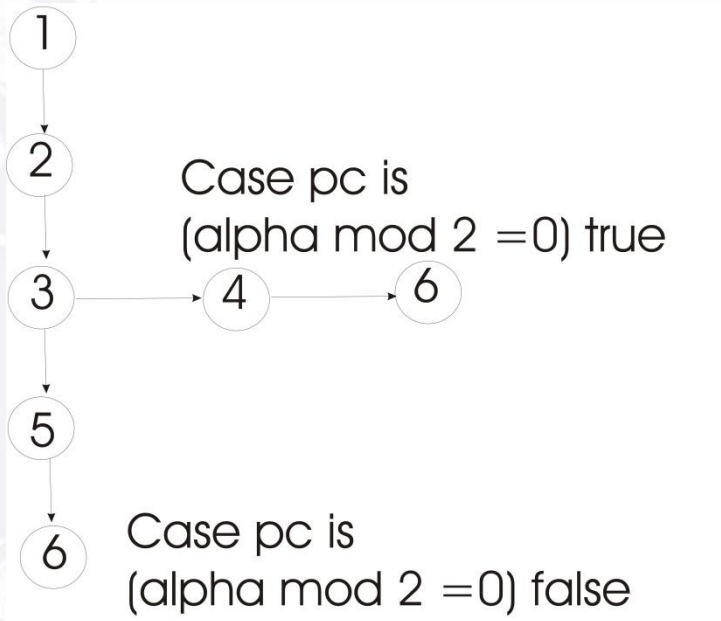
Symbolic Execution Tree

```
1 : int Sum(int a, int b, int c)
2 :     int x := a + b;
3:      int y := b + c;
4:      int z := x + y - b;
5: return z;
6:
```

Symbolic execution

Symbolic Execution Tree

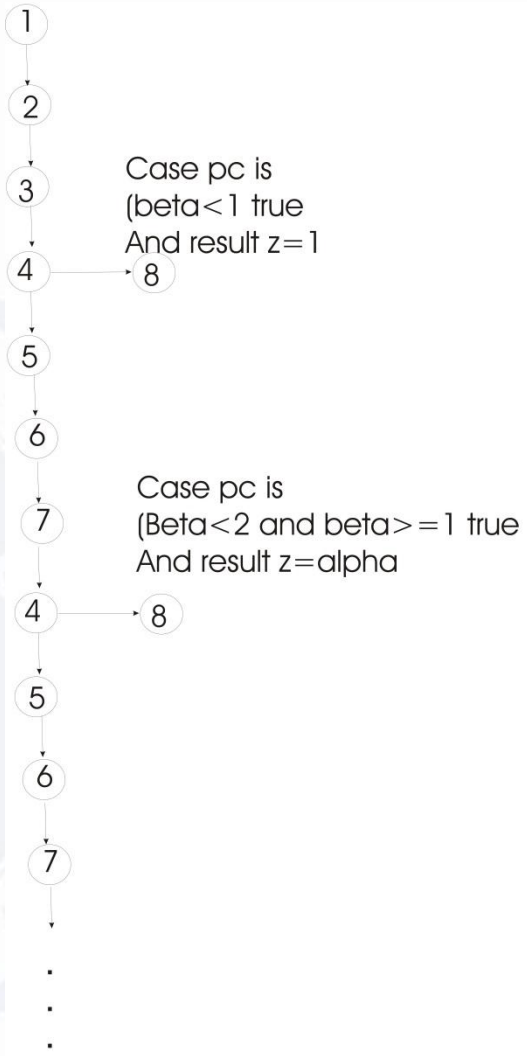
```
1 : boolean IsEven(int a)
2 : boolean b := False;
3: If (x modulo 2 ==0) then
4:        b:=true;
    else
5:        b:=false;
6: IsEven:=b;
```



Symbolic execution

Symbolic Execution Tree

```
1 : Power(int x, int y, int z)
2 :     z := 1;
3:      u:=1
4:      while(u ≤ y)
5:          z:=z*x;
6:          u:=u+1
7:      endwh;
8:
```



Symbolic execution

Properties of the Symbolic Execution Tree

- For each terminal leaf exists a particular non symbolic input.
- The pc associated with any two terminal leaves are distinct.
- Test case generation
  - to execute every statement at least once
  - to include execution of each branch both ways
  - finding input values to reach a particular point in a program

  Symbolic execution
  - Symbolic variables for input variables
  - Execute the program symbolically
  - Collect symbolic path constraints
  - Use constraint solver to generate test inputs for each execution path
- **Remaining problem** - to instantiate the pc with particular values.
- The **pc** specifies **a class of equivalent tests**, and any feasible solution to the constraints (represented by the pc) would be a representative member.
- The symbolic execution also provides expressions describing the program outputs for all inputs in this set.

Symbolic execution

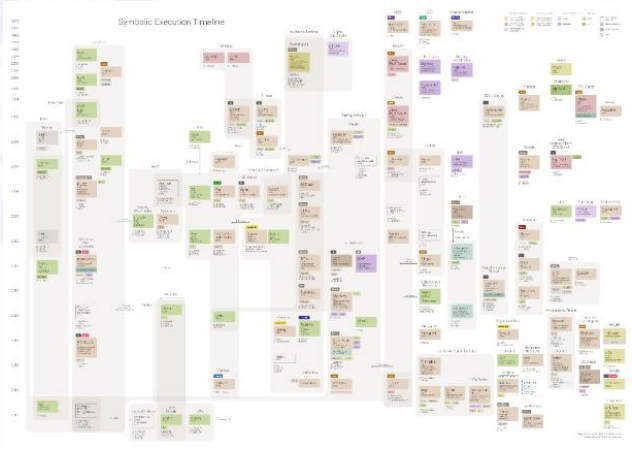
Symbolic execution – research- revisited

- **1976 - King [Kin76], Clarke [Cla76]**
- ....
- 2005 - Microsoft: DART [God05]
- 2006 - Univ. Stanford: EXE, Univ. Illinois: CUTE and jCUTE [SA06]
- ...
- 2008 - KLEE (Stanford) [CDE08]
- ...
- 1999 - 2016 - NASA: Symbolic (Java) Path Finder [PV09], [CS13]
  - http://javapathfinder.sourceforge.net/
  - http://babelfish.arc.nasa.gov/trac/jpf

- Modern Symbolic Execution Techniques
  - mix concrete and symbolic execution
  - Concolic testing (DART – Directed Automated Random Testing)
  - EGT (Execution-Generated Testing)
- 2017 -Learn&Fuzz: Machine Learning for Input Fuzzing
  - https://patricegodefroid.github.io/
- 2018 -Chopped Symbolic Execution (ICSE) (2006 -EXE)
  - https://www.doc.ic.ac.uk/~cristic/
- 2018 -Deep Reinforcement Fuzzing, Konstantin Böttinger, Patrice Godefroid, Rishabh Singh
- 2022 – SBSE conference - Fuzzing vs SBST Intersections and Differences

- SAGE (2005 -DART)
  - https://patricegodefroid.github.io/
  - https://channel9.msdn.com/blogs/peli/automated-whitebox-fuzz-testing-with-sage- **video**
- PEX
  - https://www.microsoft.com/en-us/research/project/pex-and-moles-isolation-and-white-box-unit-testing-for-net/?from=http%3A%2F%2Fresearch.microsoft.com%2Fen-us%2Fprojects%2Fpex%2F

- Symbolic execution timeline



- Fuzzing with Grammars
- Andreas Zeller
- https://www.youtube.com/watch?v=Jc8Whz0W41o&ab_channel=AndreasZeller

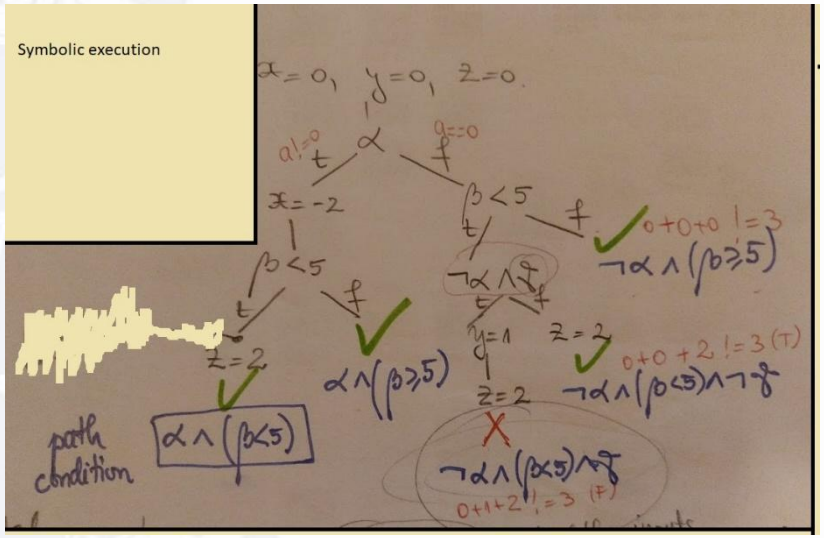
Symbolic execution

https://klee-se.org/

Run KLEE via Docker

Symbolic Execution – example -http://klee.github.io/getting-started/ http://klee.github.io/tutorials/testing-function/

```
// Edit SymbolicExecutionExample.c
void SymbolicExecutionExample(int
a, int b, int c){
    int x=0, y=0, z=0;
    if (a!=0){
        x=-2;
     }
     if (b<5){
        if(!a && c){y=1;}
        z=2;
      }
     assert(x+y+z!=3);
}
```



- http://klee.github.io/getting-started/

KLEE web: run tiny code examples in your browser

Execution and test cases created https://klee.github.io/tutorials/testing-function/

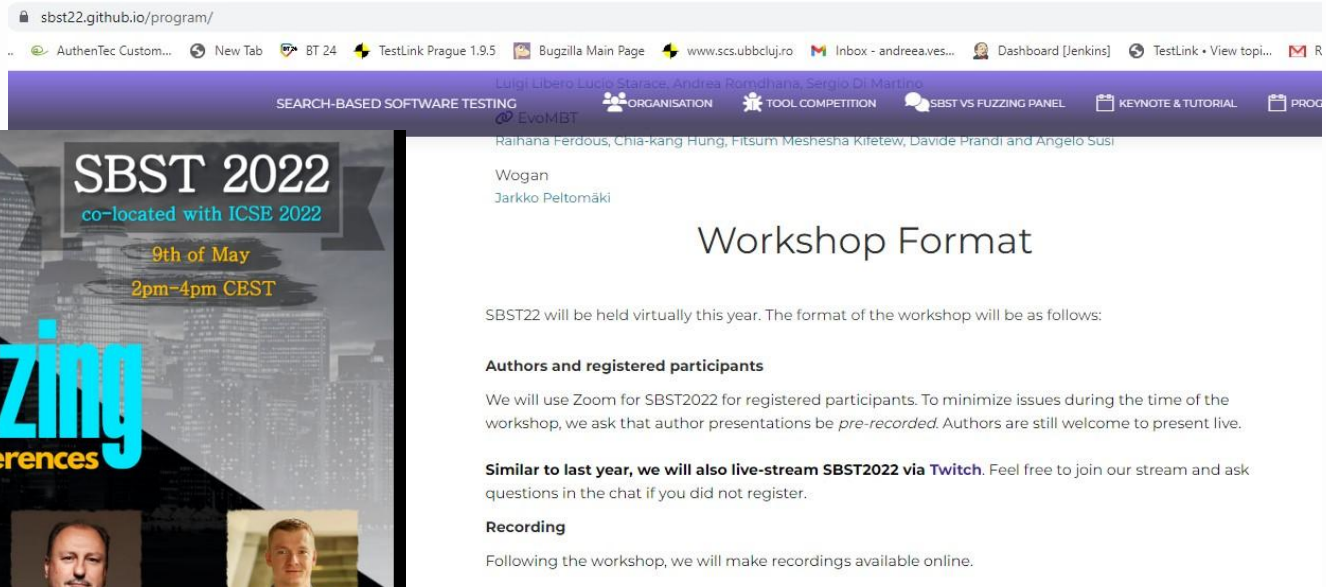
Cristian Cadar

http://www.doc.ic.ac.uk/~cristic/

https://www.youtube.com/watch?v=z6bsk-lsk1Q&ab_channel=AdaLogics

23 minutes

2022 - Fuzzing vs SBST Intersections and Differences



https://www.youtube.com/watch?v=OS0-k7WQkus&t=2s&ab_channel=Search-BasedSoftwareTestingWorkshop%28SBST%29

Outline

- Static analysis, Testing, Symbolic execution
- Conventional vs Symbolic execution
- Symbolic execution for sequential, alternative, repetitive structures
  - Sequential structure execution
  - Alternative structure execution
  - Repetitive structure execution
- Symbolic Execution Tree
  - Symbolic Execution Tree
  - Properties
- Questions

- Next lecture
  - Model checking

References

- [Kin76] James C. King. Symbolic execution and program testing. Commun. ACM, 19(7):385–394, 1976.
- [Cla76] L. A. Clarke. A system to generate test data and symbolically execute programs.
- IEEE Transactions on Software Engineering, SE-2(3):215–222, 1976.
- [God05] P. Godefroid. Dart: directed automated random testing. pages 213–223, 2005.
- [SA06] Koushik Sen and Gul Agha. Cute and jcute: Concolic unit testing and explicit path model-checking tools. In Proceedings of the 18th International Conference on Computer Aided Verification, pages 419–423, 2006.
- [CDE08] Cristian Cadar, Daniel Dunbar, and Dawson Engler. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, pages 209–224, 2008.
- [PV09] Corina S. Pasareanu and Willem Visser. A survey of new trends in symbolic execution for software testing and analysis. Int. J. Softw. Tools Technol. Transf., 11(4):339–353, 2009.
- [CS13] Cristian Cadar and Koushik Sen. Symbolic execution for software testing: Three decades later. Commun. ACM, 56(2):82–90, 2013.

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

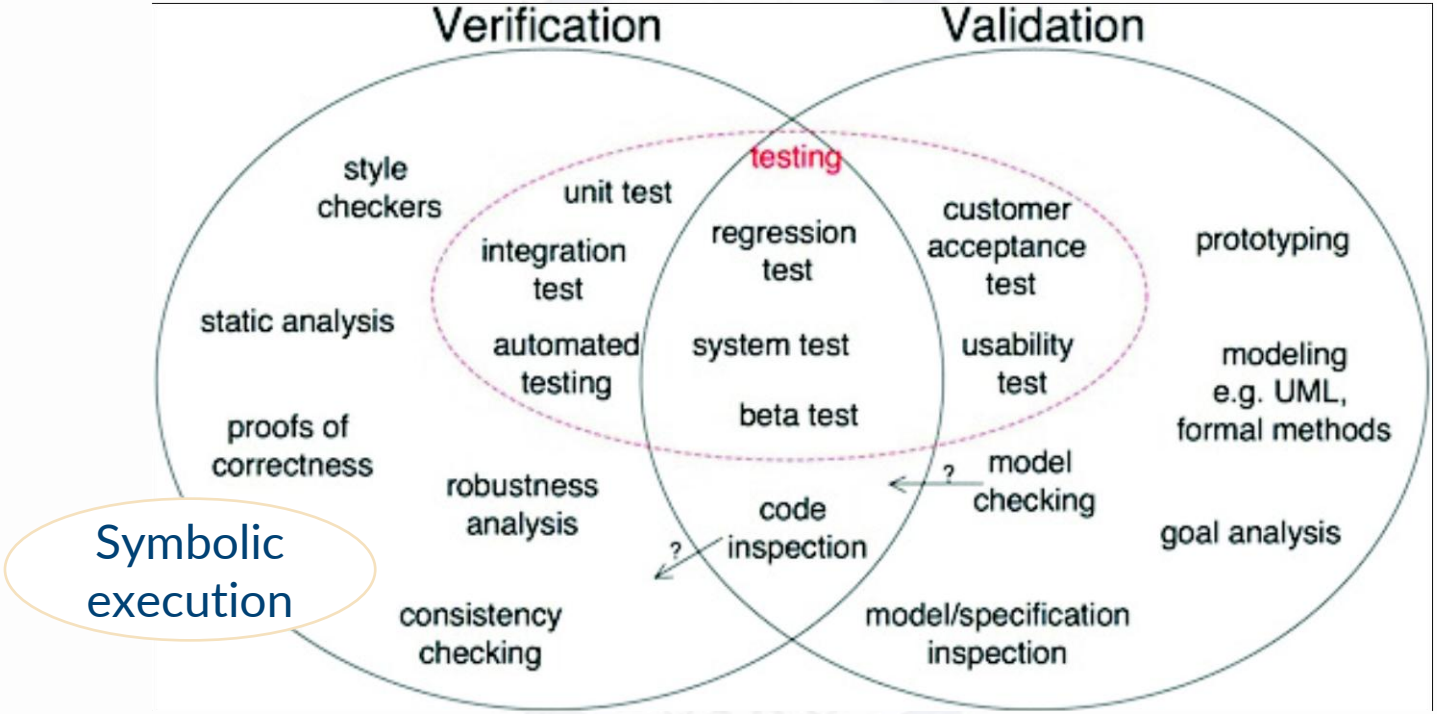
"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Model checking

What we will learn!



- http://www.easterbrook.ca/steve/2010/11/the-difference-between-verification-and-validation/

Outline

Model checking

- System verification
- Model checking
- Transition system
- Linear-Time Properties
- Linear-Time Logic
- Computation Tree Logic

Spin Model Checker

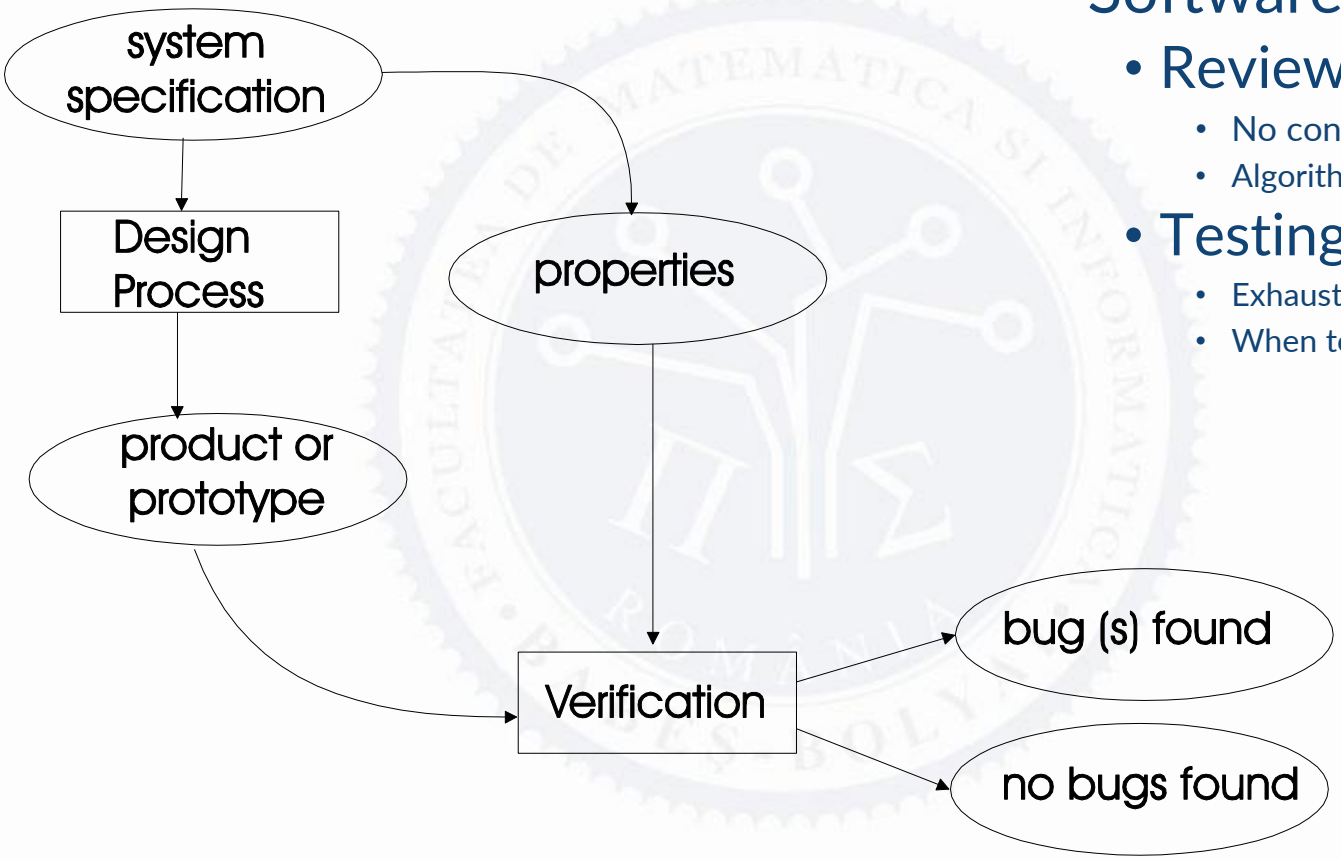
- Spin
- Promela Model
  - Statements
  - Examples
- Concurrency and Interleaving Semantics
  - Examples
- Linear Temporal Logic
  - Examples
- JSpin

- Questions

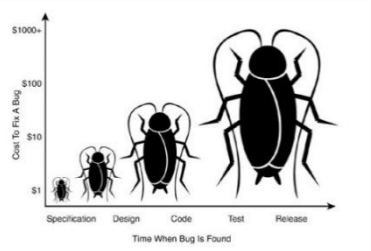
System verification (1)

- Information and Communication Technology (ICT)
- Correct ICT systems
  - It is all about money.
  - It is all about safety.
- Reliability of the ICT systems
  - Interactive systems - concurrency & nondeterminism
  - Pressure - to reduce system development time
- System verification techniques

System verification (2)

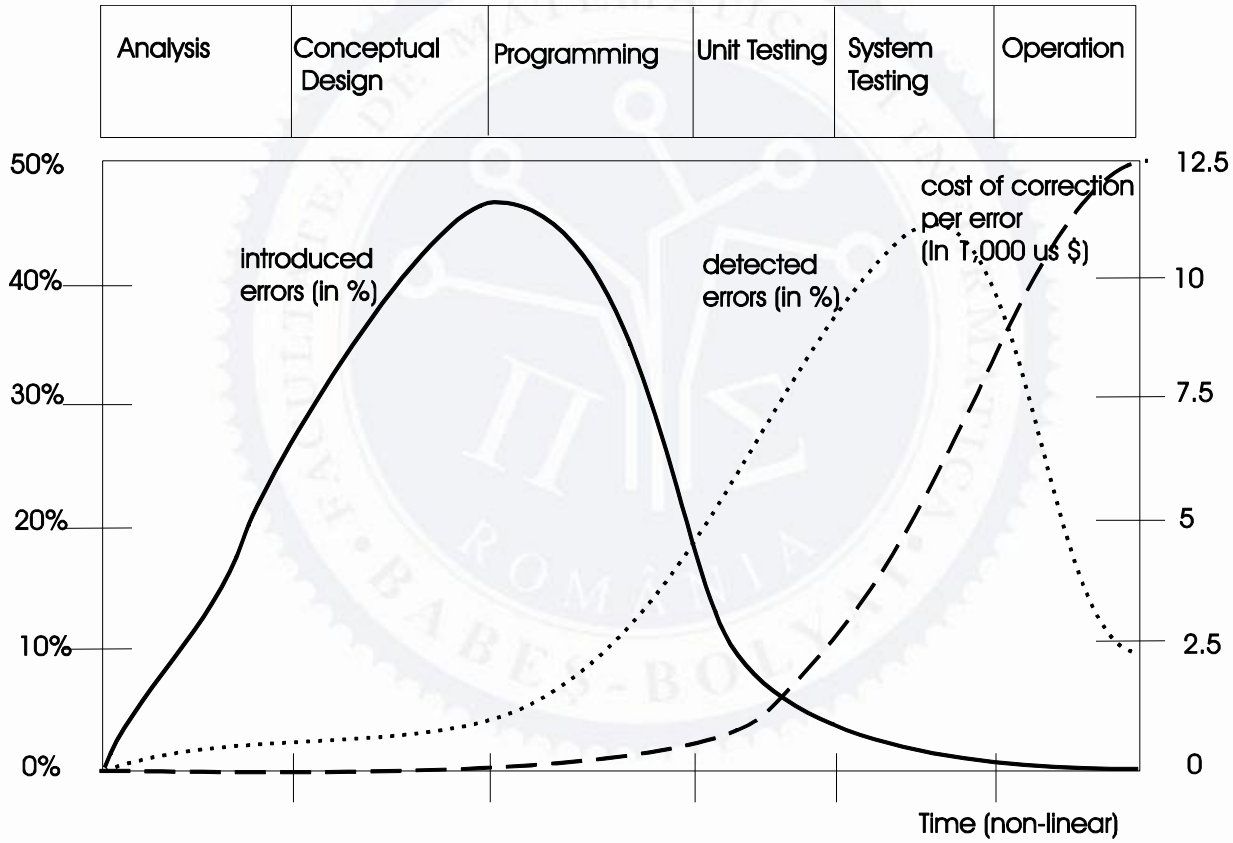


- Software verification
  - Review
    - No concurrency defects
    - Algorithm defects
  - Testing
    - Exhaustive testing?
    - When to stop?



System verification (2)

- Catching software errors: the sooner the better



Model checking (1)

Formal methods

- More time and effort spend on verification than on construction
  - in software/hardware design of complex systems.
- The role of formal methods:
  - To establish system correctness with mathematical rigor.
  - To facilitate the early detection of defects.
- Verification techniques
  - Testing – small subset of paths is treated
  - Simulation - restrictive set of scenarios in the model
  - Model checking - exhaustive exploration
- **Remark.** Any verification using **model-based techniques** is only as good as the model of the system.

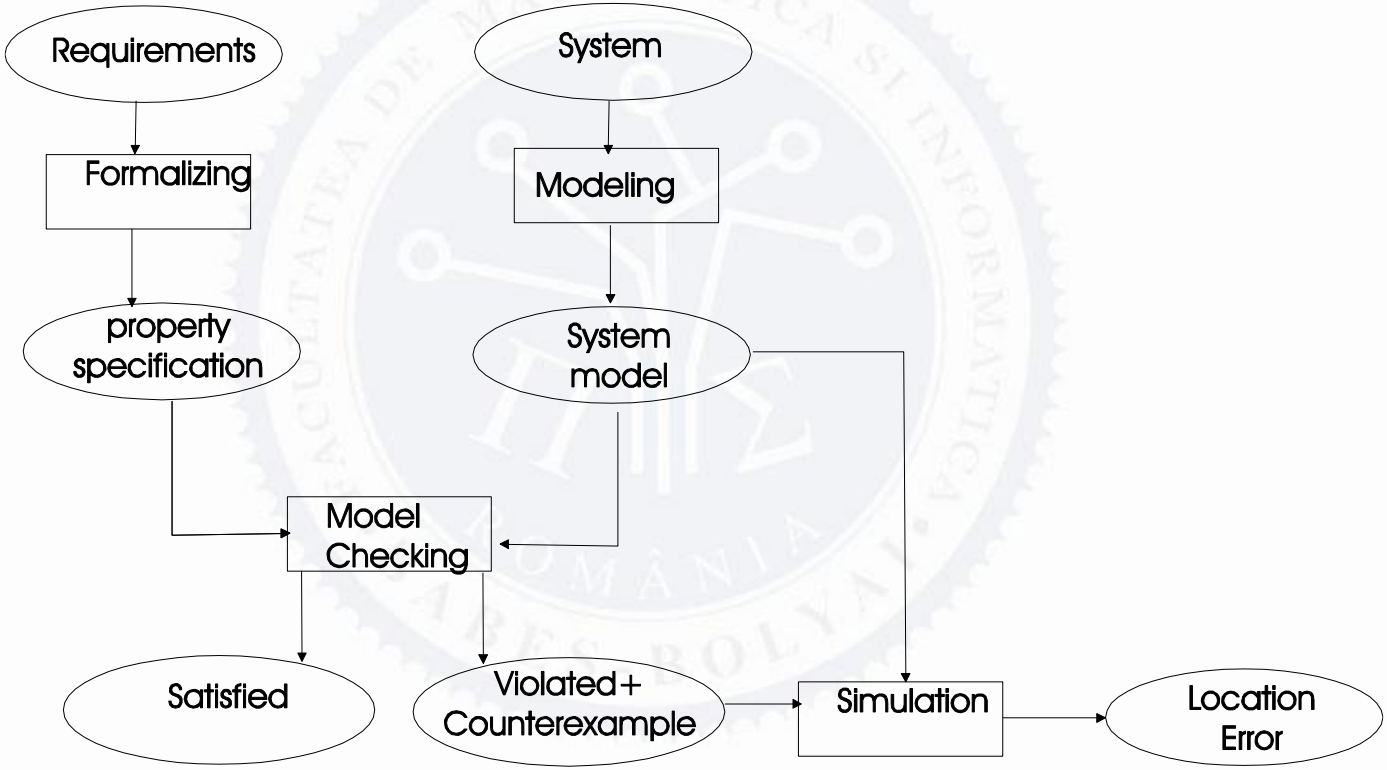
Model checking (1)

Formal methods

- Mechanical Engineering is like looking for a black cat in a lighted room.
- Chemical Engineering is like looking for a black cat in a dark room.
- Software Engineering is like looking for a black cat in a dark room in which there is no cat.
- Systems Engineering is like looking for a black cat in a dark room in which there is no cat and someone yells, “I got it!”

Model checking (2)

Approach



Model checking (3)

Characteristics

- Model checking is an automated technique that, given a finite-state model of a system and a formal property, systematically checks whether this property holds for (a given state in) that model.
- The model checking process
  - Modeling phase
    - model the system under consideration
    - formalize the property to be checked.
  - Running phase
  - Analysis phase
    - property satisfied?
    - property violated?

Model checking (4)

Strengths	Weaknesses
• General verification approach • Supports partial verification • Provides diagnostic information • Potential “push-button” technology • Increasing interest by industry • Easily integrated in existing development cycles	• Appropriate to control-intensive applications • Its applicability is subject to decidability issues • It verifies a system model • Checks only stated requirements • Suffers from the state-space explosion problem • Requires some expertise

Transition system (1)

Definition

- Transition systems - used in computer science as models to describe the behavior of the systems.
- Transition systems - directed graphs:
  - Nodes - represent states;
  - Edges - model transitions, i. e. state changes.
- A Transition System (TS) is tuple (S, Act, $\rightarrow$, I, Ap, L), where
  - S is a set of states,
  - Act is a set of actions,
  - $\rightarrow \subseteq S \times Act \times S$ is a transition relation,
  - $I \subseteq S$ is a set of initial states,
  - AP is a set of atomic propositions, and
  - $L : S \rightarrow 2^{AP}$ is a labeling function.
- TS is called finite if S, Act and AP are finite.

Transition system (2)

Remark

- Intuitive behavior of a transition system
  - Initial state $s_0 \in I$
  - Using the transition relation $\rightarrow$ the system evolves
  - Current state s, a transition $s \xrightarrow{\alpha} s'$ is selected *nondeterministically*
  - The selection procedure is repeated and finishes once a state is encountered that has no outgoing transitions.
- The labeling function L relates a set $L(s) \in 2^{AP}$ at atomic propositions to any state s. $L(s)$ intuitively stands for exactly those atomic propositions $a \in AP$ which are satisfied by state s.
- Given that ϕ is a propositional logic formula, then s satisfies the formula ϕ if the evaluation induced by $L(s)$ makes the formula ϕ true,

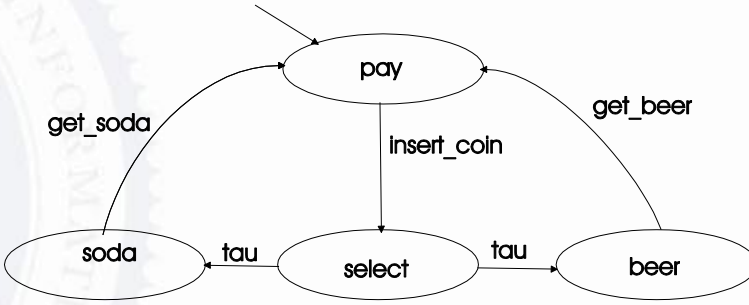
  $s \models \phi$ iff $L(s) \models \phi$.

Transition system (3)

Example

Beverage Vending Machine

- $S = \{pay, select, soda, beer\}, I = \{pay\}$
- $Act = \{insert_coin, get_soda, get_bear, \tau\}$
- Example transitions: $pay \xrightarrow{insert_coin} select$, $beer \xrightarrow{get_beer} pay$
- Atomic propositions depends on the properties under consideration.
  A simple choice - to let the state names act as atomic propositions, i. e. $L(s) = \{s\}$.
  "The vending machine only delivers a drink after providing a coin."
  $AP = \{paid, drink\}$, $L(pay) = \emptyset$, $L(soda) = L(beer) = \{paid, drink\}$, $L(select) = \{paid\}$.



Linear-Time Properties

- **Deadlock** – if the complete system is in a terminal state, although at least one component is in a (local) nonterminal state.
  - A typical deadlock scenarios occurs when components mutually wait for each other to progress.

- **Safety properties** = "nothing bad should happen".
  - The number of inserted coins is always at least the number of dispensed drinks.
  - A typical safety property is deadlock freedom
  - Mutual exclusion problem – "bad" = more than one process is in the critical section

- **Liveness properties** = "something good will happen in the future".
  - Mutual exclusion problem – typical liveness properties assert that:
    - (eventually) – each process will eventually enter its critical section
    - (repeated eventually_ = each process will enter its critical section infinitely often
    - (starvation freedom) – each waiting process will eventually enter its critical section

- **Remark**
  - **Safety properties** - are violated in finite time (a finite system run)
  - **Liveness properties** – are violated in infinite time (by infinite system runs)

Temporal Logic

- **Propositional temporal logics** - extensions of propositional logic by temporal modalities.

- The elementary temporal modalities that are present in most temporal logics include the operators
  - "**eventually**" (eventually in the future) -
  - "**always**" (now and forever in the future –

- The nature of time in temporal logics can be either **linear** or **branching**.

- The adjective "temporal"
  - specification of the relative order of events
  - does not support any means to refer to the precise timing of events

Linear-Time Logic (1)

Syntax of LTL

- Construction of LTL formulae in LTL - ingredients:
  - atomic propositions $a \in AP$, (stands for the state label a in a transition system)
  - boolean connectors like conjunction $\wedge$ and negation $\neg$,
  - basic temporal modalities "next" $\bigcirc$ and "until" $\bigcup$.
- LTL formulae over the set AP of atomic proposition are formed according to the following grammar:
  $\varphi ::= true | a | \varphi_1 \wedge \varphi_2 | \neg\varphi | \bigcirc \varphi | \varphi_1 \bigcup \varphi_2$, where $a \in AP$.

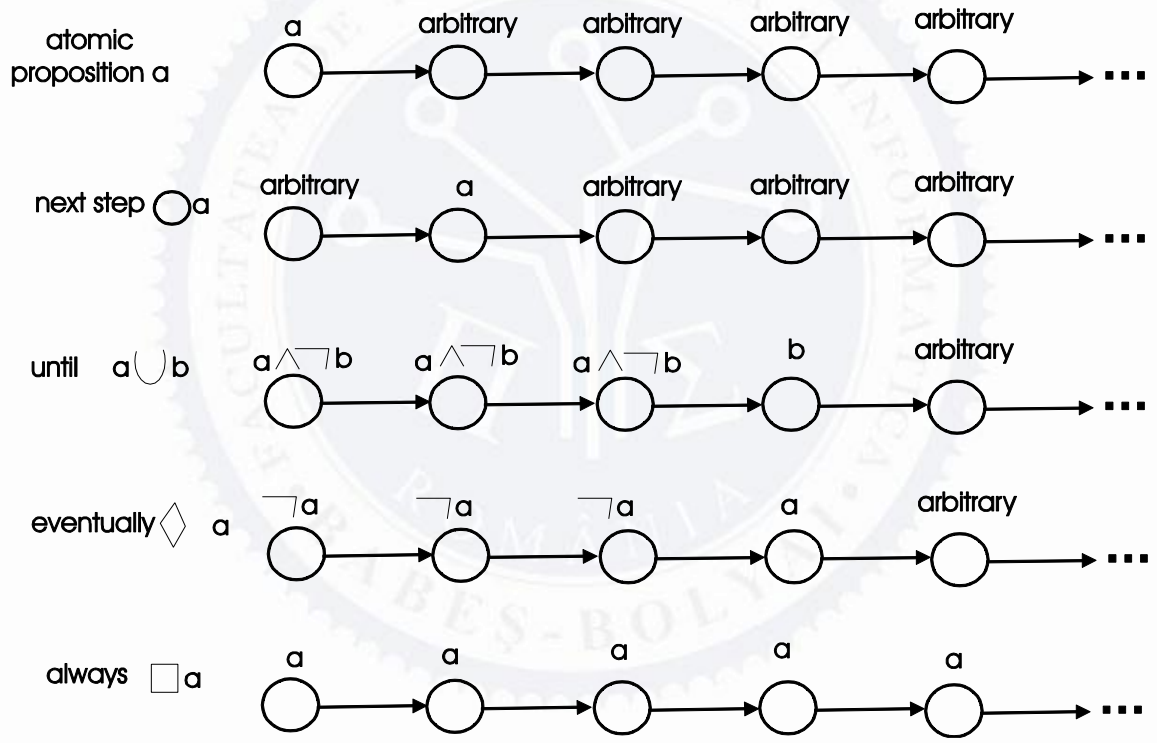
Linear-Time Logic (2)

LTL temporal modalities

- The until operator allows to derive the temporal modalities $\Diamond$ ("eventually", sometimes in the future) and $\Box$ ("always", from now on forever) as follows:
  - $\Diamond\varphi = \text{true} \bigcup \varphi$.
  - $\Box\varphi = \neg\Diamond\neg\varphi$.
- By combining the temporal modalities $\Diamond$ and $\Box$, new temporal modalities are obtained:
  - $\Box\Diamond\varphi$ - "infinitely often φ."
    at any moment j there is a moment i $i \geq j$ at which an a state is visited
  - $\Diamond\Box\varphi$ - "eventually forever φ."
    from some moment j on, only a-states are visited.

Linear-Time Logic (3)

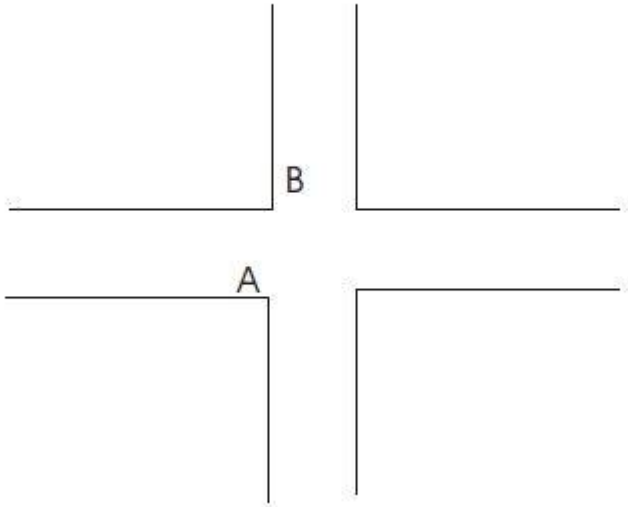
Intuitive meaning of temporal modalities



Linear-Time Logic (4)

LTL semaphore example

- $\square(\neg(A = green \wedge B = green))$
  - A and B can not be simultaneously green.
- $\square(A = yellow \rightarrow A = red)$
  - If A is yellow eventually will become red.
- $\square(A = yellow \rightarrow \bigcirc(A = red))$
  - If A is yellow then it will be red into the next state.
- $\square(\neg(B = green) \bigcup (A = red))$
  - B will not be green until A changes in red.



Computation Tree Logic (1)

Syntax of CTL

- Construction of CTL formulae:
  - as in LTL by the next-step and until operators,
  - must be not combined with boolean connectives
  - no nesting of temporal modalities is allowed.
- CTL formulae over the set AP of atomic proposition are formed according to the following grammar:
  $\phi ::= \text{true} \mid a \mid \phi_1 \wedge \phi_2 \mid \neg\phi \mid \exists\phi \mid \forall\phi$, where $a \in AP$ *and* φ is a path formula.
- CTL path formulae are formed according to the following grammar:
  $\varphi ::= \bigcirc\phi \mid \phi_1 \bigcup \phi_2$, where $\phi, \phi_1 and \phi_2$ are state fromulae.

Computation Tree Logic (2)

CTL - state and path formulae

- CTL distinguishes between state formulae and path formulae:
  - State formulae express a property of a state.
  - Path formulae express a property of a path, i.e. an infinite sequence of states.
- Temporal PATH operators $\bigcirc$ and $\bigcup$
  - $\bigcirc\phi$ holds for a path if ϕ holds in the next state of the path;
  - $\phi \bigcup \psi$ holds for a path if there is some state along the path for which ψ holds, and ϕ holds in all states prior to that state.
- Path formulae $\Rightarrow$ state formulae by prefixing them with
  - path quantifier $\exists$ (pronounced "for some path");
    $\exists\phi$ - holds in a state if there exists some path satisfying ϕ that starts in that state.
  - path quantifier $\forall$ (pronounced "for all paths".)

    $\forall\phi$-holds in a state if all paths that start in that state satisfy ϕ.

Outline

Model checking

- System verification
- Model checking
- Transition system
- Linear-Time Properties
- Linear-Time Logic
- Computation Tree Logic

Spin Model Checker

- Spin
- Promela Model
  - Statements
  - Examples
- Concurrency and Interleaving Semantics
  - Examples
- Linear Temporal Logic
  - Examples
- JSpin

- Questions

Model checking

Spin

- Developed at Bell Labs.
- In 2002, recognized by the ACM with Software System Award.
- SPIN (= Simple Promela Interpreter)
- is a tool for analyzing the logical consistency of concurrent systems
- Concurrent systems are described in the modelling language called Promela (= Protocol/Process Meta Language)

Promela

- Promela (= Protocol/Process Meta Language)
- allows for the dynamic creation of concurrent processes.
- communication via message channels can be defined to be
  - synchronous (i.e. rendezvous),
  - asynchronous (i.e. buffered).

Promela Model

- Promela model consist of:
  - type declarations
  - channel declarations
  - variable declarations
  - process declarations
  - [**init process**]
- A process type (**proctype) consist of**
  - a name
  - a list of formal parameters
  - local variable declarations
  - Body

- A process
  - is defined by a **proctype definition**
  - executes concurrently with all other processes, independent of speed of behaviour
  - communicate with other processes
    - using global (shared) variables
    - using channels
- There may be several processes of the same type.
- Each process has its own local state:
  - process counter (location within the **proctype)**
  - contents of the local variables

Promela Model - Statements

- The body of a process consists of a sequence of statements.
- A statement is either
  - executable: the statement can be executed immediately.
  - blocked: the statement cannot be executed.
- An assignment is always executable.
- An expression is also a statement; it is executable if it evaluates to non-zero
- The **skip statement is always executable.**
  - "does nothing", only changes process' process counter
- A **printf statement is always executable (but is not** evaluated during verification, of course).
- **assert(<expr>);**
  - The **assert-statement is always executable.**
  - If **<expr> evaluates to zero, SPIN will exit with an error, as**
- the **<expr> "has been violated".**
  - The **assert-statement is often used within Promela models,**
- to check whether certain properties are valid in a state.

Examples (01 Simple Examples)

- ReversingDigits.pml
  - Check
  - Random
- DiscriminantOfQuadraticEquation.pml
  - Check
  - Random
- NumberDaysInMonth.pml
  - Check
  - Random
- MaximumNondeterminism.pml
  - Check
  - Random
  - “Branch 1” and “Branch 2”
- Maximum –second example-MaximumIfElse.pml
  - Check
  - Random
- GCD.pml
  - Check
  - Random
- IntegerDivison01.pml
  - Check
  - Random

Concurrency and Interleaving Semantics

02 Concurrency and interleaving semantics

- Promela processes execute concurrently.
  - Non-deterministic scheduling of the processes.
  - Processes are interleaved (statements of different processes do not occur at the same time).
  - exception: rendez-vous communication.
- All statements are atomic; each statement is executed without interleaving with other processes.
- Each process may have several different possible actions enabled at each point of execution - only one choice is made, non-deterministically.
- InterleavingStatements.pml
  - Check
  - Random
  - 6 possibilities of the execution
    - n1,p,n2,q;
    - n1,n2,p,q;
    - n1,n2,q,p;
    - n2,q,n1,p;
    - n2,n1,q,p;
    - n2,n1,p,q.
  - Interactive simulation – Interactive button
- InterferenceBetweenProcesses.pml
- InterferenceBetweenProcessesDeterministic.pml

Examples (03 Critical section)

- CriticalSection_Incorrect.pml
  - both processes – in the critical section
- CriticalSection_MutualExclusion.pml – not satisfied
  - Mutual exclusion – at most one process is executing its critical section at any time.
- CriticalSection_With_Deadlock.pml
  - Blocking on an expression – user Interactive simulation
  - Absence of deadlock – it is impossible to reach a state in which come processes are trying to enter their critical sections, but no process is successful.
- CriticalSection_SolutionAtomic.pml
  - The atomic sequence may be blocked from executing, but once it starts executing, both statements are executed without interference from the other process.

Linear Temporal Logic

- Temporal logic formulae can specify both safety and liveness properties.

- LTL ≡ propositional logic + temporal operators
  - `[]P` always P
  - `<>P` eventually P
  - `P U Q` P is true until Q becomes true

Examples (04 LTL examples)

- CriticalSection_MutualExclusionLTL.pml
  - LTL formula:
    - []mutex
  - Translate
  - Verify
- CriticalSection_MutualExclusionLTL02.pml
  - LTL formula:
    - []mutex
  - Translate
  - Verify
- CriticalSection_With_Starvation.pml
  - LTL formula:
    - <>csp
  - Translate
  - Acceptance
  - Verify

Channels in Promela 05

Channels

- A channel in Promela = a data type with two operations:
  - send
    - The send statement consists of a channel variable followed by **an exclamation point** and then a sequence of expressions whose number and types match the message type of the channel.
  - receive
    - The receive statement consists of a channel variable followed by **question mark** and a sequence of variables.
- Every channel has associated with it a message type.
- The message type that specifies the structure of each message that can be send on the channel as a sequence of fields).

  Chan ch = [capacity] of {typname, …, typename}
- There are two types of channels with different semantics:
  - Rendezvous channels of capacity 0
  - Buffered channels of capacity greater than 0
- Examples
- **Client-server-channels.plm**

Rendezvous Channels in Promela
05 Channels (Book [2]: pages 107-109)

- Rendezvous channel – with capacity 0.
  - The transfer of the message from the sender (a process with a send statement) to the receiver (a process with the receive statement) is synchronous and is executed as a single atomic operation.
- Examples
- **Simple-Rendezvous.pml**
  - The rendezvous is one atomic operation; even if there were other processes, no interleaving could take place between the execution of the send statement and the receive statement.

Traffic-Pedestrian 06 Channels

- Examples

- **PromelaMarryMe_Simple.pml**
- **PromelaMarryMe.pml**
- **traffic_pedestian.pml**

JSpin

- http://spinroot.com/
- Installation JSpin

  http://jspin.software.informer.com/5.0/

Outline

Model checking

- System verification
- Model checking
- Transition system
- Linear-Time Properties
- Linear-Time Logic
- Computation Tree Logic

Spin Model Checker

- Spin
- Promela Model
  - Statements
  - Examples
- Concurrency and Interleaving Semantics
  - Examples
- Linear Temporal Logic
  - Examples
- JSpin

- Questions

References

- [1] Baier Christel, Katoen Joost-Pieter, Principles of Model Checking , ISBN 9780262026499, The MIT Press, 2008
  - Chapter 1 - System verification, Chapter 2 – Modelling Concurrent systems (pag. 19-20), Chapter 3 (pag. 89, 107, 120-121), Chapter 5 – Linear Temporal Logic (pag. 229-233), Chapter 6 – Computation Tree Logic (pag. 313-323)

- [2] Ben-Ari, Mordechai, Principles of the Spin Model Checker, ISBN 978-1-84628-770-1, Springer-Verlag London, 2008

Next Lecture

- Correctness

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Software Systems Verification and Validation

Vescan Andreea, PHD, Assoc. Prof.

Faculty of Mathematics and Computer Science
Babeș-Bolyai University

2024-2025

Software Systems Verification and Validation

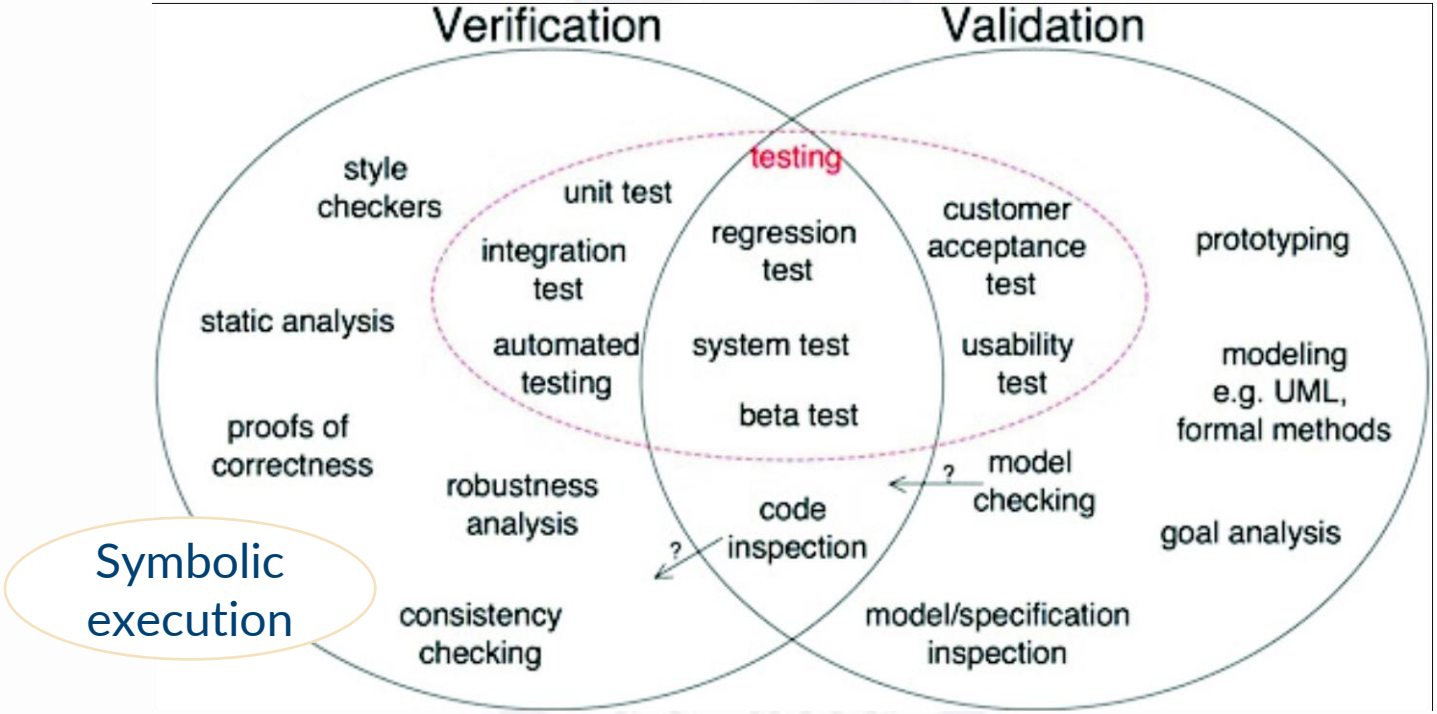
"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

(Next)/Today Lecture

- Correctness

What we will learn!



- http://www.easterbrook.ca/steve/2010/11/the-difference-between-verification-and-validation/

Outline

- Correctness

- Floyd's Method -Inductive assertions, Partial correctness, Termination
- Hoare Logic, Semantics of Hoare triples, Partial correctness, Total correctness
- Dijkstra's Language, Guarded commands, Nondeterminacy, Formal Derivation of Programs

- Developing correct programs from specification, Refinement, Rules of Refinement, Examples

- Static analysis, JML- Java Modeling Language, ESC/Java2- Extended Static Checker for Java

- Questions

Program verification methods - Correctness

- Lecture 1 - Verification and Validation
  - Verification/Validation
    - reviews products to ensure their quality → correctness
    - static and dynamic analysis techniques
  - A **correct program** is one that does exactly what it is intended to do, no more and no less.
  - A formally correct program is one whose correctness can be proved mathematically.
    - This requires a language for specifying precisely what the program is intended to do.
    - Specification languages are based in mathematical logic.
  - Until recently, correctness has been an academic exercise. – Now it is a key element of critical software systems.
- **Program verification - correctness**
  1. proof-based, computer-assisted, program-verification approach, mainly used for programs which we expect to terminate and produce a result
  2. model-based, automatic, property-verification approach, mainly used for concurrent, reactive systems (originally used in a post-development stage) - model checking
  3. Developing correct algorithms from specification (Carroll Morgan, "Programming from Specification)

     Correctness-by-Construction.

     Originally intended as a mere means of programming algorithms that are correct by construction - -Dijkstra (1968), Hoare (1971),

     the approach found its way into commercial development processes of complex systems - Hall (2002), Hall and Chapman (2002)

     2012, The Correctness-by-Construction Approach to Programming, Authors: **Kourie**, Derrick G., **Watson**, Bruce W.

     2015, Experience with correctness-by-construction, B.W. Watson a, D.G. Kourie b, L. Cleophas b,∗

     2016, Correctness-by-Construction and Post-hoc Verification: Friends or Foes?, Maurice H. ter Beek1(B) , Reiner H¨ahnle2, and Ina Schaefer3

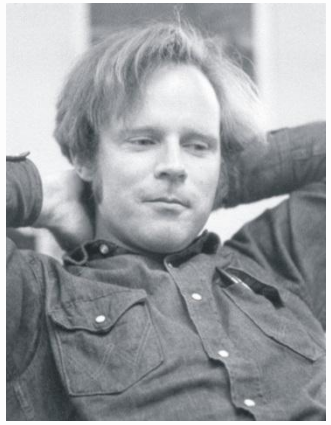
     2023, Automated Software Engineering Conference, The 5th International Workshop on Automated and verifiable Software sYstem DEvelopment (ASYDE)

     Topic: Correct-by-construction software development

     (https://conf.researchr.org/track/ase-2023/ase-2023--workshop--asyde#the-5th-international-workshop-on-automated-and-verifiable-software-system-development-aside)
- **Correctness Tools**
  - Theorem provers (PVS), Modeling languages (UML and OCL), Specification languages (JML), Programming language support (Eiffel, Java, Spark/Ada), Specification Methodology (Design by contract)
- **Methods for prooving program correctness**
  - Floyd's Method - Inductive assertions
  - Hoare - Semantics of Hoare triples
  - Dijkstra's Language- Guarded commands, Nondeterminacy and Formal Derivation of Programs

Floyd's Method - Inductive assertions [Flo67]

- **Aplicability**
  - Partial correctness of the program
  - Termination of the program
  - Total correctness = Partial correctness + Termination of the program
- **Uses**
  - The condition satisfied by the initial values of the program.
  - The condition to be satisfied by the output of the program.
  - Source code of the program.
- **Method**:
    - Cut the loops
    - Find an appropriate set of inductive assertions.
    - Construct the verification/termination conditions.
- **Theorem**: If all verification conditions are true, then the program is partially correct, i.e., whenever it terminates the result is correct.
- **Remark**. The method is useful when it is combined with termination.

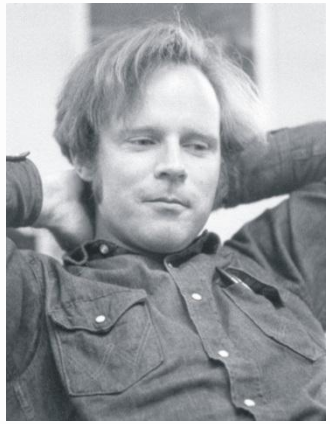


Robert W Floyd
(June 8, 1936 - September 25, 2001)

Floyd's Method - Inductive assertions [Flo67]

Partial correctness - steps

- Cutting points are chosen inside the algorithm
  1. 1 point at the beginning of the algorithm, 1 point at the end;
  2. At least 1 point for each *loop* statement
- For each cutting point an assertion (invariant predicate) is chosen.
  1. Entry point - $\varphi(X)$;
  2. Ending point - $\psi(X, Z)$.
- Construction of the verification conditions
  1. Path from i to j - α;
  2. P_i and P_j are assertions in i and j;
  3. $R_\alpha(X, Y)$ - predicate that gives the condition for path α;
  4. $r_\alpha(X, Y)$ - function that gives the transformations of the variables Y from path α;
  5. $\forall X \forall Y(P_i(X, Y) \wedge R_\alpha(X, Y) \rightarrow P_j(X, r_\alpha(X, Y)))$.
- Theorem: If all the verification conditions are true then P is partial correct.



Robert W Floyd
(June 8, 1936 - September 25, 2001)

Floyd's Method - Inductive assertions [Flo67]

Partial correctness - example

- Algorithm for $z = x^y$

  $z := 1;\ u := x;\ v := y;$
  While $(v > 0)$ execute
  If (v is even)
  then $u := u * u;\ v := v/2;$
  else $v := v - 1;\ z := z * u;$
  endIf
  endWhile
  endAlg;

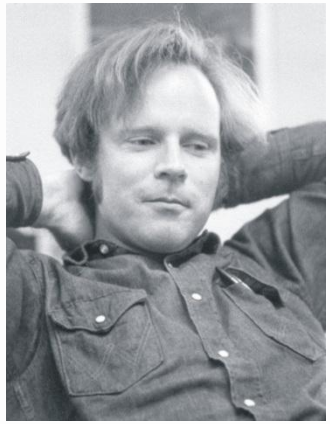
  A: $\varphi(X) ::= (v > 0 \wedge (y \geq 0))$
  B: $\eta(X, Y) ::= z * u^v = x^y$
  C: $\psi(X, Z) ::= z = x^y$

Robert W Floyd
(June 8, 1936 - September 25, 2001)

Floyd's Method - Inductive assertions [Flo67]

Termination- steps

- Cut the loops and find "good" inductive assertions.
- Choose a well-formed set M (i.e., an ordered set without infinite strictly decreasing sequences)
- To demonstrate that some termination conditions hold: passing from one cutting point to another the values of some functions in the well-ordered set decrease.
- In point i a function is chosen $u_i : D_X \times D_Y \to M$ and the termination condition on α is:
  $\forall X \forall Y(\varphi(X) \wedge R_\alpha(X, Y) \to (u_i(X, Y) > u_j(X, r_\alpha(X, Y))))$.
- **Remark**. If partial correctness was demonstrated then the termination condition can be:
  $\forall X \forall Y(P_i(X) \wedge R_\alpha(X, Y) \to (u_i(X, Y) > u_j(X, r_\alpha(X, Y))))$.
- Theorem: If all the termination conditions hold then the program P terminates.



Robert W Floyd
(June 8, 1936 - September 25, 2001)

Floyd's Method - Inductive assertions [Flo67]

Termination- example

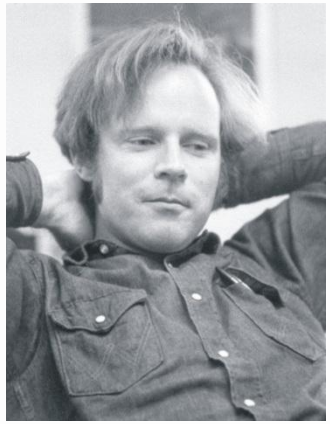
- Algorithm for $z = x^y$

  $z := 1;\ u := x;\ v := y;$
  While $(v > 0)$ execute
  If (v is even)
  then $u := u * u;\ v := v/2;$
  else $v := v - 1;\ z := z * u;$
  endIf
  endWhile
  endAlg;

  A: $\varphi(X) ::= (v > 0 \wedge (y \geq 0))$

  B: $\eta(X, Y) ::= z * u^v = x^y$

  C: $\psi(X, Z) ::= z = x^y$



Robert W Floyd
(June 8, 1936 - September 25, 2001)

Outline

- Correctness

- Floyd's Method -Inductive assertions, Partial correctness, Termination
- Hoare Logic, Semantics of Hoare triples, Partial correctness, Total correctness
- Dijkstra's Language, Guarded commands, Nondeterminacy, Formal Derivation of Programs

- Developing correct programs from specification, Refinement, Rules of Refinement, Examples

- Static analysis, JML- Java Modeling Language, ESC/Java2- Extended Static Checker for Java

- Questions

Hoare triples [Hoa69]

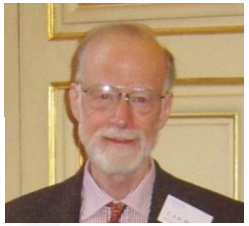
- The meaning of a statement is described by a triple
  - $\{\varphi\}\ P\ \{\psi\}$, where φ is called the precondition and ψ is called the postcondition.

{P} S {Q}

"when started in a state satisfying P, any terminating execution of S ends in a state satisfying Q "

- If P does not terminate, we make no guarantees.

- Partial correctness
  - $\models_{par} \{\varphi\}P\{\psi\}$
  - only if P actually terminates.
- Total correctness
  - $\models_{tot} \{\varphi\}P\{\psi\}$
  - the program P is guaranteed to terminate.



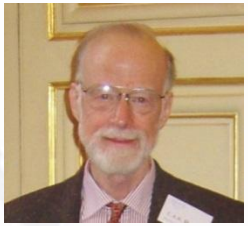
- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

An Advanced Study Institute of the
NATO Security Through Science Committee
and
the Institut fÃ¼r Informatik,
Technische UniversitÃ¤t MÃ¼nchen, Germany,
on
Software System Reliability and Security
August 1 to August 13 2006
M. Broy (director)
O. Kupferman (director)
C.A.R. Hoare (co-director)
A. Pnueli (co-director)
Katharina Spies (secretary)
The Summer School is also substantially supported by the DAAD under the program "Deutsche Sommerakademie 2006", and the town and the county of Marktoberdorf

Hoare triples [Hoa69]

Partial correctness



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

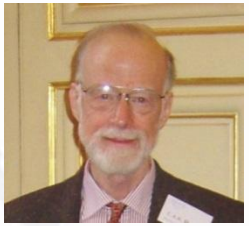
Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

Rules

- Assignment
- Sequencing
- Conditional
- Loop

Hoare triples [Hoa69]

Partial correctness



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

Assignment

General Form: for any expression E

- $\{P\}\ X := E\{Q\}$ *provided* $[P \Rightarrow \langle X \leftarrow E\rangle\,(Q)]$

- Consider the triple $\{P\}X := Y + 2\{Q\}$
  - Given predicate Q, for what predicate P does this hold?
  - for any P such that $[P \Rightarrow \langle X \leftarrow Y + 2\rangle\,(Q)]$
- Examples
  - $\{P_0\}\ X := Y + 2\ \{X \leq Y + 2\}$
    $P_0 \equiv true$
  - $\{P_1\}\ X := Y + 2\ \{X < 0\}$
    $P_1 \equiv (Y + 2 < 0)$
  - $\{P_2\}\ X := Y + 2\ \{Y < 0\}$
    $P_2 \equiv (Y < 0)$
  - $\{P_3\}\ X := X + 2\ \{X\ is\ even\}$
    $P_3 \equiv (X\ is\ even)$

Hoare triples [Hoa69]

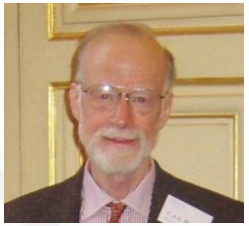
Partial correctness

Sequencing

- We can conclude
  $\{P\}\ S; T\{Q\}$
  if we can find a predicate R such that
  $\{P\}\ S\{R\}$ *and* $\{R\}T\{Q\}$

Examples

- $\{P_0\}\ X := 2 * X;\ X := X + 1\{X > 0\}$
  $P_0 \equiv (2 * X + 1 > 0)]$
- $\{P_1\}\ X := Y;\ Y := 3\ \{X + Y < 5\}$
  $\{P_1 \equiv (Y + 3 < 5)]$

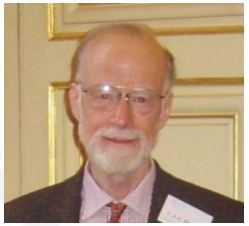


- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

Hoare triples [Hoa69]

Partial correctness



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

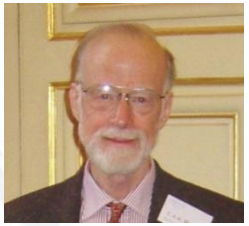
Conditional

- We can conclude
  $\{P\}$ *IF* (C) *THEN* S *ELSE* T *END*$\{Q\}$
  provided we can show
  $\{P \wedge C\}\ S\{Q\}$ *and* $\{P \wedge \neg C\}T\{Q\}$

- Examples
  - $\{?\}$ $\{((x > y) \Rightarrow Q_0) \wedge ((x \leq y) \Rightarrow Q_1)\}$
    IF $(x > y)$ *THEN* $Q_0 : \{(m|x - y) \wedge (m|y)\}$
    $x := x - y$
    ELSE $Q_1 : \{(m|x) \wedge (m|y - x)\}$
    $y := y - x$
    END
    $Q : \{(m|x) \wedge (m|y)\}$
  - So our final proof obligations are
    $[(x > y) \Rightarrow (m|x - y) \wedge (m|y)$ *and*
    $[(x \leq y) \Rightarrow (m|x) \wedge (m|y - x)]$

Hoare triples [Hoa69]

Partial correctness



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

Loop

- How can we conclude
  $\{P\}$ *WHILE* (G) *DO* S *END* $\{Q\}$
  At the end of the loop (assuming it terminates), we know $\neg G$
  But in general we dont know how often S is executed...
- Suppose we have a predicate J that is preserved by S
  $\{J\}S\{J\}$ such a J is called a loop invariant
  Then, at the end of the loop, we can conclude
  $J \wedge \neg G$
  To establish the postcondition, we need J such that
  $[J \wedge \neg G \Rightarrow Q]$

- We can conclude
  $\{P\}$ *WHILE* (G) *DO* S *END* $\{Q\}$
  provided we can find a loop invariant J such that

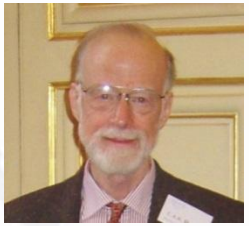
$[P \Rightarrow J]$	J holds at loop entry
$[J \wedge \neg G \Rightarrow Q]$	J establishes Q at loop exit
$\{G \wedge J\}S\{J\}$	J is preserved by each iteration

- Exponentiation using multiplication
  - $\{(A > 0) \wedge (B \geq 0)\}$ S $\{R = A^B\}$

$\{(A > 0) \wedge (B \geq 0)\}$
$R :=?; b := 0$ R:=1
WHILE $(b \neq B)$ DO $J : R = A^b$
$R :=?;\ R := R * A;$
$b := b + 1$
END
$\{R = A^B\}$

Hoare triples [Hoa69]

- The meaning of a statement is described by a triple
  - $\{\varphi\}\ P\ \{\psi\}$, where φ is called the precondition and ψ is called the postcondition.



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

{P} S {Q}

"when started in a state satisfying P, any terminating execution of S ends in a state satisfying Q "

- If P does not terminate, we make no guarantees.

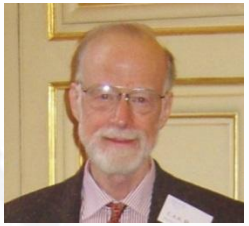
- Partial correctness
  - $\models_{par} \{\varphi\}P\{\psi\}$
  - only if P actually terminates.
- Total correctness
  - $\models_{tot} \{\varphi\}P\{\psi\}$
  - the program P is guaranteed to terminate.

- The "total correctness" interpretation also requires termination

"when started in a state satisfying P, any execution of S must terminate in a state satisfying Q "

Hoare triples [Hoa69]

Termination



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

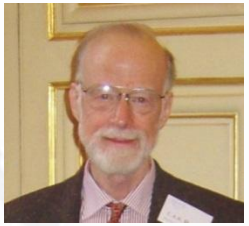
Rules

- Assignment
- Sequencing
- Conditional
- Loop

- Assignment
  $\{P\}\ X := E\ \{Q\}$ *provided* $[P \Rightarrow \langle X \leftarrow E \rangle (Q)]$
- Sequencing
  $\{P\}\ S; T \{Q\}$ provided
  $\{P\}\ S\ \{R\}$ *and* $\{R\} T\ \{Q\}$ for some R
- Conditional
  $\{P\}$ *IF* (G) *THEN* S *ELSE* T *END* $\{Q\}$ provided
  $\{P \wedge G\}\ S\ \{Q\}$ *and* $\{P \wedge \neg G\}\ T \{Q\}$
- Note: Same as the rules for partial correctness!

- Total correctness rule for loops
- Consider
  $\{P\}$ *WHILE* (G) *DO* S *END* $\{Q\}$
- How do we show that the loop terminates?
- One method
  find an integer expression V such that
  the value of V is nonnegative (that is $V \geq 0$), and
  the value of V (strictly) decreases in every iteration that is,
  $\{V = K\}\ S\ \{V < K\}$
- Such an expression is called a "loop variant"

Hoare triples [Hoa69]



- The Grand Verification Challenge Hoare 2003
- Develop a compiler which verifies that the program is correct
- https://vimeo.com/39256698

Charles Antony Richard Hoare
(11 January 1934, Colombo, Sri Lanka)

Exponentiation using multiplication

- $\{(A > 0) \wedge (B \geq 0)\}$ S $\{R = A^B\}$
- Recall loop invariant $J : R = A^b \ \wedge (B \geq b)$;

$\{(A > 0) \wedge (B \geq 0)\}$
$R := 1; b := 0$
WHILE $(b \neq B)$ DO $\quad J : \ R = A^b \wedge (B \geq b)$;
$R := R * A;$
$b := b + 1$
END
$\{R = A^B\}$

Outline

- Correctness

- Floyd's Method -Inductive assertions, Partial correctness, Termination
- Hoare Logic, Semantics of Hoare triples, Partial correctness, Total correctness
- Dijkstra's Language, Guarded commands, Nondeterminacy, Formal Derivation of Programs

- Developing correct programs from specification, Refinement, Rules of Refinement, Examples

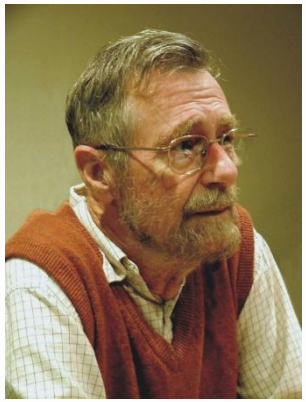
- Static analysis, JML- Java Modeling Language, ESC/Java2- Extended Static Checker for Java

- Questions

Edsger Wybe Dijkstra [Dij75]

Guarded command

- "guarded command" - a statement list prefixed by a boolean expression: only when this boolean expression is initially true, is the statement list eligible for execution
- $< \textit{guarded command} > ::= < \textit{guard} > \rightarrow < \textit{guarded list} >$
- $< \textit{guard} > ::= < \textit{boolean expression} >$
- $< \textit{guarded list} > ::= < \textit{statement} > \{; < \textit{statement} >\}$
- $< \textit{guarded command set} > ::= < \textit{guarded command} > \{\square < \textit{guarded command} >\}$
- $< \textit{alternative construct} > ::= \textbf{if} < \textit{guarded command set} > \textbf{fi}$
- $< \textit{repetitive construct} > ::= \textbf{do} < \textit{guarded command set} > \textbf{do}$
- $< \textit{statement} > ::= < \textit{alternative construct} > \mid < \textit{repetitive construct} > \mid$ "other statements"



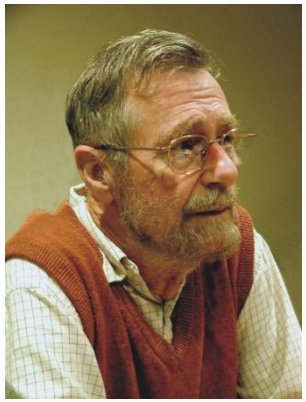
Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

Nondeterminacy

- Example 1
  if $x \geq y \rightarrow m := x$
  $\square y \geq x \rightarrow m := y$
  fi
- Example 2 - compute k s.t. for fixed value n and fixed function $f(i)$ *(defined for* $0 \leq i < n$*),* k *will eventually satisfy* $0 \leq k < n$ *and* $(\forall i : 0 \leq i < n : \ f(k) \geq f(i))$.

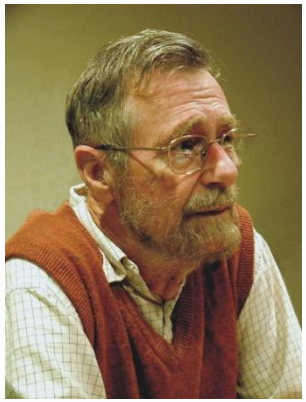
  $k := 0;\ j := 1;$
  do $j \neq n \rightarrow$ **if** $f(j) \leq f(k) \rightarrow j := j + 1$
  $\square f(j) \geq f(k) \rightarrow k := j;\ j := j + 1$
  fi
  od



Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

Weakest pre-conditions



Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

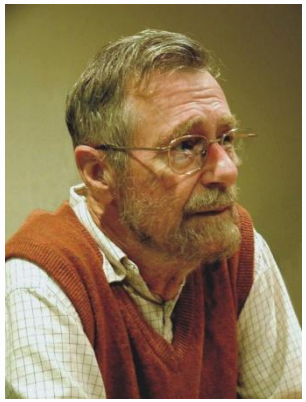
- Hoare - introduced sufficient pre-conditions such that the mechanism will not produce the wrong result but may fail to terminate.
- Dijkstra - introduced necessary and sufficient pre-conditions such that the mechanism are guaranteed to produce the right result.

  = weakest pre-conditions
- wp(S, R), where S denotes a statement list, R some condition on the state of the system.
- wp - called a " predicate transformer" - because it associates a pre-condition to any post-condition R.

Edsger Wybe Dijkstra [Dij75]

Properties of wp

1. Law of the Excluded Miracle
   For any S, for all states: $wp(S, F) = F$
2. For any S and any two post-conditions, such that for all states $P \Rightarrow Q$, for all states:
   $$wp(S, P) \Rightarrow wp(S, Q)$$
3. For any S and any two post-conditions P and Q, for all states:
   $$wp(S, P) \textbf{ and } wp(S, Q) = wp(S, P \textbf{ and } Q)$$
4. For any deterministic S and any post-conditions P and Q, for all states:
   $$(wp(S, P) \textbf{ or } wp(S, Q)) \Rightarrow wp(S, P \textbf{ or } Q)$$

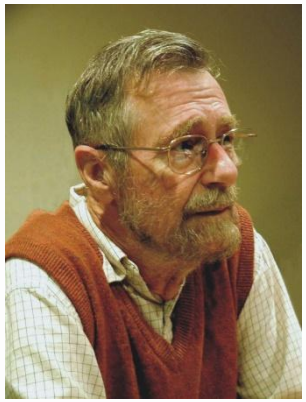


Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

Assignment and concatenation operator

- Assignment
  The semantics of $x := E$ are given by:
  $wp(\text{“}x := E'', R) = R^x_E$, R^x_E -denotes a copy of the predicate defining R in which each occurrence of the variable x is replaced by E.
- Concatenation operator ;
  The semantics of the ; operator are given by:
  $wp(\text{“}S1\ ; S2'', R) = wp(S1, wp(S2, R))$.

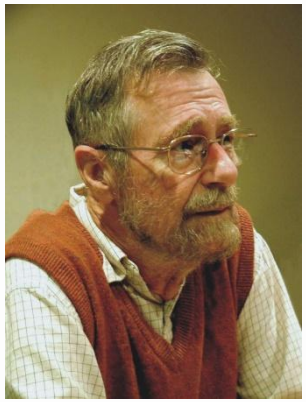


Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

The Alternative Construct

- Let IF denote: $\textbf{if} B_1 \rightarrow SL_1 \square ... \square B_n \rightarrow SL_n \textbf{fi}$
  Let BB denote: $(\exists i : \ 1 \leq i \leq n : \ B_i)$, then, by definition
  $wp(IF, R) = (BB \ and \ (\forall i : \ 1 \leq i \leq n : \ B_i \Rightarrow wp(SL_i, R)))$.
- Theorem 1
  From $(\forall i : 1 \leq i \leq n : \ (Q \ \textbf{and} \ B_i) \Rightarrow wp(SL_i, R)$ for all states we can conclude that $(Q \ \textbf{and} \ BB) \Rightarrow wp(IF, R)$ holds for all states.)
- Let t denote some integer function, and $wdec(S, t)$
- Theorem 2
  From $(\forall i : 1 \leq i \leq n : (Q \ \textbf{and} \ B_i) \Rightarrow wdec(SL_i, t))$ for all states we can conclude that $(Q \ \textbf{and} \ BB) \Rightarrow wdec(IF, t)$ hold for all states.
- By definition,
  $wdec(S, t) = (tmin(X) \leq t(X) - 1) = (tmin(X) < t(X))$.

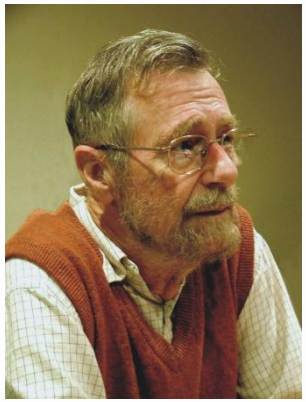


Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

The Alternative Construct - example

- The formal requirements for performing $m := max(x, y)$ is:
  $R: \ (m = x$ **or** $m = y)$ **and** $m \geq x$ **and** $m \geq y$.
- Assignment $m := x$ for $m = x$?
  $wp(\text{“}m := x\text{”}, R) = (x = x$ **or** $x = y)$ **and** $x \geq x$ **and** $x \geq y = x \geq y$
- Theorem 1: **if** $x \geq y \rightarrow m := x$ **fi**
- But $B \neq T$, so we weakening BB means looking for alternatives which might introduce new guards.
- Alternative: $\text{“}m := y\text{”}$ that introduces the new guard
  $wp(\text{“}m\text{”} := y, R) = y \geq x$
  if $x \geq y \rightarrow m := x$
  $\square y \geq x \rightarrow m := y$
  fi

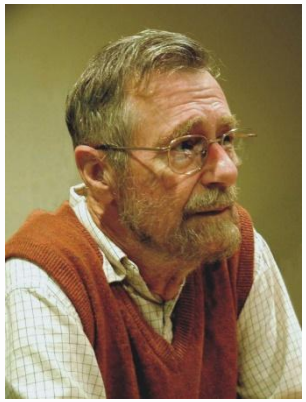


Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

The Repetitive Construct

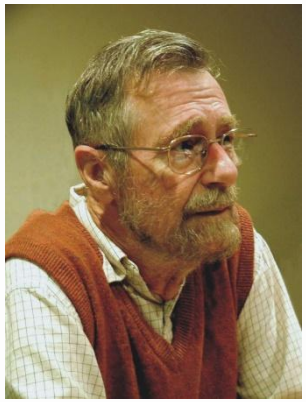
- Let DO denote: $\mathbf{do} B_1 \rightarrow SL_1 \square ... \square B_n \rightarrow SL_n \mathbf{do}$
  Let $H_0 = (R \textbf{ and non } BB)$
  and for $k > 0, H_k(R) = (wp(IF, H_{k-1}(R))) \textbf{ or } H_0(R)$
  then, by definition: $wp(DO, R) = (\exists k : k \geq 0 : H_k(R))$.
- Theorem 3
  If we have all the states
  $(P \textbf{ and } BB) \Rightarrow (wp(IF, P) \textbf{ and } wdec(IF, t) \textbf{ and } t \geq 0)$ we can conclude that we have for all states $P \Rightarrow wp(DO, P \textbf{ and non } BB)$
- T is the condition satisfied by all states, and $wp(S, T)$ is the weakest pre-condition guaranteeing proper termination of S.
- Theorem 4
  From $(P \textbf{ and } BB) \Rightarrow wp(IF, P)$ for all states, we can conclude that we have for all states
  $(P \textbf{ and } wp(DO, T) \Rightarrow wp(DO, P \textbf{ and non } BB))$



Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

Edsger Wybe Dijkstra [Dij75]

The Repetitive Construct - example



Edsger Wybe Dijkstra
(May 11, 1930 - August 6, 2002)

- The greatest common divisor: $x = gcd(X, Y)$
- Choose an invariant relation and variant function.
  establish the relation P to be kept invariant
  do "decrease t as long as possible under variance of P" **od**
- invariant relation (established by $x := X; y := Y$):
  $P : gcd(X, Y) = gcd(x, y)$ **and** $x > 0$ **and** $y > 0$
- $(P$ **and** $B) \Rightarrow wp(\text{“}x, y : E1, E2'', P))$
  $= (gcd(X, Y) = gcd(E1, E2)$ *and* $E1 > 0$ **and** $E2 > 0)$.
  - $gcd(X, Y) = gcd(E1, E2)$ is implied by P
  - invariant for $(x, y) : wp(\text{“}x := x - y, P) = (gcd(X, Y) = gcd(x - y, y)$ **and** $x - y > 0$ **and** $y > 0)$, and guard $x > y$
  - decrease of the variant function $t = x + y$
    $wp(\text{“}x := x - y'', t \leq t_0) = (x \leq t_0)$
    $tmin = x$, $wdec(''x := x - y'', t) = (x < x + y) = y > 0$

- x:=X; y:=Y
  do $x > y \rightarrow x := x - y$ **od**
- But P **and** BB - are not allowed to conclude $x = gcd(X, Y)$
  the alternative $y := y - x$ requires a guard $y > x$
- x:=X; y:=Y
  do $x > y \rightarrow$ x:=x-y
  $\square y > x \rightarrow y := y - x$
  od

Outline

- Correctness

- Floyd's Method -Inductive assertions, Partial correctness, Termination
- Hoare Logic, Semantics of Hoare triples, Partial correctness, Total correctness
- Dijkstra's Language, Guarded commands, Nondeterminacy, Formal Derivation of Programs

- Developing correct programs from specification, Refinement, Rules of Refinement, Examples

Correctness-by-Construction.

Originally intended as a mere means of programming algorithms that are correct by construction - -Dijkstra (1968), Hoare (1971),

the approach found its way into commercial development processes of complex systems - Hall (2002), Hall and Chapman (2002)

2012, The Correctness-by-Construction Approach to Programming, Authors: **Kourie**, Derrick G., **Watson**, Bruce W.

2015, Experience with correctness-by-construction, B.W. Watson a, D.G. Kourie b, L. Cleophas b,

2016, Correctness-by-Construction and Post-hoc Verification: Friends or Foes?, M. Beek , R. Hahnle,I. Schaefer

2016, Correctness-by-Construction and Post-hoc Verification: A Marriage of Convenience? B. Watson, D. Kourie, I. Schaefer, L. Cleophas

2023, Automated Software Engineering Conference, The 5th International Workshop on Automated and verifiable Software sYstem DEvelopment (ASYDE)

Topic: Correct-by-construction software development

(https://conf.researchr.org/track/ase-2023/ase-2023--workshop--asyde#the-5th-international-workshop-on-automated-and-verifiable-software-system-development-aside)

Static analysis, JML- Java Modeling Language, ESC/Java2- Extended Static Checker for Java

- Questions

Developing correct programs from specification[Mor98]

Refinement

- Input data: X $\varphi(X)$
  Output data: Z $\psi(X, Z)$
- Abstract program
  $Z : [\varphi, \psi]$
- Refinement
  $P_1 \prec P_2 \prec \ldots \prec P_{n-1} \prec P_n$
- Rules of refinement
  - Assignment rule
  - Sequential composition rule
  - Alternation rule
  - Iteration rule

Carroll
Morgan

https://my.cse.unsw.edu.au/staff/staff_details.php?ID=carrollm

Developing correct programs from specification[Mor98]

Rules of Refinement

- Assignment rule: $[\varphi(v/e), \psi] \prec v := e$
- Sequential composition rule ($\gamma - \textit{middlepredicate}$)
  $[\eta_1, \eta_2] \prec [\eta_1, \gamma]$
  $[\gamma, \eta_2]$
- Alternation rule, $G = g_1 \vee g_2 \vee ... \vee g_n$
  $[\eta_1, \eta_2] \prec$
  if $g_1 \rightarrow [\eta_1 \wedge g_1, \eta_2]$
  $\square g_2 \rightarrow [\eta_1 \wedge g_2, \eta_2]$
  $\vdots$
  $\square g_n \rightarrow [\eta_1 \wedge g_n, \eta_2]$
  fi
- Iteration rule $G = g_1 \vee g_2 \vee ... \vee g_n$
  $[\eta, \eta \wedge \neg G] \prec$
  do $g_1 \rightarrow [\eta \wedge g_1, \eta \wedge TC]$
  $\square g_2 \rightarrow [\eta \wedge g_2, \eta \wedge TC]$
  $\vdots$
  $\square g_n \rightarrow [\eta \wedge g_n, \eta \wedge TC]$
  do

Program verification methods - Correctness

- Lecture 1 - Verification and Validation
  - Verification/Validation
    - reviews products to ensure their quality → correctness
    - static and dynamic analysis techniques
  - A **correct program** is one that does exactly what it is intended to do, no more and no less.
  - A formally correct program is one whose correctness can be proved mathematically.
    - This requires a language for specifying precisely what the program is intended to do.
    - Specification languages are based in mathematical logic.
  - Until recently, correctness has been an academic exercise. – Now it is a key element of critical software systems.
- **Program verification - correctness**
  1. proof-based, computer-assisted, program-verification approach, mainly used for programs which we expect to terminate and produce a result
  2. model-based, automatic, property-verification approach, mainly used for concurrent, reactive systems (originally used in a post-development stage) - model checking (Lecture 9)
  3. Developing correct algorithms from specification (Carroll Morgan, "Programming from Specification)

     Correctness-by-Construction.

     Originally intended as a mere means of programming algorithms that are correct by construction - -Dijkstra (1968), Hoare (1971),

     the approach found its way into commercial development processes of complex systems - Hall (2002), Hall and Chapman (2002)

     2012, The Correctness-by-Construction Approach to Programming, Authors: **Kourie**, Derrick G., **Watson**, Bruce W.

     2015, Experience with correctness-by-construction, B.W. Watson a, D.G. Kourie b, L. Cleophas b,∗

     2016, Correctness-by-Construction and Post-hoc Verification: Friends or Foes?, Maurice H. ter Beek1(B) , Reiner H¨ahnle2, and Ina Schaefer3
- **Correctness Tools**
  - Theorem provers (PVS), Modeling languages (UML and OCL), Specification languages (JML), Programming language support (Eiffel, Java, Spark/Ada), Specification Methodology (Design by contract)
- **Methods for prooving program correctness**
  - Floyd's Method - Inductive assertions
  - Hoare - Semantics of Hoare triples
  - Dijkstra's Language- Guarded commands, Nondeterminacy and Formal Derivation of Programs

Program verification methods - Correctness

- **Software engineering problem:** building/maintaining **correct** systems.
  - How?
    - Specification
    - Tools
- Formal Methods in Software Engineering
  - Formal languages guarantee
    - Precision (no ambiguity)
    - Certainty (modeling errors)
    - Automation (automatic verification tools).

- Things to do:
  1) make a ***formal model***
  2) ***specify properties*** for the model
  3) ***verify/check*** the properties

- Formal methods and JML (Java Modeling Language):
  1) formal model is ***Java programming language***
  2) the properties are specified in ***JML***
  3) Properties may be
     - ***Tested*** using ***jmlrac***
     - ***Checked*** using ***ESC2Java***

What is JML?

- Gary T. Leavens's JML group at the University of Central Florida
- http://www.eecs.ucf.edu/~leavens/JML//index.shtml

- a behavioral interface specification language
- used to specify the behavior of Java modules
- combines
  - design by contract approach
  - the model-based specification approach
  - some elements of the refinement calculus

Tools for using JML

- Runtime assertion checkers (e.g. **jmlc/jmlrac**)
- Static checkers (**ESC2Java**)

- Test generation (e.g. jmlunit)
- Formal verification tools (e.g. KeY)
- Design tools (e.g. AutoJML)

Tools for JML

Runtime assertion checking with jmlc/jmlrac

- Special compiler inserts runtime tests for all JML assertions. Any assertion violation results in a special exception.
- checks specs at run-time
- only **tests** correctness of **specs**.
- **Find violations at runtime.**

Extended static checking with ESC/Java

- Automatically tries to prove simple JML assertions at compile time.
- checks specs at compile-time
- **proves** correctness of **specs**.
- **Warn about likely runtime exceptions and violations.**

JML web page

- http://www.eecs.ucf.edu/~leavens/JML//index.shtml

ESC/Java2 web page

http://www.kindsoftware.com/products/opensource/ESCJava2/download.html

Design by contract

Contract?

Method contract

Precondition

Specifies "caller's responsibility"

- Constraints on parameter values and target object's state.
- Valid object's states, in which a method can be called.

Intuitively

- Expression that must hold at the entry to the method.

Postcondition

Specifies "implementation's responsibility"

- Constraints on the method's return value and side effects.
- Relation between initial and final state of the method.

Intuitively

- Expression that must hold at the exit from the method.

Class contract

Invariant

- Specifies caller's responsibility at the entry to a method and implementation's responsibility at the exit from a method.
- Valid states of class instances (values of fields).

Intuitively

- Expression that must hold at the entry and exit of each method in the class.

Tools for JML

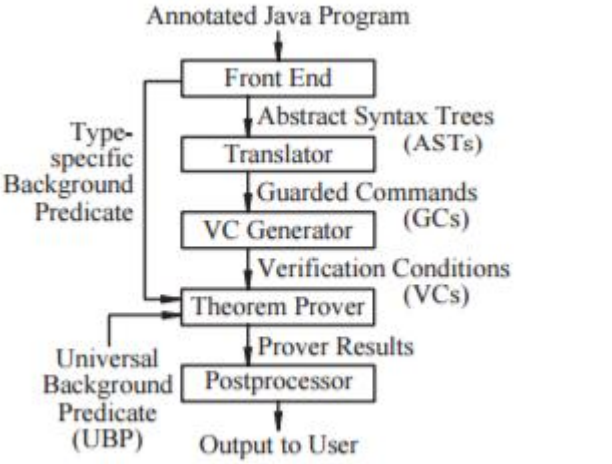
Runtime assertion checking with jmlc/jmlrac

- Special compiler inserts runtime tests for all JML assertions. Any assertion violation results in a special exception.
- checks specs at run-time
- only **tests** correctness of **specs**.
- **Find violations at runtime.**

***jmlc* and *jmlrac* – by example**

- Compile and Run
- Compile
- jmlc FileName.java
- Run
- jmlrac FileName listOfParam

- Demo 01: Factorial
- Demo02: Integer sqrt



- **Unsound ?**
- **Incomplete ?**

Tools for JML

Extended static checking with ESC/Java

- Automatically tries to prove simple JML assertions at compile time.
- checks specs at compile-time
- **proves** correctness of **specs**
- **Warn about likely runtime exceptions and violations.**

ESC/Java2 – by example

- Run
- escj FileName.java

- Demo 01: Fast exponentiation
- Demo 02: MyArray
- Demo 03: MySet

Program verification methods - Correctness

- Lecture 1 - Verification and Validation
  - Verification/Validation
    - reviews products to ensure their quality → correctness
    - static and dynamic analysis techniques
  - A **correct program** is one that does exactly what it is intended to do, no more and no less.
  - A formally correct program is one whose correctness can be proved mathematically.
    - This requires a language for specifying precisely what the program is intended to do.
    - Specification languages are based in mathematical logic.
  - Until recently, correctness has been an academic exercise. – Now it is a key element of critical software systems.

Correct-by-construction
Correctness by Construction How Can We Build Better Software, Date: May 31, 2023
SPEAKER Ina Schaefer, Professor of Software Engineering, Karlsruhe Institute of Technology (KIT), Germany
https://www.youtube.com/watch?v=5Nno9lSggPo

- **Program verification - correctness**
  1. proof-based, computer-assisted, program-verification approach, mainly used for programs which we expect to terminate and produce a result
  2. model-based, automatic, property-verification approach, mainly used for concurrent, reactive systems (originally used in a post-development stage) - model checking (Lecture 9)
  3. Developing correct algorithms from specification (Carroll Morgan, "Programming from Specification)

     Correctness-by-Construction.

     Originally intended as a mere means of programming algorithms that are correct by construction - -Dijkstra (1968), Hoare (1971),

     the approach found its way into commercial development processes of complex systems - Hall (2002), Hall and Chapman (2002)

     2012, The Correctness-by-Construction Approach to Programming, Authors: **Kourie**, Derrick G., **Watson**, Bruce W.

     2015, Experience with correctness-by-construction, B.W. Watson a, D.G. Kourie b, L. Cleophas b,*

     2016, Correctness-by-Construction and Post-hoc Verification: Friends or Foes?, Maurice H. ter Beek1(B) , Reiner H¨ahnle2, and Ina Schaefer3

     2023, Automated Software Engineering Conference, The 5th International Workshop on Automated and verifiable Software sYstem DEvelopment (ASYDE)

     Topic: Correct-by-construction software development

     (https://conf.researchr.org/track/ase-2023/ase-2023--workshop--asyde#the-5th-international-workshop-on-automated-and-verifiable-software-system-development-aside)

- **Correctness Tools**
  - Theorem provers (PVS), Modeling languages (UML and OCL), Specification languages (JML), Programming language support (Eiffel, Java, Spark/Ada), Specification Methodology (Design by contract)
- **Methods for prooving program correctness**
  - Floyd's Method - Inductive assertions
  - Hoare - Semantics of Hoare triples
  - Dijkstra's Language- Guarded commands, Nondeterminacy and Formal Derivation of Programs

Floyd, Dijkstra, Hoare (25XP)

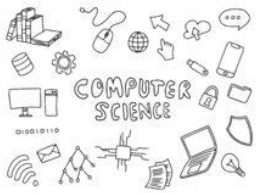
- **Robert Floyd** OR **Edsger Wybe Dijkstra** OR **Charles Antony Richard Hoare**

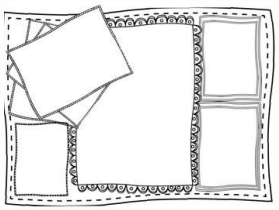
- 1 page A4 information (electronic format and printed format)
  - short bio
  - profession
  - Institution
  - known by...
  - awards
  - interesting facts

- Feel free to select a format: plain text, mindmap, other
- Delivery:
  - Today, after 10 minutes
  - 3-4 students per team

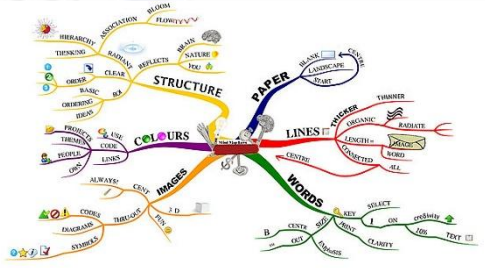
In-class

Doodle map





Maindmap



Next Lecture

Lecture attendance: 25XP

- Altom Invited Lecture

Challenges with Automation for Games

Presenters:

Rotaru Alexandru

Tuesday

6 May 2025, 8-10 am

Room: 6/II (Main Building)

Outline

- Correctness

- Floyd's Method -Inductive assertions, Partial correctness, Termination
- Hoare Logic, Semantics of Hoare triples, Partial correctness, Total correctness
- Dijkstra's Language, Guarded commands, Nondeterminacy, Formal Derivation of Programs

- Developing correct programs from specification, Refinement, Rules of Refinement, Examples

- Static analysis, JML- Java Modeling Language, ESC/Java2- Extended Static Checker for Java

- Questions

References

- [Flo67] Robert W. Floyd, Assigning meanings to programs, In *Proceedings of Symposia in Applied Mathematics*, pages 19–32, 1967.
- [Hoa69] C. A. R. Hoare, An axiomatic basis for computer programming, *Commun. ACM*, 12(10):576–580, 1969.
- [Dij75] E. Dijkstra, Guarded commands, nondeterminacy and formal derivation of programs.
- CACM, 8(18):453–457, 1975.
- [Fre10] M. Frentiu, Verificarea si validarea sistemelor soft, Presa Universitara Clujeana, 2010.
- [Mor98] C. Morgan, Programming from Specifications, Prentice Hall International Series in Computing Science, 1998

Software Systems Verification and Validation

"Tell me and I forget, teach me and I may remember, involve me and I learn."

(Benjamin Franklin)

Developing correct algorithms from specification

Example 1

Integer division (quotient and remainder)

Specification

φ: $(x \geq 0) \wedge (y > 0)$

ψ: $(x = q*y+r) \wedge (0 \leq r < y)$

A_0:	Subalgorithm IntegerDivision(x,y,q,r) is: $[\varphi, \psi]$ endIntegerDivision

Let η: $(x = q*y+r) \wedge (0 \leq r)$ a middle predicate and we can use Sequential rule:

A_1:	Subalgorithm IntegerDivision (x,y,q,r) is: $[\varphi, \eta]$ $[\eta, \psi]$ endIntegerDivision

The predicate η becomes true if we do the assignment (q,r):= (0,x).

A_2:	Subalgorithm IntegerDivision (x,y,q,r) is: (q,r)←(0,x) [η, η and r<y] endIntegerDivision

The predicate η is an invariant predicate and we can apply the iteration rule:

A_3:	Subalgorithm IntegerDivision (x,y,q,r) is: (q,r) ← (0,x) DO r≥y -> [r≥y and η, η and TC] OD endIntegerDivision

For the DO to terminate we must decrease r, and because r≥y, we can use r←r-y.

But η should hold also in the post-condition, so we must have:

q*y+r = q*y+r-y +y=(q+1)*y + (r-y).

So, both r and q must be changed.

A_4:	Subalgorithm IntegerDivision (x,y,q,r) is: (q,r) ← (0,x) DO r≥y -> (q,r) ← (q+1,r-y) OD endIntegerDivision

Example 2

Square root. r = [sqrt(n)]

But we know that: $r \leq sqrt(n) < r+1$

⇨ The output predicate is $r^2 \leq n < (r+1)^2$

Specification:

φ: n>1

ψ: $r^2 \leq n < (r+1)^2$

A_0

Subalgorithm RADICALINT(n,r) is:
[φ, ψ]
endSquare

We rewrite the output predicate: $(r^2 <= n < q^2) \wedge (q=r+1)$

We use the middle predicate: $\eta ::= (r^2 \leq n < q^2)$

A_1

Subalgorithm Square(n,r) is:
[φ, η]
[η,ψ]
endSquare

The predicate η becomes True (in the first abstract program) for r=0 şi q=n+1.

A_2

Subalgorithm Square(n,r) is:
(q,r) ← (n+1,0)
[η,η ∧ (q=r+1)]
{ψ}
endSquare

For the second abstract program we can apply the iteration rule.

A_3

Subalgorithm Square (n,r) is:
(q,r) ← (n+1,0)
DO q≠r+1 ->
[η∧ q≠r+1,η ∧ TC]
OD
endSquare

For the DO to terminate we must decrease r or q (loop condition, q-r must become 1 at the end). The value of p=(q+r)/2 satisfies r<p<q, and (q-r) is changed by changing the interval [r,q] to [r,p] or [p,q].

But η should hold also in the post-condition, so we must have:

If $(p^2 \leq n)$ then the assignment r ← p keeps η invariant.

If $(p^2 > n)$ then the assignment q ← p keepsη invariant.

A_4

Subalgorithm Square (n,r) is:
(q,r) ← (n+1,0)
DO q>r+1 ->
p← (q+r)/2
If $p^2 \leq n$ → r←p
□$p^2 < n$→ q←p

fi
OD
endSquare

Example 3

Multiplication by repeated additions

Specification:

φ : $(x \geq 0) \wedge (y \geq 0)$

ψ : $z = x*y$

A_0

Subalgorithm Product(x,y,z) is:
[φ, ψ]
endProduct

The postcondition ψ may be satisfied if we use the following predicate:

$\eta ::= (z+u*v = x*y) \wedge (v \geq 0)$

We use this predicate as a middle predicate and the sequential composition rule:

A_1

Subalgorithm Product (x,y,z) is:
[φ, η]
[η,ψ]
endProduct

The first abstract program becomes true with the assignment $(u,v,z) \leftarrow (x,y,0)$

A_2

Subalgorithm Product (x,y,z) is:
$(z,u,v) \leftarrow (0,x,y)$
[η,ψ]
endProduct

The abstract program [η,ψ] may be rewritten as [η,$\eta \wedge (v=0)$] and now we can apply the iteration rule:

A_3

```
Subalgorithm Product (x,y,z) is:
    (z,u,v) ← (0,x,y)
    DO v≠0 ->
       [η∧v≠0,η ∧ TC]
    OD
endProduct
```

For the DO to terminate we must decrease v:

First possibility: v←v-1. But η should hold also in the post-condition, so we must have: $z+u*v = z + u + u*(v-1)$. So also the assignment z←z+u is needed.

Second possibility: v←v/2, if v is even. . But η should hold also in the post-condition, so we must have: $z+u*v = z + (u*2)* v/2$So also the assignment (u,v):=(u+u, v/2) is needed.

A_4

```
Subalgorithm Product (x,y,z) is:
    (z,u,v) ← (0,x,y)
    DO v>0}
```

```
            Do  (v even) →
                    (u,v) ← (u+u,v div 2)
            OD
            (z,v) ← ( z+u, v-1)
        OD
endProduct
```

Example 4

Raising a number to a power by multiplications

Compute $z = x^y$ by multiple multiplications

Specification:

A0:	φ : $(x>0) \wedge (y\geq 0)$ ψ : $z = x^y$

The predicate $\eta ::= (z*u^v = x^y) \wedge (v\geq 0)$ implies ψ if v=0. Using it as a middle predicate we can apply the sequential composition rule:

A1:	Subalgorithm RaisingPower(x,y,z) is: $[\varphi , \eta]$ $[\eta , \psi]$ endRaisingPower

The η becomes true if (z,u,v) = (1,x,y) (in the first abstract program):

A2:	Subalgorithm RaisingPower(x,y,z) is: (z,u,v) ← (1,x,y) [η,$\eta \wedge$ v=0] endRaisingPower

The predicate η is invariant, we can apply the iteration rule.

A3:	Subalgorithm Putere(x,y,z) is: (z,u,v) ← (1,x,y) DO v≠0 → [$\eta \wedge$v≠0,$\eta \wedge$ TC] OD endRaisingPower

For the DO to terminate we must decrease v:

First possibility: v←v-1. But η should hold also in the post-condition, so we must have: $z*u^v = z * u*u^{v-1}$. So also the assignment (z,v)←(z*u,v-1) is needed.

Second possibility: v←v/2, if v is even. . But η should hold also in the post-condition, so we must have: $z*u^v = z* (u*u)^{v/2}$ .So also the assignment (u,v) ← (u*u,v/2) is needed.

A4	Subalgorithm RaisingPower1 (x,y,z) is: (z,u,v) ← (1,x,y) DO v≠0 (z,v) ← (z*u,v-1)

	OD endRaisingPower

A4	Subalgorithm RaisingPower2 (x,y,z) is: (z,u,v) ← (1,x,y) DO v≠0 DO (v even) →(u,v) ← (u*u,v/2) OD (z,v) ← (z*u,v-1) OD endRaisingPower

Example 5

Greatest common divisor

Specification:

φ : x>0, y > 0

ψ : d=gcd(x,y)

A_0

Subalgorithm GCD(x,y,d) is:
[φ, ψ]
endGCD

The middle predicate η::= gcd(d,s)=gcd(x,y) is used for the sequential composition rule.

A_1

Subalgorithm GCD(x,y,d) is:
[φ, η]
[η,ψ]
endGCD

The first abstract program becomes true if (d,s)=(x,y), so we use assignement rule:

A_2

Subalgorithm GCD(x,y,d) is:
(d,s) ← (x,y)
[η,ψ]
endGCD

If d=s then η implies ψ. So, we can write:

A_3

Subalgorithm GCD(x,y,d) is:
(d,s) ← (x,y)
[η,η ∧ (d=s)]
endGCD

We can apply now the iteration rule:

A_2

Subalgorithm GCD(x,y,d) is:
(d,s) ← (x,y)
DO d≠s →
[η∧d≠s,η ∧ TC]
OD
endGCD

For d≠s we have d>s or d<s. We know that for d>s we have gcd(d,s)=gcd(d-s,s) and the assignment d ←d-s keepsη invariant.

A_3

```
Subalgorithm GCD (x,y,d) este:
        (d,s) ← (x,y)
        DO d≠s →
                IF d>s  → d←d-s
                  □  d<s → s←s-d
                FI
        OD
        gcd←s
endGCD
```

Example 6

Insertion

A = (a_1, a_2,...,a_n) an array with n components ordered in decrease order and x a value. Insert x in A such that A remains ordered and A containes a new value x.

The predicate ORD is define by:

ORD(n,A) ::= ($\forall i,j: 1 \le i,j \le n,\ i \le j \Rightarrow a_i \le a_j$)

Specification:

φ::= ORD(n,A) ∧ (n natural)

ψ ::=ORD(n+1,A) and (A contains the initial elements and a new element x)

A_0:	[φ, ψ]

There are two possibilities ($x<a_n$ and $n\neq0$) or ($x\ge a_n$ or (n=0)):

A_1:	Subalgorithm Insert(n,A,x) is: Dacă $x<a_n$ şi $n\neq0$ atunci [φ ∧ ($x<a_n$) ∧ $n\neq0$, ψ] altfel [φ ∧ (($x\ge a_n$)∨(n=0)), ψ] sfdacă endInsert

A doua propoziţie nestandard se rafinează printr-o atribuire

A_2:	Subalgorithm Insert(n,A,x) is: Dacă $x<a_n$ şi $n\neq0$ atunci [φ ∧ ($x<a_n$) ∧ $n\neq0$, ψ] altfel $(n,a_{n+1}) \leftarrow (n+1,x)$ sfdacă endInsert

Să notăm prin η următorul predicat

$$ORD(n,A) \wedge [(x<a_1) \wedge (p=1) \vee (a_{p-1} \le x < a_p) \wedge (1<p \le n)]$$

Care este o postconditie pentru o problemă de căutare şi să folosim regula secvenţei. Ajungem la:

A_3:	Subalgorithm Insert(n,A,x) is: IF $x<a_n$ and $n\neq 0$ → $[\varphi \wedge (x<a_n) \wedge n\neq 0, \eta]$ $[\eta , \psi]$ □ (not $x<a_n$ and $n\neq 0$) →$(n,a_{n+1}) \leftarrow (n+1,x)$ FI endInsert

Vom satisface postcondiţia η în urma apelului subalgoritmului de căutare, astfel că ajungem la:

A_4:	Subalgorithm Insert(n,A,x) is: IF $x<a_n$ and $n\neq 0$ → CALL SEARCH(x,n,A,p) $[\eta , \psi]$ □ (not $x<a_n$ and $n\neq 0$) →$(n,a_{n+1}) \leftarrow (n+1,x)$ FI endInsert

After the search we know that x is between a_{p-1} and a_p , so x must be inserted on position p, so we have

$a'_{i+1} \leftarrow a_i$, for i=n,n-1,...,p

and $a'_p \leftarrow x$.

n'←n+1

We use the assignments:

i←n;

DO i≥p →

$a_{i+1} \leftarrow a_i$

i←i-1

OD

A_5:	Subalgorithm Insert(n,A,x) is: IF $x<a_n$ and $n\neq 0$ → CALL SEARCH(x,n,A,p) i←n DO i≥p → $a_{i+1}\leftarrow a_i$ i←i-1 OD $a_p\leftarrow x$ n←n+1 □ (not $x<a_n$ and $n\neq 0$) →$(n,a_{n+1}) \leftarrow (n+1,x)$ FI endInsert

Another refinement regardi8nt the n←n+1 assignment:

A_5:	Subalgorithm Insert(n,A,x) is: IF $x<a_n$ and $n\neq 0$ → CALL SEARCH(x,n,A,p) i←n DO i≥p → $a_{i+1}\leftarrow a_i$ i←i-1 OD $a_p\leftarrow x$ □ (not $x<a_n$ and $n\neq 0$) →$(n,a_{n+1}) \leftarrow (n+1,x)$ FI n←n+1 endInsert

Example 7

InsertionSort

Let A = (a_1, a_2,...,a_n) be an array with n integer components. The problem requires to order the components of A.

Specification

φ::= n≥2, A has integer components

ψ ::= ORD(n,A) and A has the same elements as in the precondition

A_0:	[φ, ψ]

We use the middle predicate ORD(k,A) and apply the sequential composition rule:

A_1:	Subalgorithm InsertSort(n,A) is: [φ, ORD(k,A)] [ORD(k,A), ψ] endInsertSort

The first abstract program may be refined to an assignment

A_2:	Subalgorithm InsertSort(n,A) is: k←1 [ORD(k,A), ψ] endInsertSort

Wer can rewrite the remained abstract program remarking that

$$ORD(k,A) \wedge (k=n) \Rightarrow \psi$$

A_3:	Subalgorithm InsertSort(n,A) is: k←1 [ORD(k,A), ORD(k, A) şi (n=k)] endInsertSort

We now can apply the iteration rule

A_4:	Subalgorithm InsertSort(n,A) is: k←1 DO k<n → [ORD(k,A) and k<n, ORD(k,A) and TC] OD endInsertSort

For the DO to terminate we must increase k:
First possibility: k←k+1. But η(k)::=ORD(k,A) invariant – by modifying k by k+1 the predicate η(k|k+1) must be true.

A_5:	Subalgorithm InsertSort(n,A) is: k←1 DO k<n → [k<n and η(k), η(k|k+1)] OD endInsertSort

The abstract program

$$[\ (k<n) \wedge\ ORD(k,A), ORD(k+1,A)]$$

Corresponds to the following subproblem:

If ORD(k,A) (the first k elements in A are orderes) then modify the A such that the first k+1 elements to be ordered. This can be achieve by calling a subalgorithn that inserts the a_{k+1} component such that after insertion the postcondition ORD(k+1,A) is true.

A_4:	Subalgorithm InsertSort(n,A) is: k←1 DO k<n → CALL INSERT(k,A, a_{k+1}) OD endInsertSort

Challenges with Automation for Games

Altom

Nice to meet you!

Altom

What do we do?

- We are a company offering testing services
- We teach through our BBST courses
- We are building tools that help with test automation (AltTester®, AltWalker, AltTap)

Altom

How about you?

- How familiar are you with game development?
- Have you implemented apps using Unity or other engines?
- What challenges did you face while testing the app?

Game testing world

Game World

- Game Engines (like Unity, Unreal, Custom Engines)
    - develop once, export to many platforms
    - a lot is common, but each platform has its own quirks

Traditional software development
vs.
Game software development

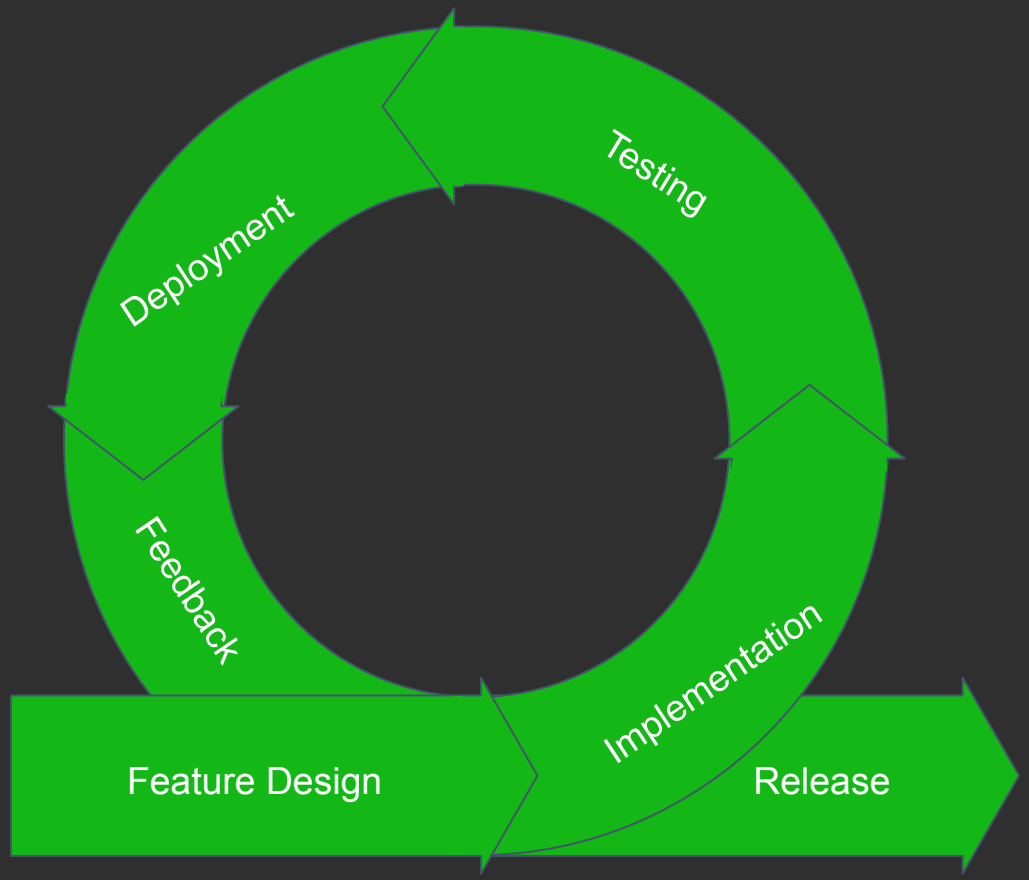
Traditional / Agile Software Development

- Every change in the code is tested
  - functional tests as automated scripts
  - give continuous feedback on how other parts of the software are affected by the change

Game Software Development

- QA team performs functional testing once features are completed
- Playtesting, Alpha testing, Beta testing

Traditional / Agile Software Development



Altom

Challenges with non-automated approach

- Games are generally complex and testing all functionalities manually becomes too time consuming
- Games are frequently updated and you have to retest the same thing all over again
- You have to test the same functionalities on multiple platforms
- Make sure nothing broke from the existing features

Options for Game Test Automation

Many programming languages

Supports iOS and Android,
Mac, Windows

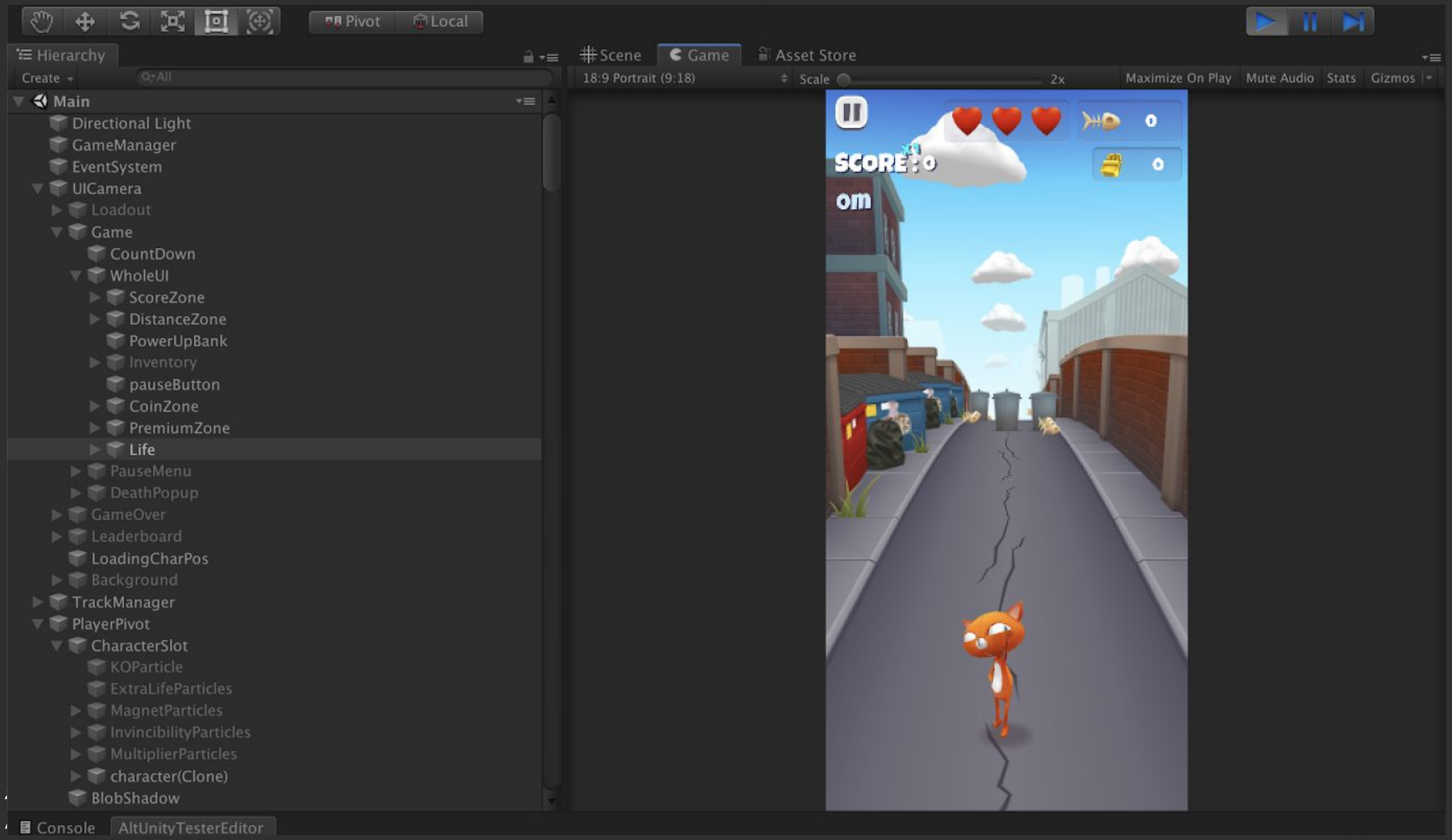
Offers great interaction with
the target platform

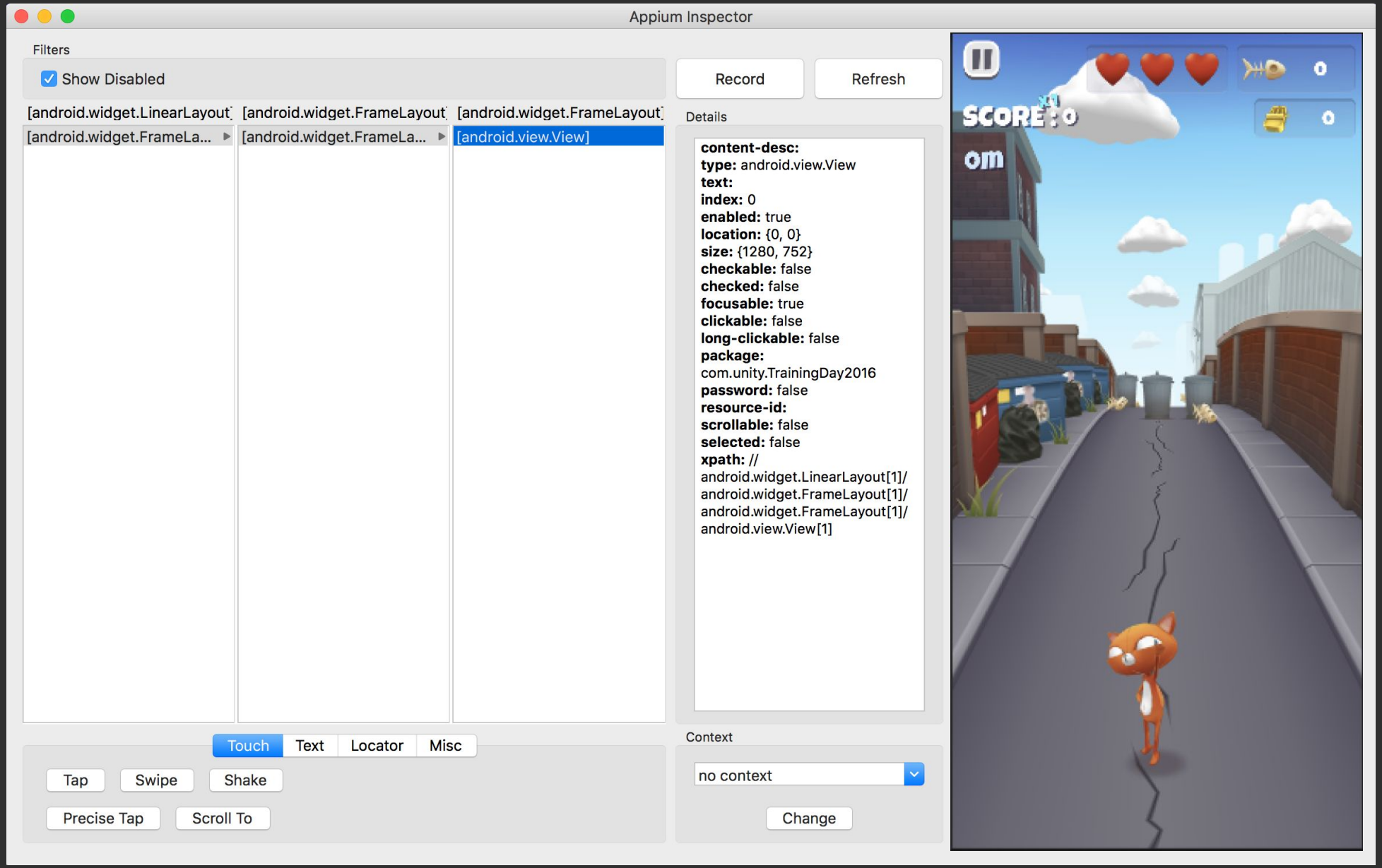
APPIUM

Testers who are familiar with
Selenium find it easy

Works with Browserstack, Bitbar Cloud,
AWS and other Device Cloud platforms

Altom


Pivot
Local
Hierarchy
Create
Main
Directional Light
GameManager
EventSystem
UICamera
Loadout
Game
CountDown
WholeUI
ScoreZone
DistanceZone
PowerUpBank
Inventory
pauseButton
CoinZone
PremiumZone
Life
PauseMenu
DeathPopup
GameOver
Leaderboard
LoadingCharPos
Background
TrackManager
PlayerPivot
CharacterSlot
KOParticle
ExtraLifeParticles
MagnetParticles
InvincibilityParticles
MultiplierParticles
character(Clone)
BlobShadow
Scene
Game
Asset Store
18:9 Portrait (9:18)
Scale
2x
Maximize On Play
Mute Audio
Stats
Gizmos
SCORE: 0
0
0
Console
AltUnityTesterEditor


Appium Inspector
Filters
Show Disabled
[android.widget.LinearLayout
[android.widget.FrameLayout
[android.widget.FrameLayout
[android.widget.FrameLa...
[android.widget.FrameLa...
[android.view.View]
Record
Refresh
Details
content-desc:
type: android.view.View
text:
index: 0
enabled: true
location: {0, 0}
size: {1280, 752}
checkable: false
checked: false
focusable: true
clickable: false
long-clickable: false
package:
com.unity.TrainingDay2016
password: false
resource-id:
scrollable: false
selected: false
xpath: //
android.widget.LinearLayout[1]/
android.widget.FrameLayout[1]/
android.widget.FrameLayout[1]/
android.view.View[1]
Touch
Text
Locator
Misc
Tap
Swipe
Shake
Precise Tap
Scroll To
Context
no context
Change
SCORE: 0
0
0

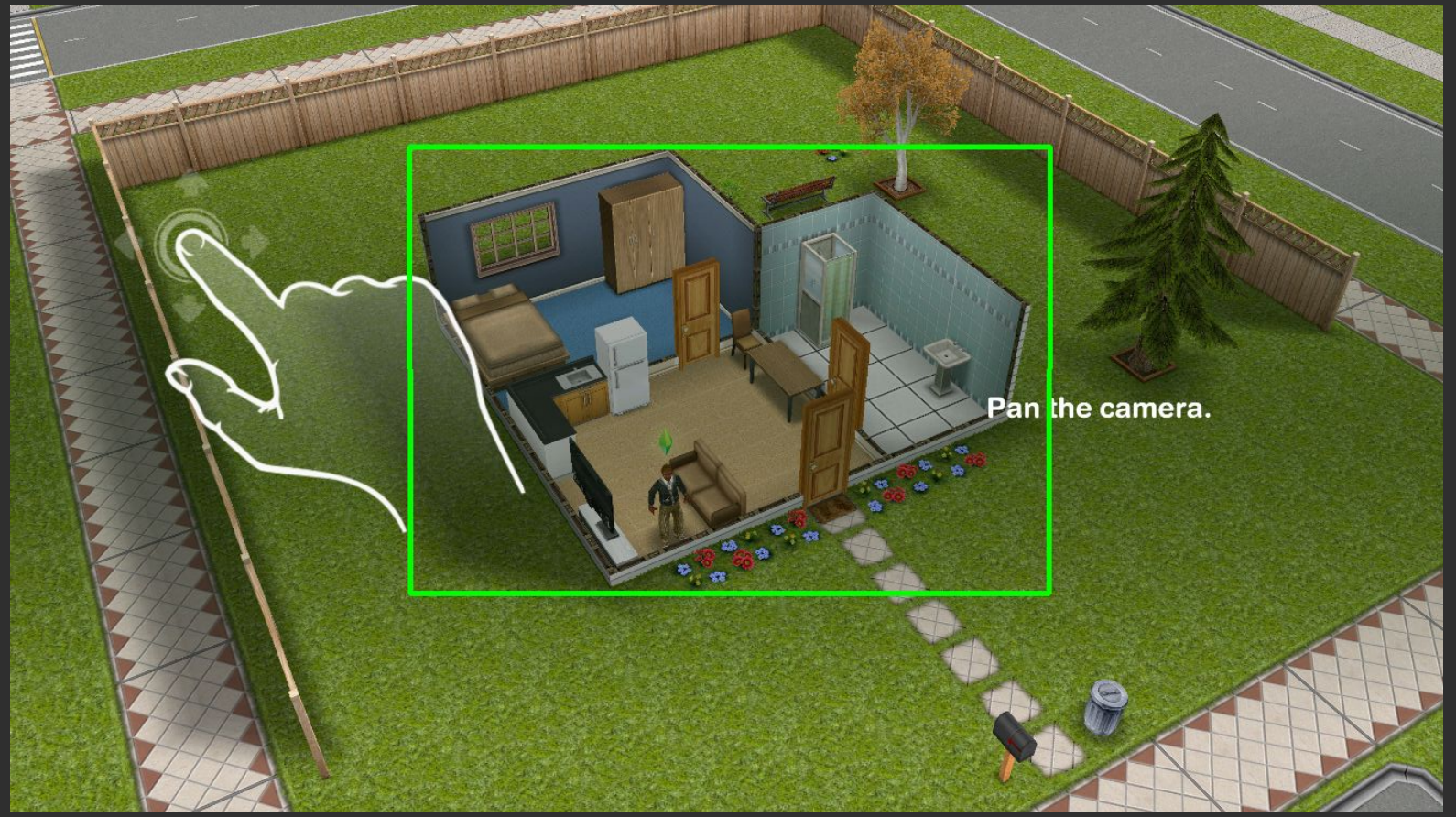
Altom

~~Click at coordinates~~

Altom

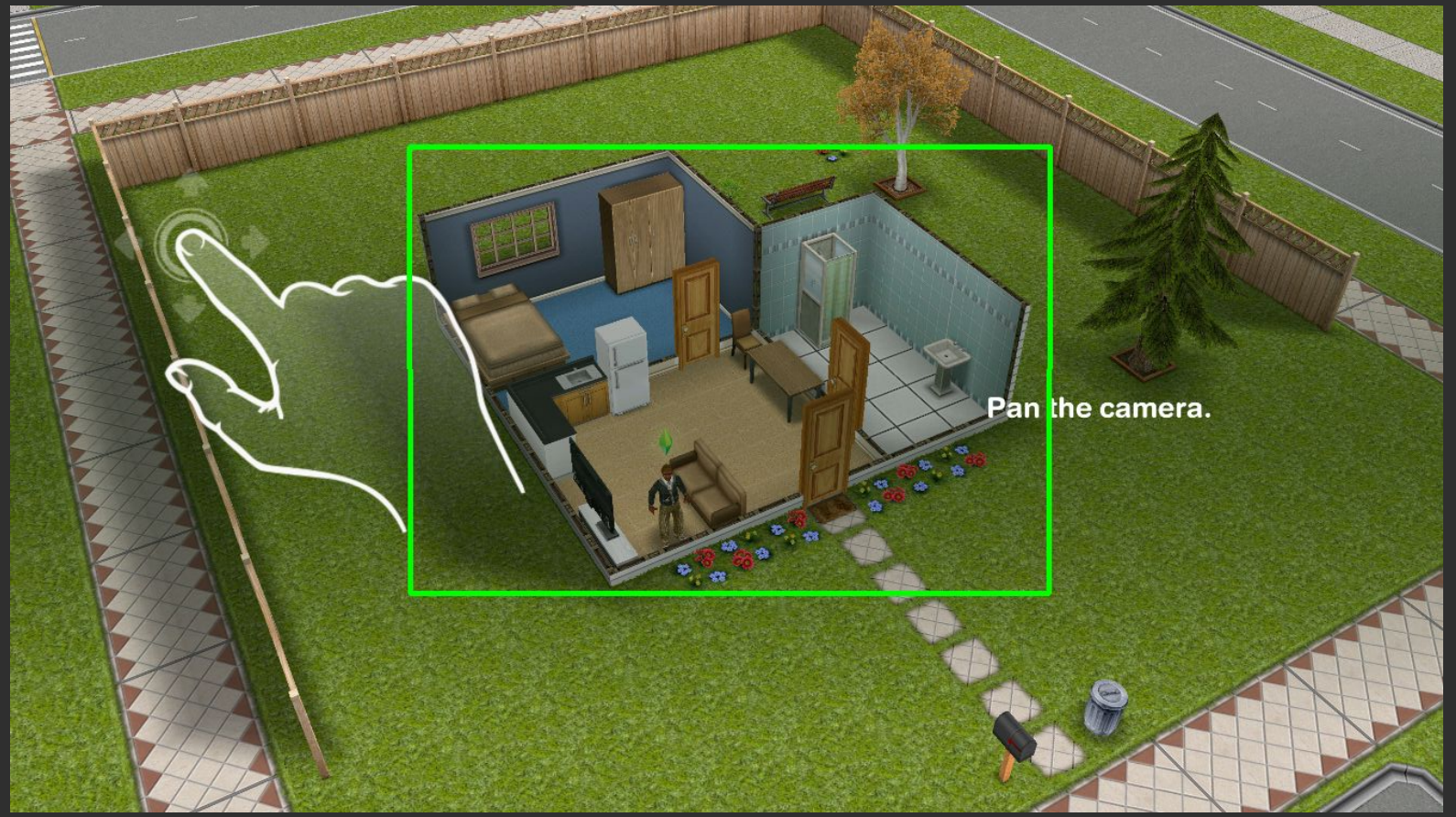
Image Recognition

Define “House” element using an image



Altom

Define “House” element using an image



Altom

OpenCV - Feature detection and matching

Altom

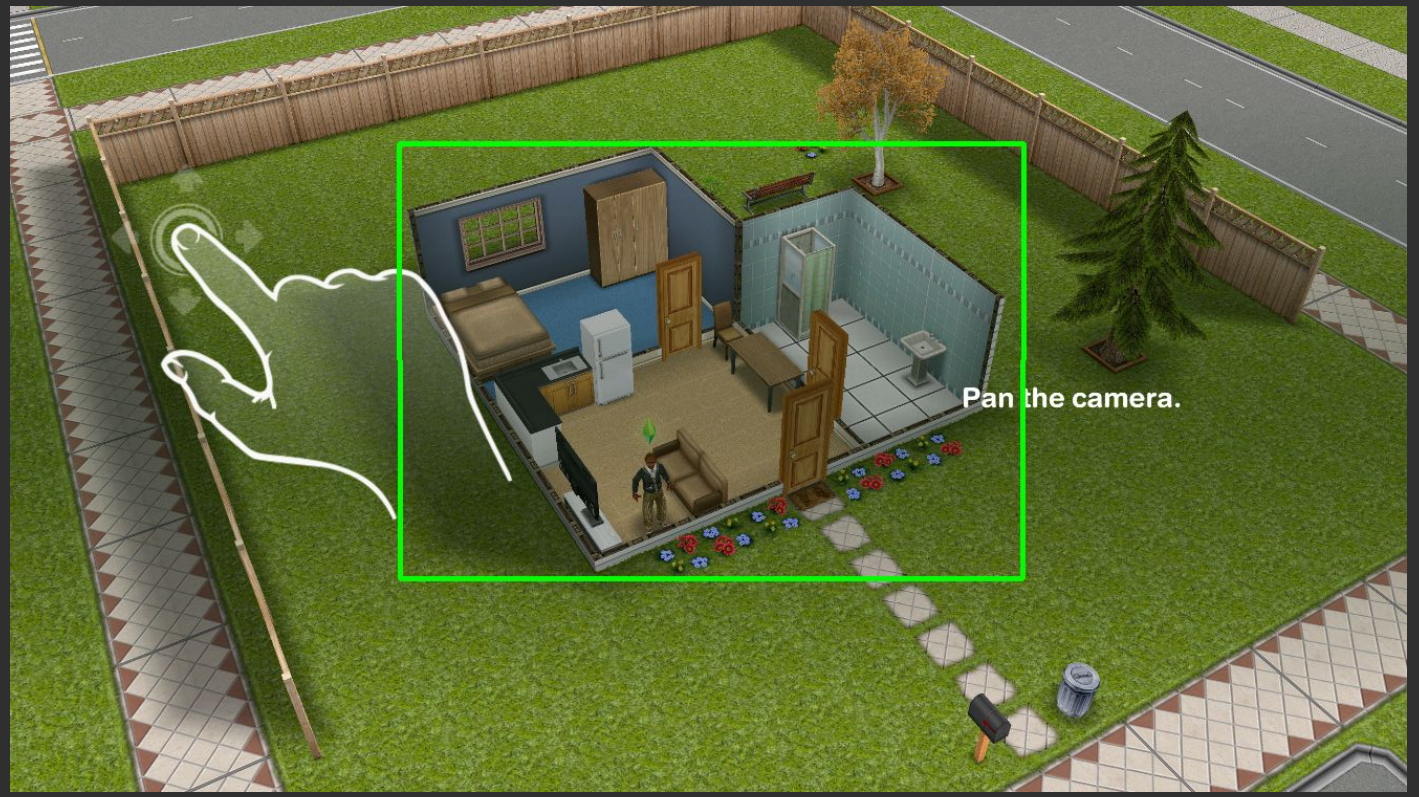
OpenCV - Feature detection and matching

Altom

OpenCV - Feature detection and matching

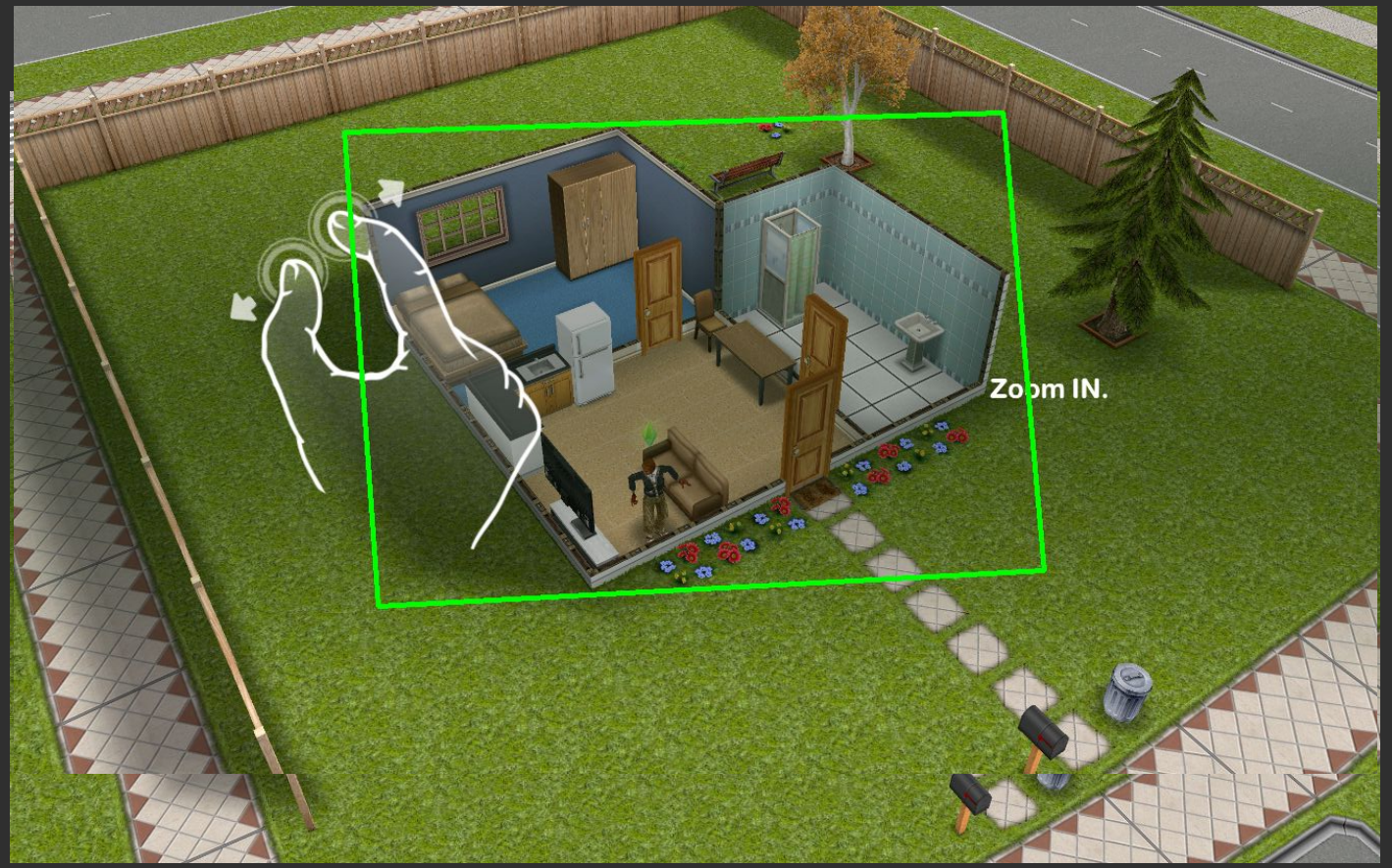
Altom

Resolution Agnostic, works rotated and stretched items



Altom

Resolution Agnostic, works rotated and stretched items



Altom

Resolution Agnostic, works rotated and stretched items

Altom

Resolution Agnostic, works rotated and stretched items

Altom

Image Recognition + Appium

https://github.com/bitbar/testdroid-samples/tree/master/image-recognition

- worked well for simple scenarios
- didn’t require any changes to the game
- found really useful issues
  - out of memory problems
  - crashes
  - graphics missing/not displayed correctly

Image Recognition + Appium

https://github.com/bitbar/testdroid-samples/tree/master/image-recognition

- was not fast enough
- graphics change all the time in games
- didn't work well for apps that had a lot of text
- didn’t give enough granularity when it came to identifying objects

Struggles with Test Automation in Games

Struggles with UI Automation in Games

- Small teams => no dedicated testers, developers don’t have time to invest in automated tests
- Large teams => large teams of “manual” testers with not enough coding skills; manual testing takes a lot of time

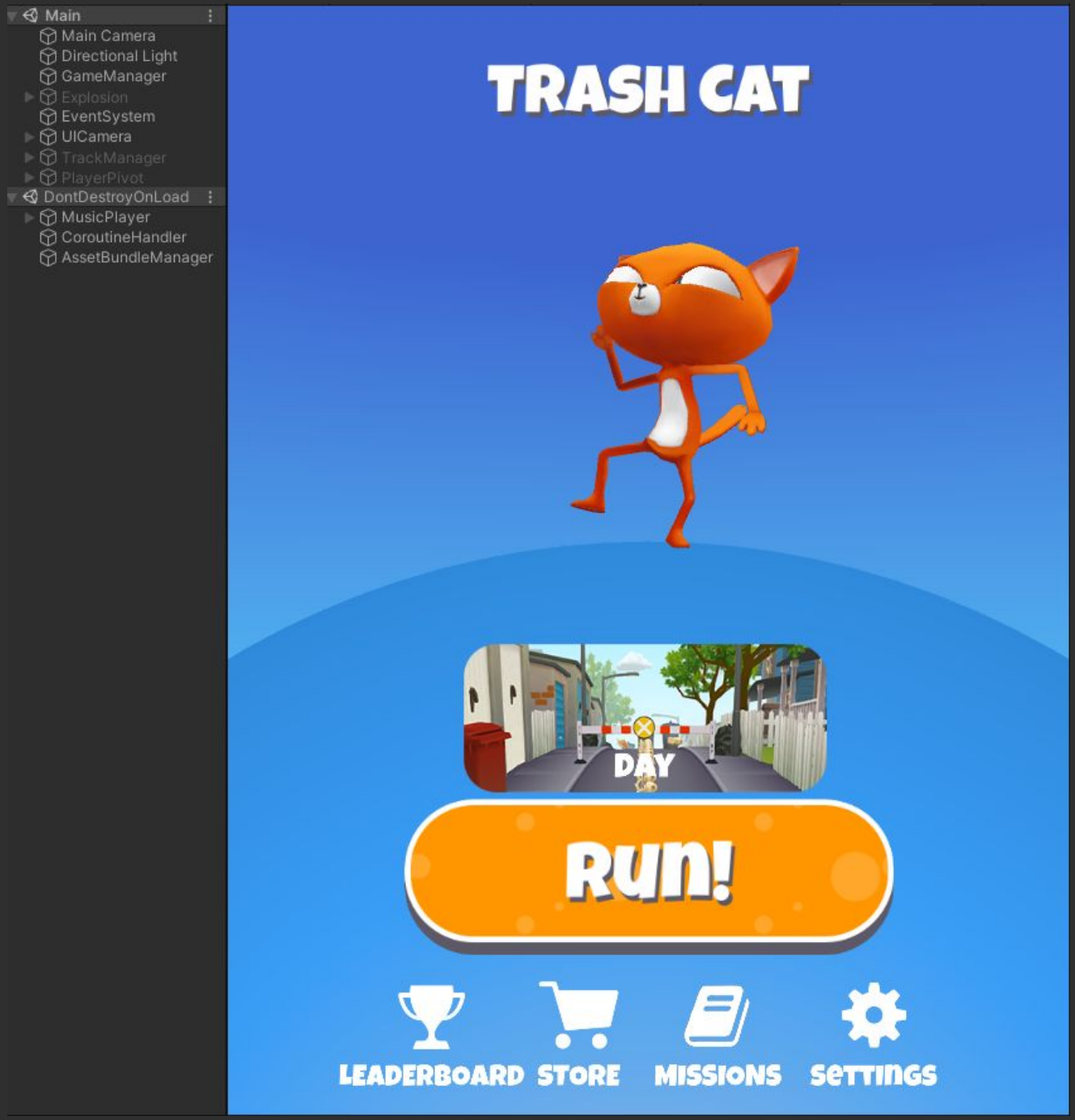
Struggles with Test Automation in Games

- Make the tools more accessible to testers that have no coding skills
- Cover multiple engines (Unity, Unreal, Godot, o3de)
- Cover multiple platforms
- Keep the development engagement

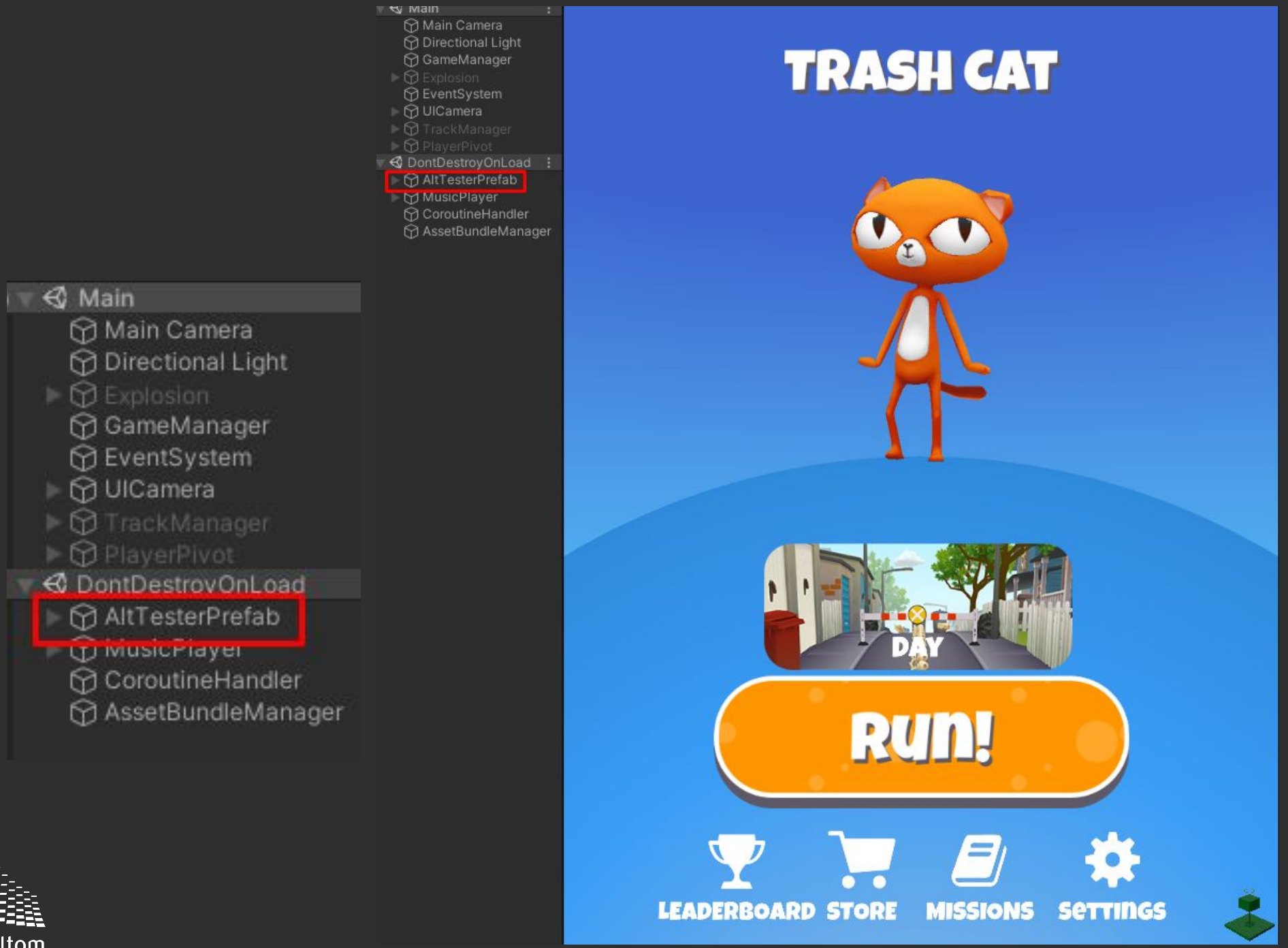
Beyond Image Recognition

AltTester®

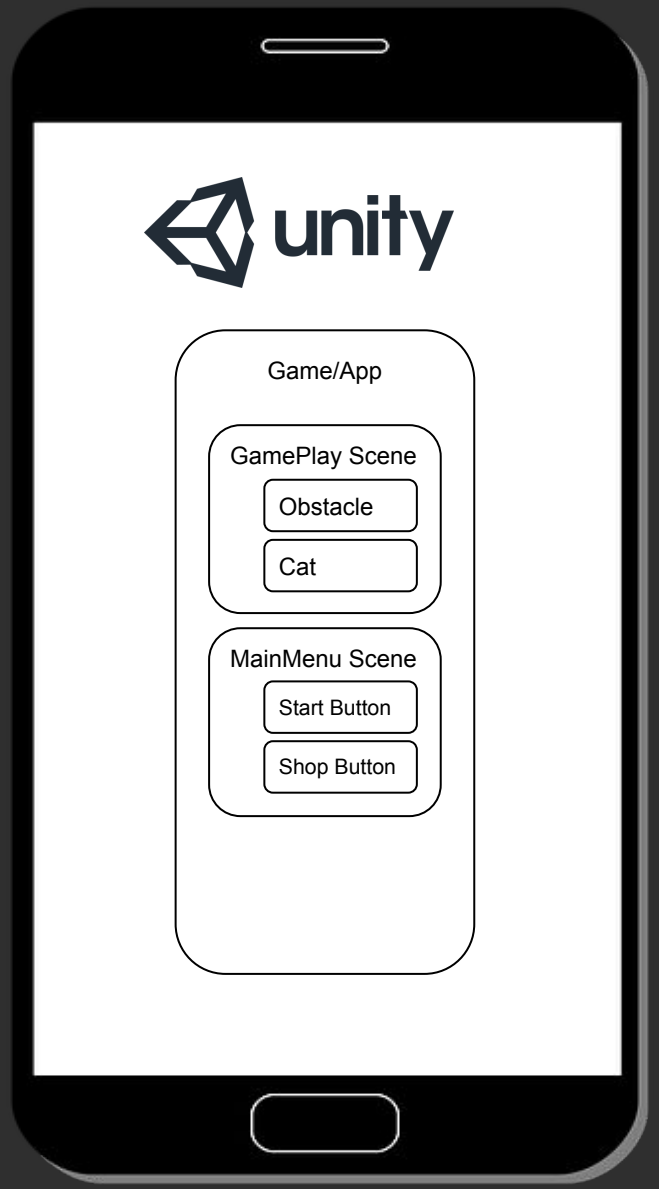
Altom


Main
Main Camera
Directional Light
GameManager
Explosion
EventSystem
UICamera
TrackManager
PlayerPivot
DontDestroyOnLoad
MusicPlayer
CoroutineHandler
AssetBundleManager
TRASH CAT
DAY
Run!
LEADERBOARD
STORE
MISSIONS
SETTINGS

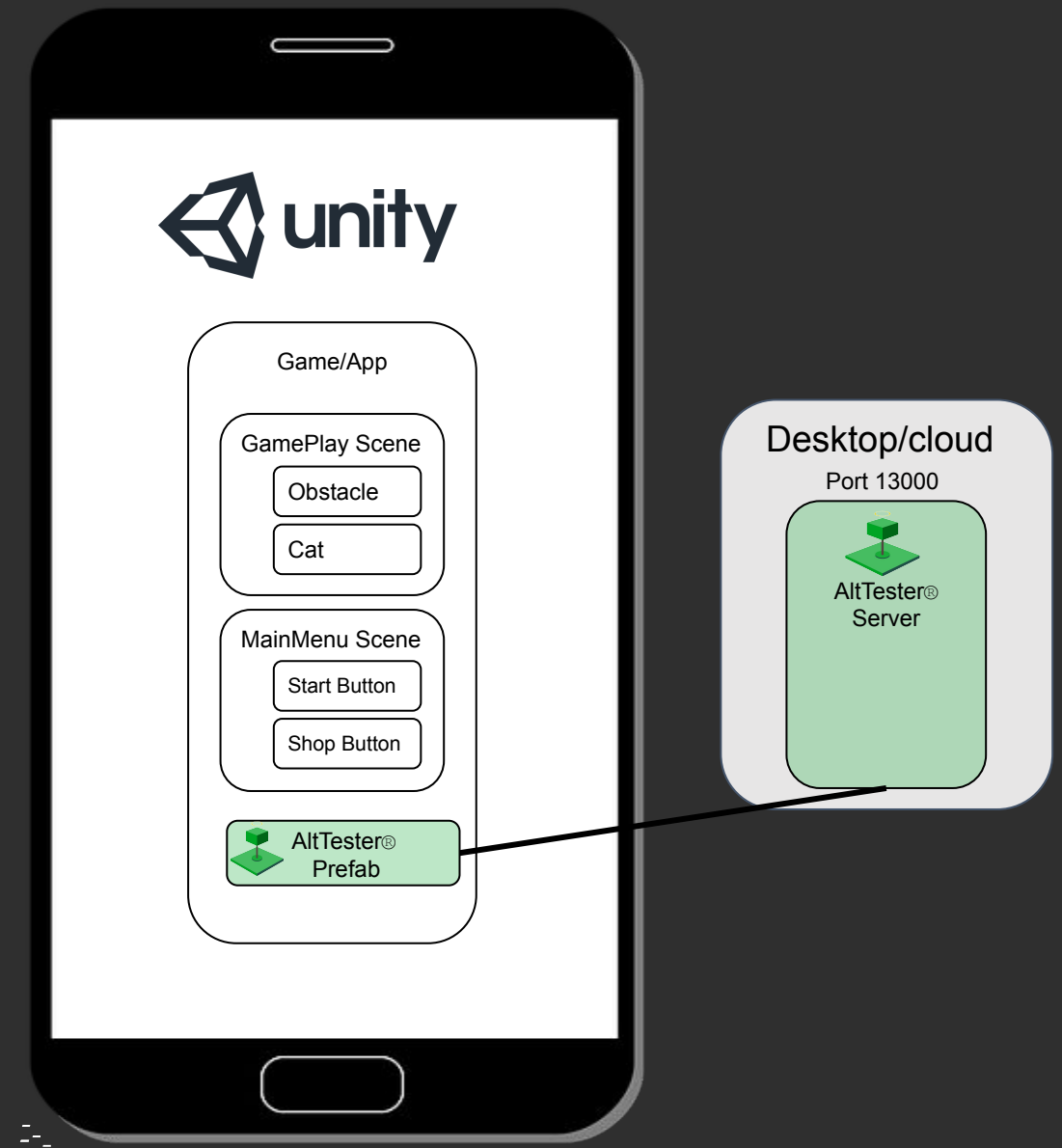
Altom


Main
Main Camera
Directional Light
Explosion
GameManager
EventSystem
UICamera
TrackManager
PlayerPivot
DontDestroyOnLoad
AltTesterPrefab
MusicPlayer
CoroutineHandler
AssetBundleManager
TRASH CAT
DAY
Run!
LEADERBOARD
STORE
MISSIONS
SETTINGS

Altom


unity
Game/App
GamePlay Scene
Obstacle
Cat
MainMenu Scene
Start Button
Shop Button

Altom

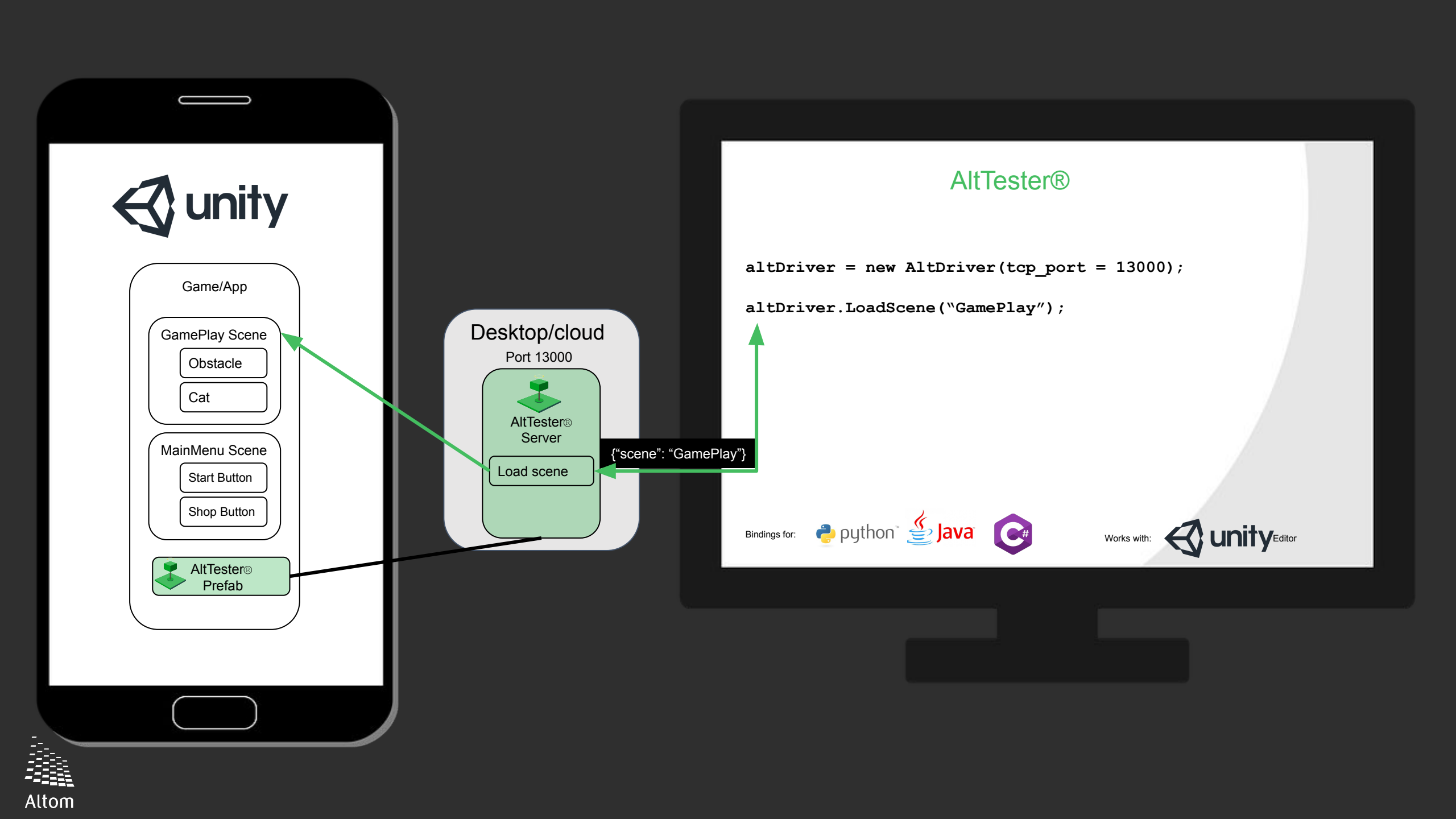

unity
Game/App
GamePlay Scene
Obstacle
Cat
MainMenu Scene
Start Button
Shop Button
AltTester®
Prefab

Desktop/cloud
Port 13000
AltTester®
Server

Altom

unity
Game/App
GamePlay Scene
Obstacle
Cat
MainMenu Scene
Start Button
Shop Button
AltTester®
Prefab
socket connection
Desktop/cloud
Port 13000
AltTester®
Server
AltTester®
altDriver = new AltDriver(tcp_port = 13000);
Bindings for:
python
Java
Works with:
unity Editor

Altom


unity
Game/App
GamePlay Scene
Obstacle
Cat
MainMenu Scene
Start Button
Shop Button
AltTester®
Prefab
Desktop/cloud
Port 13000
AltTester®
Server
Load scene
{“scene”: “GamePlay”}
AltTester®
altDriver = new AltDriver(tcp_port = 13000);
altDriver.LoadScene(“GamePlay”);
Bindings for:
python
Java
Works with:
unity Editor

Altom

unity
Game/App
GamePlay Scene
Obstacle
Cat
MainMenu Scene
Start Button
Shop Button
AltTester®
Prefab
Desktop/cloud
Port 13000
AltTester®
Server
Find “Cat”
{“el”: “cat”, “x”: 50, “y”: 60}
AltTester®
altDriver = new AltDriver(tcp_port = 13000);
altDriver.LoadScene(“GamePlay”);
var cat = altDriver.FindObject(By.NAME, “Cat”);
cat.Tap();
Bindings for:
python
Java
Works with:
unity Editor
Altom

AltTester®

https://github.com/alttester

- easier access to objects from the scene
- reduces the time taken for testing - automated tests give fast feedback
- concurrent test execution on multiple devices

AltTester®

- test execution in the cloud (AWS, BitBar, BrowserStack)
- easy to learn by non-technical people (Inspect, Record)
- people skilled in web testing or mobile testing (apps, not games) could learn faster

AltTester®

- worked well even for complex scenarios
- really fast and worked for all types of games
- tests are easier to maintain than with image recognition

Challenges implementing AltTester®

- A more technical person is needed to create the instrumented build. What are the consequences?
- Is click on coordinates reliable?
- Unity has multiple types of objects (objects from the scene, menus, UI toolkit).
- What if the game uses meshes and not objects?

Challenges implementing AltTester®

- Unity has an old and a new input system
- What if the game to be tested uses a custom input system?
- Recorder - what actions do we record?
- Running the same set of tests on 2 standalone builds on the same machine. What happens with the resources?

- Manual testing or automation?
  - ex1. : VuCity - add floors to a newly created building. Has the building the expected height?
  - ex2. : test fonts

Keep the conversation going

Thank you!